

Block 1: Artificial Intelligence

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 1

Introduction to Artificial Intelligence

Knowledge-ഉം intelligence-ഉം തമ്മിലുള്ള വ്യത്യാസത്തിൽ തുടങ്ങി, AI എന്താണ്, അതിന്റെ types, ചരിത്രം, intelligence-ന്റെ components, AI-യെ അളക്കാനുള്ള നാല് approaches (Turing test ഉൾപ്പെടെ), AI / ML / DL വ്യത്യാസം, applications, ഒടുവിൽ intelligent agents-ഉം PEAS-ഉം ഈ unit-ൽ പഠിക്കുന്നു.

Knowledge vs intelligence

What is AI? Definitions and the four goals

AI technologies and levels of AI

Types of AI: ANI, AGI, ASI

Brief history of AI

Components of intelligence: learning

Learning from feedback: supervised, unsupervised, reinforcement

Reasoning, problem-solving, perception, language

Approaches to AI

The Turing test and the Chinese Room

AI vs ML vs DL

Descriptive, predictive and prescriptive analytics

Application areas of AI

Intelligent agents: definition and properties

Task environment and PEAS

Agent structure; simple reflex and model-based agents

Goal-based, utility-based and stimulus-response agents

Learning agents

Knowledge vs intelligence

Knowledge: the information and abilities a person accumulates through learning and life experience.

Intelligence: the capacity to comprehend, analyse and apply knowledge, especially in new situations.

Knowledge: പഠനത്തിലൂടെയും ജീവിതാനുഭവത്തിലൂടെയും ഒരാൾ നേടുന്ന information-ഉം abilities-ഉം.

Intelligence: knowledge മനസ്സിലാക്കാനും analyse ചെയ്യാനും, പ്രത്യേകിച്ച് പുതിയ സാഹചര്യങ്ങളിൽ, apply ചെയ്യാനുള്ള കഴിവ്.

Knowledge is the accumulation of information and abilities a person gains through life experience; it keeps growing with time. **Intelligence** is the capacity to put that knowledge into practice: to comprehend, analyse and use information. Having information does not automatically make one intelligent. The two are linked: knowledge lets us learn how others understand things, while intelligence lets us grasp the things themselves.

മലയാളത്തിൽ

Knowledge എന്നാൽ നമ്മൾ വർഷങ്ങളായി ശേഖരിച്ച information ആണ്; intelligence എന്നാൽ അത് ശരിയായ സമയത്ത് ശരിയായി ഉപയോഗിക്കാനുള്ള കഴിവ്. PSC guide മുഴുവൻ കാണാതെ പഠിച്ച ഒരാൾക്ക് ധാരാളം knowledge ഉണ്ട്; പക്ഷേ ബസ് കിട്ടാതെ വഴിയിൽ പെട്ടുപോകുമ്പോൾ വേറെ വഴി കണ്ടുപിടിക്കുന്നത് intelligence ആണ്. അതുകൊണ്ടാണ് "marks are not the measure of intelligence" എന്ന് പറയുന്നത്.

Knowledge = what we have learned (information + abilities gained from experience)

Intelligence = the ability to comprehend, analyse and apply that knowledge

Knowledge engineering = the part of AI that applies rules to data to simulate how an expert thinks

Knowledge	Intelligence
What you know	How well you use what you know
Grows with time and experience	A capacity to understand, reason and apply
Can be stored (books, databases)	Shown in action: solving new problems
Learning how others understand things	Grasping the things themselves

EXAMPLE

A student has memorised every formula in the calculus textbook but cannot decide which one to use on an unfamiliar exam question. Another student knows fewer formulas but works out the right method. Who shows more knowledge and who shows more intelligence?

The first student has more **knowledge** (stored information). The second shows more **intelligence** (applying knowledge to a new situation). This is exactly the unit's point: knowledge alone does not make one smart.

EXAMPLE

A database stores 10 lakh medical records. Is the database intelligent?

No. It holds knowledge but cannot use it. A diagnosis program that reasons over those records to suggest a disease is using the knowledge, which is where AI begins.

Exam tip: "How does knowledge differ from intelligence?" is the first part of Check Your Progress Q1. Give one-line definitions of both, one sentence on how they are linked, and an example such as the "marks are not the measure of intelligence" proverb.

What is AI? Definitions and the four goals

Definition: Artificial Intelligence (AI) is the branch of computer science concerned with making machines show human-like intelligent capabilities such as reasoning, learning, planning and creativity.

reasoning, learning, planning, creativity തുടങ്ങിയ മനുഷ്യസമാനമായ intelligent കഴിവുകൾ machines-ൽ ഉണ്ടാക്കുന്നതിനെക്കുറിച്ചുള്ള computer science-ന്റെ ശാഖയാണ് Artificial Intelligence (AI).

Artificial Intelligence is the field concerned with incorporating intelligence into machines: the ability of a machine to show human-like capabilities such as reasoning, learning, planning and creativity. Textbook authors define AI in different ways, and their definitions fall along two dimensions: **thinking vs acting**, and measuring success against **humans vs an ideal of rationality**. That gives four possible goals for AI.

മലയാളത്തിൽ

AI-ക്ക് ഒരൊറ്റ definition ഇല്ല. ചിലർ "machine മനുഷ്യനെപ്പോലെ ചിന്തിക്കണം" എന്ന് പറയുന്നു, ചിലർ "മനുഷ്യനെപ്പോലെ പെരുമാറിയാൽ മതി" എന്ന്, മറ്റു ചിലർ "logical ആയി, ശരിയായി ചിന്തിക്കണം / പ്രവർത്തിക്കണം" എന്ന്. ഈ രണ്ട് ചോദ്യങ്ങൾ (think or act? human or rational?) ചേർത്തു വെച്ചാൽ നാല് goals കിട്ടും. ഈ 2 × 2 table ഓർത്തുവെച്ചാൽ മിക്ക definition questions-ഉം എളുപ്പമാകും.

	Human-like	Rational (ideal , logical)
Thinking	Systems that think like humans Haugeland (1985): "The exciting new effort to make computers think... machines with minds, in the full and literal sense." Bellman (1978): "The automation of activities that we associate with human thinking, such as decision-making, problem solving, learning..."	Systems that think rationally Charniak and McDermott (1985): "The study of mental faculties through the use of computational models." Winston (1992): "The study of the computations that make it possible to perceive, reason, and act."
Acting	Systems that act like humans Kurzweil (1990): "The art of creating machines that perform functions that require intelligence when performed by people." Rich and Knight (1991): "The study of how to make computers do things at which, at the moment, people are better."	Systems that act rationally Schalkoff (1990): "A field of study that seeks to explain and emulate intelligent behaviour in terms of computational processes." Luger and Stubblefield (1993): "The branch of computer science that is concerned with the automation of intelligent behaviour."

EXAMPLE: WHICH GOAL DOES THIS DEFINITION BELONG TO?

(a) "Machines with minds." (b) "Automation of intelligent behaviour." (c) "Computations that make it possible to perceive, reason and act." (d) "Machines that perform functions that need intelligence when done by people."

(a) Haugeland: **thinking humanly** (a mind like ours). (b) Luger and Stubblefield: **acting rationally** (behaviour, judged by an ideal). (c) Winston: **thinking rationally** (computations for reasoning). (d) Kurzweil: **acting humanly** (success is measured by comparing with people).

EXAMPLE

A chess program is judged only by whether it wins, not by whether it plays the way grandmasters think. Which goal is it pursuing?

It is judged on its actions against an ideal (winning = best outcome), not on human-like thought. So it pursues **acting rationally**.

Exam tip: The unit announces "eight definitions" but its numbered list jumps from 6 to 8 and prints only seven. The missing one is Rich and Knight (1991), shown in the table above. The quoted wording in the unit is paraphrased; the table gives the standard wording (Russell and Norvig).

AI technologies and levels of AI

Software-level AI: AI that exists only as software, with no physical body, e.g. search engines, virtual assistants, speech and face recognition.

Hardware-level (embodied) AI: AI built into a physical device that senses and acts in the real world, e.g. robots, autonomous vehicles, drones.

Software-level AI: physical body ഇല്ലാതെ software ആയി മാത്രം നിലനിൽക്കുന്ന AI; ഉദാ. search engines, virtual assistants, speech, face recognition.

Hardware-level (embodied) AI: യഥാർത്ഥ ലോകത്തെ sense ചെയ്യുകയും അതിൽ act ചെയ്യുകയും ചെയ്യുന്ന ഒരു physical device-നുള്ളിൽ നിർമ്മിച്ച AI; ഉദാ. robots, autonomous vehicles, drones.

The unit's Figure 1 groups emerging AI technologies by what they let a machine do, just as a human brain **senses**, **comprehends** and **acts**. Each group feeds illustrative solutions such as virtual agents, identity analytics, cognitive robotics, speech analytics, recommendation systems and data visualisation. The unit also divides AI into a software level and a hardware (physical) level.

മലയാളത്തിൽ

മനുഷ്യൻ കണ്ണും ചെവിയും കൊണ്ട് sense ചെയ്യുന്നു, തലച്ചോർ കൊണ്ട് comprehend ചെയ്യുന്നു, കൈകാലുകൾ കൊണ്ട് act ചെയ്യുന്നു. AI-യിലും ഇതുപോലെ: Computer Vision, Audio Processing കൊണ്ട് sense; NLP, Knowledge Representation കൊണ്ട് comprehend; Machine Learning, Expert Systems കൊണ്ട് act. Google Assistant പോലുള്ളത് software level AI ആണ്; robot, self-driving car, drone എന്നിവ ശരീരമുള്ള (hardware) AI ആണ്.

Stage	AI technologies	Illustrative solutions
Sense	Computer vision, audio processing	Virtual agents, identity analytics, speech analytics
Comprehend	Natural language processing, knowledge representation	Cognitive robotics, recommendation systems
Act	Machine learning, expert systems	Recommendation systems, data visualisation

Level	Examples
Software level (virtual AI, no physical body)	Search engines, virtual assistants, speech and face recognition, image-analysis tools
Hardware level (embodied / embedded AI, AI inside a physical device)	Robots, autonomous vehicles, drones, Internet of Things devices

EXAMPLE: CLASSIFY THE LEVEL

(a) A spam filter in Gmail. (b) A drone that sprays pesticide by spotting weeds. (c) A face-unlock feature in a phone app. (d) A warehouse robot.

(a) Software level. (b) Hardware level (the AI is in a flying body). (c) Software level (the phone is only the platform; the AI is the recognition software). (d) Hardware level.

Exam tip: The unit labels the software level "Embodied AI". That is backwards: embodied AI means AI with a physical body (robots, cars, drones), which is the hardware level. Software-only AI is sometimes called disembodied or virtual AI. Write it the correct way round.

Types of AI: ANI, AGI, ASI

ANI (weak / narrow AI): AI built for one specific task that only simulates thinking, without self-awareness; all present-day AI is ANI.

AGI (strong / general AI): hypothetical AI that could think, learn and perform any intellectual task a human can, across all domains.

ASI (super AI): hypothetical AI that would surpass the best human minds in practically every field.

ANI (weak / narrow AI): self-awareness ഇല്ലാതെ thinking-നെ simulate ചെയ്യുക മാത്രം ചെയ്യുന്ന, ഒരു specific task-നായി നിർമ്മിച്ച AI; ഇന്നുള്ള എല്ലാ AI-യും ANI ആണ്.

AGI (strong / general AI): എല്ലാ മേഖലകളിലും മനുഷ്യന് ചെയ്യാവുന്ന ഏത് intellectual task-ഉം think ചെയ്യാനും learn ചെയ്യാനും ചെയ്യാനും കഴിയുന്ന, ഇപ്പോഴും hypothetical ആയ AI.

ASI (super AI): ഏതാണ്ട് എല്ലാ മേഖലകളിലും ഏറ്റവും മികച്ച മനുഷ്യബുദ്ധിയെയും മറികടക്കുന്ന hypothetical AI.

Classified by **capability** (the unit's "Type 1"), AI has three stages. **Artificial Narrow Intelligence (ANI)**, also called weak or narrow AI, is built for one specific job and only simulates thinking, without awareness of what it is doing. **Artificial General Intelligence (AGI)**, strong or general AI, would think and act like a human across the board with a conscious, subjective mind. **Artificial Super Intelligence (ASI)** would be smarter than the best human brains in practically every field. Every AI system that exists today is ANI; AGI and ASI are still hypothetical.

മലയാളത്തിൽ

ANI ഒരു specialist പോലെയാണ്: chess കളിക്കുന്ന program-ന് chess മാത്രമേ അറിയൂ, താൻ ആരാണെന്നോ എന്തിനാണ് കളിക്കുന്നതെന്നോ അതിനറിയില്ല. AGI എന്നാൽ ഒരു സാധാരണ മനുഷ്യനെപ്പോലെ ഏത് വിഷയവും പഠിച്ചെടുക്കാനും മനസ്സിലാക്കാനും കഴിയുന്ന machine. ASI അതിനും മുകളിൽ, എല്ലാ കാര്യത്തിലും മനുഷ്യനെക്കാൾ മിടുക്കുള്ളത്. ഇന്ന് നമ്മൾ കാണുന്ന Siri, ChatGPT, self-driving car എല്ലാം ANI ആണ്; AGI-യും ASI-യും ഇപ്പോഴും research ലക്ഷ്യങ്ങൾ മാത്രം.

	ANI (weak / narrow)	AGI (strong / general)	ASI (super)
Stage	Stage 1: machine learning	Stage 2: machine intelligence	Stage 3: machine consciousness
Scope	One area, one problem	As smart as a human across the board	Much smarter than the best human in practically every field
Thinking	"Simulated" thinking, no self-awareness	"Actual" thinking with a conscious, subjective mind	Beyond human, including rational decisions and even emotional relationships
Exists today?	Yes (all current AI)	No (research goal)	No (speculative)
Examples	Chatbots, spam filters, face recognition, chess engines, today's self-driving cars	A machine that could learn any job a human can	None

The unit says AI can also be classified as "Type 2" but does not describe it. The standard Type 2 classification (by **functionality**) is: **reactive machines** (no memory, e.g. Deep Blue), **limited memory** (use recent past data, e.g. self-driving cars), **theory of mind** (would understand emotions and beliefs; research stage) and **self-awareness** (would have consciousness; hypothetical).

മലയാളത്തിൽ

Type 1 capability നോക്കിയുള്ള തരംതിരിവാൺ (എത്ര മിടുക്കുണ്ട്). Type 2 functionality നോക്കിയുള്ളത് (memory ഉണ്ടോ, മറ്റുള്ളവരുടെ മനസ്സ് മനസ്സിലാക്കുമോ). Reactive machine-ന് ഓർമ്മയില്ല, limited memory machine-ന് കുറച്ചുനേരത്തെ ഓർമ്മയുണ്ട്, ബാക്കി രണ്ടും ഇനി വരാൻിരിക്കുന്നവ.

EXAMPLE: WEAK OR STRONG AI?

- (a) A customer-care chatbot that answers in fluent sentences. (b) Google Maps suggesting the fastest route. (c) A hypothetical robot that can learn cooking, law and surgery on its own and knows why it is doing them.
- (a) Weak AI (ANI): the unit's own example; it talks naturally but "doesn't know who it is or why it's talking to you". (b) Weak AI: one task, route finding. (c) Strong AI (AGI): human-level, general, self-aware.

EXAMPLE

Deep Blue beat Kasparov in 1997. Is it ASI because it beat the world champion?

No. It was superhuman at one task only and could do nothing else, so it is ANI. ASI must exceed humans in practically every field.

Exam tip: The unit gives self-driving cars and hospital operating rooms as "uses" of strong/general AI. That is not correct: AGI does not exist yet, and today's self-driving cars and surgical systems are narrow AI built for one task. The unit also says the difference between AGI and ASI is "marginal"; most authors treat ASI as far beyond AGI. In the exam, give the definitions and the comparison table; use chatbots or spam filters as ANI examples.

Brief history of AI

Definition: The Dartmouth meeting (1956) is the meeting at which the name "Artificial Intelligence", coined by John McCarthy, was adopted; it marks the birth of AI as a field.

John McCarthy രൂപപ്പെടുത്തിയ "Artificial Intelligence" എന്ന പേര് സ്വീകരിച്ച meeting ആണ് Dartmouth meeting (1956); AI ഒരു field ആയി ജനിച്ചത് ഇവിടെയാണ്.

AI's ideas go back to Greek myths of thinking machines and to Aristotle's logic. Mathematical logic (Boole, Frege, Russell), mechanical calculators (Pascal, Babbage) and then electronic computers set the stage. The field was named at the Dartmouth meeting in 1956, went through early reasoning programs, knowledge-based expert systems in the 1970s, the return of neural networks and probability in the 1980s, and practical successes from the 1990s onward.

മലയാളത്തിൽ

ചരിത്രം ഓർക്കാൻ ഒരു കഥ പോലെ വായിക്കുക: ആദ്യം logic (Aristotle, Boole), പിന്നെ calculating machines (Pascal, Babbage), പിന്നെ brain-നെ circuit ആയി കണ്ടു (McCulloch-Pitts 1943), Turing 1950-ൽ "machine-ന് ചിന്തിക്കാമോ" എന്ന് ചോദിച്ചു, 1956-ൽ Dartmouth-ൽ "Artificial Intelligence" എന്ന പേര് വന്നു. പിന്നെ expert systems (DENDRAL, MYCIN), neural networks-ന്റെ തിരിച്ചുവരവ്, 1997-ൽ Deep Blue Kasparov-നെ തോൽപ്പിച്ചു. Exam-ന് വർഷവും ആളും system-ഉം ഒന്നിച്ച് ഓർക്കുക.

Year	Milestone
384–322 BC	Aristotle: informal system of syllogistic logic, the basis of the first formal system of deductive reasoning
17th century	René Descartes: animal bodies are complex machines. 1642: Blaise Pascal builds a mechanical digital calculator (the Pascaline)
19th century	George Boole: Boolean algebra, the "laws of thought" (1854). Charles Babbage and Ada Byron (Lady Lovelace) work on programmable mechanical calculating machines
Late 19th – early 20th c.	Frege, Russell, Whitehead, Gödel: mathematical representations of logical problems
1943	McCulloch and Pitts: Boolean circuit model of the brain, "A Logical Calculus of the Ideas Immanent in Nervous Activity"
1950	Alan Turing: "Computing Machinery and Intelligence" (the imitation game, now the Turing test; machines that learn)
1951	Minsky and Edmonds: SNARC, first randomly wired neural-network learning machine (40 neurons, 3,000 vacuum tubes)
1952–56	Arthur Samuel: checkers-playing programs that learned to play well
1956	Dartmouth meeting: the name "Artificial Intelligence" is adopted (coined by John McCarthy). Newell and Simon's Logic Theorist, often called the first running AI program
1959	Herbert Gelernter: Geometry Theorem Prover
1961	James Slagle: SAINT (LISP), solves freshman calculus (symbolic integration) problems
1963	Thomas Evans: ANALOGY (geometric IQ-test analogies). Feigenbaum and Feldman: Computers and Thought, the first AI anthology
1965	J. Alan Robinson: resolution method for mechanical theorem proving
1967	DENDRAL (Feigenbaum, Lederberg, Buchanan, Sutherland, Stanford): interprets mass spectra; first successful knowledge-based scientific program
1969	SRI's robot Shakey: combines mobility, vision and problem solving
1969–79	

	Knowledge-based systems. 1974: MYCIN, rule-based medical diagnosis. Minsky's frames; Prolog and Planner
1980s	Commercial Lisp machines. 1985: neural networks return. 1988: probabilistic and decision-theoretic methods. 1989: Pomerleau's ALVINN at CMU learns to steer from a 30 × 32 camera image
1990s	Progress in ML, data mining, tutoring, case-based reasoning, NLP, vision, games. Brooks' COG humanoid project at MIT
1997	First official RoboCup. IBM's Deep Blue defeats world chess champion Garry Kasparov
2000	Nomad robot searches Antarctica for meteorites; autonomous space probes
2004	NASA's Mars rovers Spirit and Opportunity complete their initial three-month missions (April 2004)

EXAMPLE: MATCH THE SYSTEM

Match: (a) MYCIN (b) DENDRAL (c) Shakey (d) Logic Theorist (e) SNARC with (i) first neural-network learning machine (ii) mobile robot (iii) medical diagnosis (iv) first running AI program (v) chemistry / mass spectra.

(a)–(iii), (b)–(v), (c)–(ii), (d)–(iv), (e)–(i).

EXAMPLE

Put in order: Deep Blue beats Kasparov, Dartmouth meeting, Turing's paper, McCulloch–Pitts model, MYCIN.

McCulloch–Pitts (1943) → Turing's paper (1950) → Dartmouth (1956) → MYCIN (1974) → Deep Blue (1997).

Exam tip: Corrections to the unit's history: (1) Samuel's programs played checkers and learned to play well; they did not "help people play". (2) SAINT itself solved first-year college calculus (integration) problems; it was not a tool for students. (3) The resolution method's author is J. Alan Robinson. (4) The coast-to-coast drive (about 2,850 miles with a human handling only about 50 miles of steering) happened in 1995 ("No Hands Across America", CMU's Navlab 5, a successor system to ALVINN), not with ALVINN in 1989. (5) The line about an "IIT Kharagpur course" in the MYCIN paragraph is stray text, not AI history.

Components of intelligence: learning

Learning: improving performance through experience; its simplest form is trial and error plus memorising what worked (rote learning).

Generalisation: applying past experience to perform well in new situations never seen before.

Learning: അനുഭവത്തിലൂടെ performance മെച്ചപ്പെടുത്തൽ; ഇതിന്റെ ഏറ്റവും ലളിതമായ രൂപം trial and error-ഉം ഫലിച്ചത് ഓർമ്മയിൽ സൂക്ഷിക്കലുമാണ് (rote learning).

Generalisation: മുൻ കണ്ടിട്ടില്ലാത്ത പുതിയ സാഹചര്യങ്ങളിൽ നന്നായി perform ചെയ്യാൻ past experience ഉപയോഗിക്കൽ.

Psychologists see intelligence not as one talent but as a set of components. AI research has focused on five: **learning, reasoning, problem-solving, perception and language comprehension**. The simplest learning is trial and error plus memorising what worked (rote learning). The harder part is **generalisation**: performing well on situations never seen before. The unit lists five ways to learn.

മലയാളത്തിൽ

Multiplication table കാണാതെ പഠിക്കുന്നത് rote learning ആണ്, അതിൽ ചിന്തയൊന്നും വേണ്ട. പക്ഷേ "jump" എന്ന verb ഒരിക്കലും കണ്ടിട്ടില്ലെങ്കിലും "-ed" ചേർത്ത് "jumped" ഉണ്ടാക്കാൻ പറ്റുന്നത് generalisation ആണ്. Car ഓടിക്കാൻ അറിയുന്ന ഒരാൾ lorry ഓടിക്കാൻ ശ്രമിക്കുന്നത് analogy; കുറേ പശുക്കളെ കണ്ട് "പശുവിന് നാല് കാൽ" എന്ന് തീരുമാനിക്കുന്നത് induction.

Method	Idea	Example
Rote learning	Information simply copied into memory; least interpretation	Memorising multiplication tables; a chess program storing the winning "mate in one" move
Learning by instruction	Told by a teacher; new knowledge must be fitted into the existing knowledge base, so more inference is needed	A teacher explaining a new rule in class
Learning by analogy	New ideas by connecting to previously learned concepts	Solving textbook exercises similar to the solved examples; a car driver learning to drive a heavy vehicle
Learning by deduction	Deductive inference: if the premises are true, the conclusion is always true (irrefutable)	Mathematical proof
Learning by induction	General conclusion from many examples; usually right but can be wrong	"Cows have four legs and two horns", until a three-legged or one-horned cow appears

An agent's learning depends on **what** it learns (concepts, problem-solving, game playing), the **representation** it uses (predicate calculus, frames, scripts) and the **critic** that judges its performance.

EXAMPLE: NAME THE LEARNING METHOD

(a) A child learns that $7 \times 8 = 56$ by repeating it. (b) From "all squares are rectangles" and "ABCD is a square", a program concludes "ABCD is a rectangle". (c) After seeing 500 spam e-mails with "lottery" in them, a filter decides "lottery" signals spam. (d) A student who knows Java picks up C# by noticing the similar syntax.

(a) Rote. (b) Deduction (true premises guarantee the conclusion). (c) Induction (generalised from examples; could be wrong for a genuine lottery department e-mail). (d) Analogy.

Exam tip: The unit says a generalising program learns the "added" rule for past tense; it means the "add -ed" rule (jump → jumped). Remember the key contrast: deduction is certain, induction is only probable.

Learning from feedback: supervised, unsupervised, reinforcement

Supervised learning: learning a function from example inputs paired with their correct outputs (labels).

Unsupervised learning: discovering patterns or structure in inputs when no correct outputs are given.

Reinforcement learning: learning from occasional rewards or penalties so as to maximise the total reward.

Supervised learning: correct outputs (labels) ചേർത്ത example inputs-ൽ നിന്ന് ഒരു function പഠിക്കൽ.

Unsupervised learning: correct outputs ഒന്നും നൽകാത്തപ്പോൾ inputs-ലെ patterns അല്ലെങ്കിൽ structure സ്വയം കണ്ടെത്തൽ.

Reinforcement learning: ഇടയ്ക്കിടെ കിട്ടുന്ന rewards അല്ലെങ്കിൽ penalties-ൽ നിന്ന് പഠിച്ച് ആകെ reward പരമാവധിയാക്കൽ.

Building on the idea of a critic, learning based on feedback is classified as supervised, unsupervised, reinforcement and delayed-reinforcement learning. In **supervised** learning the system learns a function from example inputs paired with correct outputs. In **unsupervised** learning there are no correct outputs given, so the system must discover on its own which properties of the data matter. In **reinforcement** learning the agent gets occasional rewards or penalties and tries to maximise total reward. **Delayed reinforcement** is the case where the reward depends on earlier actions and arrives later.

മലയാളത്തിൽ

Supervised learning ടീച്ചർ ഉത്തരസൂചിക തന്ന് പഠിപ്പിക്കുന്നതുപോലെയാണ്: ഓരോ ചോദ്യത്തിനും ശരിയായ ഉത്തരം കൂടെയുണ്ട്. Unsupervised-ൽ ഉത്തരസൂചിക ഇല്ല; ഒരു കുട്ടി മീൻ കിട്ടിയാൽ പേരിരിയല്ലെങ്കിലും വലുപ്പവും നിറവും നോക്കി കൂട്ടങ്ങളായി തിരിക്കുന്നതുപോലെ. Reinforcement എന്നാൽ നായക്കുട്ടിയെ പരിശീലിപ്പിക്കുന്നതുപോലെ: ശരി ചെയ്താൽ biscuit, തെറ്റിയാൽ ശകാരം. Chess-ൽ കളി ജയിച്ചാൽ മാത്രമാണ് reward, ഏത് നീക്കമാണ് സഹായിച്ചതെന്ന് പിന്നീട് കണ്ടുപിടിക്കണം: അതാണ് delayed reinforcement.

Type	Feedback given	Example
Supervised	Correct output (label) for every training input	Photos labelled "cat"/"dog"; e-mails labelled spam / not spam
Unsupervised	No labels; find structure yourself	Grouping customers into segments by buying habits
Reinforcement	Occasional reward or punishment	A game-playing agent scoring points; a robot rewarded for reaching a goal
Delayed reinforcement	Reward arrives later and depends on earlier actions	Chess: win or loss known only at the end of the game

For tasks that never end, total future reward could be unbounded, so the agent may **discount** the future: it prefers rewards that come soon over rewards far away, or ignores rewards after some point.

EXAMPLE: WHICH TYPE OF LEARNING?

(a) A bank trains a model on 1 lakh past loans marked "repaid" or "defaulted". (b) A news site groups articles into topics without any topic names given. (c) A robot vacuum gets +1 for each clean tile and -10 for bumping into furniture.

(a) Supervised (each input has the correct output). (b) Unsupervised (clustering, no labels). (c) Reinforcement (rewards and penalties).

Exam tip: The unit's unsupervised example ("find the shortest way between two cities in a country you know nothing about") is really a search or exploration problem, and its line "no way to know what the inputs are" should read "the inputs are given but the expected outputs are not". Use a clustering example (grouping customers or documents) for unsupervised learning.

Reasoning, problem-solving, perception, language

Reasoning: drawing conclusions appropriate to the situation, by deduction (conclusion certain) or induction (conclusion probable).

Problem-solving: a systematic search through possible actions to reach a given goal; means-end analysis does this by reducing the current-state-to-goal difference step by step.

Perception: scanning the environment through sense organs (real or artificial) and analysing the scene into objects, their features and relations.

Language comprehension: understanding a system of signs whose meaning is fixed by convention.

Reasoning: സാഹചര്യത്തിന് യോജിച്ച conclusions എത്തിച്ചേരൽ; deduction (conclusion ഉറപ്പ്) വഴിയോ induction (conclusion സാധ്യത മാത്രം) വഴിയോ.

Problem-solving: ഒരു നിശ്ചിത goal-ൽ എത്താൻ സാധ്യമായ actions-യുടെ നടത്തുന്ന systematic search; means-end analysis ഇത് current state-ഉം goal-ഉം തമ്മിലുള്ള വ്യത്യാസം പടിപടിയായി കുറച്ചാണ് ചെയ്യുന്നത്.

Perception: sense organs (യഥാർത്ഥമോ artificial-ഓ) വഴി environment scan ചെയ്ത് scene-നെ objects, അവയുടെ features, relations എന്നിങ്ങനെ analyse ചെയ്യൽ.

Language comprehension: convention കൊണ്ട് അർത്ഥം നിശ്ചയിക്കപ്പെട്ട signs-ന്റെ ഒരു system മനസ്സിലാക്കൽ.

Reasoning is drawing conclusions appropriate to the situation, by deduction (true premises guarantee the conclusion) or induction (premises only support it). Drawing conclusions is not enough: the hard part is knowing what is relevant. **Problem-solving** means "given these data, find x"; methods are special-purpose or general, and a key general method is **means-end analysis**, which reduces the difference between the current state and the goal step by step. **Perception** is scanning the surroundings through sense organs (real or artificial) and analysing the scene into objects, features and relations. **Language comprehension** is understanding a system of signs with meaning fixed by convention; human languages are **productive**, allowing infinitely many different sentences.

മലയാളത്തിൽ

"Fred museum-ലോ café-യിലോ ഉണ്ട്; café-യിൽ ഇല്ല, അതിനാൽ museum-ൽ" എന്നത് deduction, ഉറപ്പുള്ള ഉത്തരം. "മുമ്പ് ഇതുപോലുള്ള accidents instrument failure കൊണ്ടായിരുന്നു, അതിനാൽ ഇതും അങ്ങനെയാവാം" എന്നത് induction, പുതിയ തെളിവ് വന്നാൽ മാറാം. Means-end analysis എന്നാൽ കോഴിക്കോട് നിന്ന് തിരുവനന്തപുരത്തേക്ക് പോകാൻ "ഇപ്പോൾ എവിടെ, ലക്ഷ്യം എവിടെ, ഈ ദൂരം കുറയ്ക്കാൻ ഏത് step" എന്ന് ഓരോ തവണയും നോക്കുന്നതുപോലെ. Traffic signs ഒരു ചെറിയ language ആണ്, പക്ഷേ മലയാളം പോലുള്ള മനുഷ്യഭാഷ productive ആണ്: അനന്തമായ പുതിയ വാക്യങ്ങൾ ഉണ്ടാക്കാം.

Deductive: premises true $\Rightarrow$ conclusion certainly true

Inductive: premises true $\Rightarrow$ conclusion probably true (new evidence may overturn it)

Means-end analysis: repeatedly pick an action that reduces (current state – goal state)

Conventional meaning (hazard sign = danger ahead) $\neq$ natural meaning (dark clouds mean rain)

EXAMPLE: DEDUCTIVE OR INDUCTIVE?

(a) "Every time the server crashed this month, memory usage was above 95%. It is above 95% now, so it will probably crash." (b) "All employees get a PF account. Anu is an employee, so Anu has a PF account."

(a) Inductive: based on past cases; the crash is only likely. (b) Deductive: if both premises are true the conclusion must be true.

EXAMPLE: MEANS-END ANALYSIS

A simple robot with actions {pick up, put down, move forward, back, left, right} must take a box from cell A to cell C, two cells to its right.

Difference 1: robot not holding the box $\rightarrow$ pick up. Difference 2: box is 2 cells from C $\rightarrow$ move right, move right. Difference 3: box still held at goal $\rightarrow$ put down. Each action is chosen because it reduces the current difference from the goal state.

EXAMPLE

Why is perception hard for a machine? Give an early system that combined perception and action.

The same object looks different from different angles, under shadow or when partly hidden. FREDDY (Edinburgh University, 1966–1973, directed by Donald Michie) had a moving TV "eye" and a pincer "hand" and could assemble simple objects such as a toy car from a jumble of parts.

Approaches to AI

Definition: The approaches to AI are acting humanly (Turing test), thinking humanly (cognitive modelling), thinking rationally (laws of thought) and acting rationally (rational agent).

AI-യുടെ approaches: acting humanly (Turing test), thinking humanly (cognitive modelling), thinking rationally (laws of thought), acting rationally (rational agent).

How do we measure whether AI makes a machine behave like a human? The unit gives four approaches, which match the four goals of AI. **Acting humanly:** the Turing test approach. **Thinking humanly:** the cognitive modelling approach, which builds AI models on human cognition using introspection, psychological experiments and brain imaging. **Thinking rationally:** the laws of thought approach, which encodes logical rules of correct reasoning as algorithms. **Acting rationally:** the rational agent approach, which tries to reach the best possible (or best expected) outcome in every situation.

മലയാളത്തിൽ

Human-centred approaches (thinking / acting humanly) ഒരു empirical science ആണ്: മനുഷ്യരെ നിരീക്ഷിച്ച്, hypothesis വെച്ച്, പരീക്ഷിച്ച് നോക്കണം. Rationalist approaches (thinking / acting rationally) maths-ഉം engineering-ഉം ചേർന്നതാണ്. Laws of thought-ന്റെ പ്രശ്നം: നിയമം പുസ്തകത്തിൽ എഴുതി solve ചെയ്യുന്നതും യഥാർത്ഥ ലോകത്ത് solve ചെയ്യുന്നതും രണ്ടും രണ്ടാണ്. Rational agent കൂടുതൽ flexible ആണ്: ഒരു "logically correct" ഉത്തരം ഇല്ലാത്തപ്പോഴും, സാഹചര്യം നോക്കി ഏറ്റവും നല്ലത് തിരഞ്ഞെടുക്കും.

Approach	Goal	How it works	Weakness / note
Turing test	Acting humanly	Machine passes if an interrogator cannot tell it from a human	Imitating a quality is not having it (Chinese Room)
Cognitive modelling	Thinking humanly	Model the human mind via introspection, psychological experiments, brain imaging (MRI)	Needs an accurate theory of how the mind works
Laws of thought	Thinking rationally	Write down logical rules of correct reasoning and implement them	Solving "by the law" is very different from solving real, messy problems
Rational agent	Acting rationally	Choose the action with the best outcome given the circumstances	Most flexible; works even when there is no single logically right action

EXAMPLE: WHICH APPROACH?

(a) Researchers record volunteers thinking aloud while solving puzzles and write a program that follows the same steps. (b) A theorem prover that derives conclusions only by valid rules of logic. (c) A self-driving car that picks the route with the best mix of safety and travel time. (d) A chatbot built so that judges cannot tell it apart from a person in conversation.

(a) Thinking humanly (cognitive modelling, psychological experiments/introspection). (b) Thinking rationally (laws of thought). (c) Acting rationally (rational agent: best outcome under the circumstances). (d) Acting humanly (Turing test).

EXAMPLE

Why do we need to separate "human" behaviour from "rational" behaviour?

Humans are often irrational (tired, emotional, biased). A system that copies humans will copy their mistakes; a rational system aims for the best outcome instead. A human driver may speed when angry; a rational driving agent would not.

Exam tip: Check Your Progress Q5 ("How do we measure if AI makes a machine behave like a human?") is answered by listing and briefly explaining these four approaches. Always pair each approach with its goal.

The Turing test and the Chinese Room

Turing test: Turing's 1950 test in which a human interrogator questions a machine and a human through text terminals; if the interrogator cannot tell them apart, the machine is considered intelligent.

Chinese Room: Searle's (1980) argument that a system can appear to understand by manipulating symbols by rules without really understanding them: imitation $\neq$ understanding.

Turing test: 1950-ൽ Turing നിർദ്ദേശിച്ച test: ഒരു human interrogator text terminals വഴി ഒരു machine-നോടും ഒരു മനുഷ്യനോടും ചോദ്യങ്ങൾ ചോദിക്കുന്നു; ഏതാണ് machine എന്ന് തിരിച്ചറിയാൻ കഴിഞ്ഞില്ലെങ്കിൽ machine intelligent ആയി കണക്കാക്കും.

Chinese Room: rules അനുസരിച്ച് symbols കൈകാര്യം ചെയ്ത് ഒരു system-ന് യഥാർത്ഥത്തിൽ മനസ്സിലാക്കാതെ തന്നെ മനസ്സിലാക്കുന്നതായി തോന്നിപ്പിക്കാം എന്ന Searle-ന്റെ (1980) argument: imitation $\neq$ understanding.

Alan Turing proposed the test in 1950 (as the "imitation game") to decide whether a machine is intelligent. Three parties sit in separate rooms: the machine (A), a human (B) and a human interrogator (C). C talks to both only through computer terminals, puts questions to both without knowing which is which, and must identify the computer. If C cannot tell the machine from the human, the machine is considered intelligent. The machine should even avoid answering too fast (for example, not adding two 20-digit numbers in under a hundredth of a second), since that would reveal it.

മലയാളത്തിൽ

ഒരു WhatsApp chat സങ്കൽപ്പിക്കുക: നിങ്ങൾ രണ്ടു പേരോട് chat ചെയ്യുന്നു, ഒന്ന് മനുഷ്യൻ, മറ്റേത് computer. ആരാണ് computer എന്ന് നിങ്ങൾക്ക് പറയാൻ പറ്റിയില്ലെങ്കിൽ ആ computer Turing test pass ആയി. പക്ഷേ John Searle ചോദിച്ചു: അഭിനയിക്കുന്നത് മനസ്സിലാക്കുന്നതാണോ? Chinese അറിയാത്ത ഒരാൾ പുസ്തകം നോക്കി Chinese ഉത്തരം അയച്ചാൽ പുറത്തുള്ളവർക്ക് അയാൾക്ക് Chinese അറിയാമെന്ന് തോന്നും, പക്ഷേ അറിയില്ല. ഇതാണ് Chinese Room argument.

Turing test (1950): passes $\Rightarrow$ interrogator C cannot reliably tell machine A from human B, using text-only communication

Chinese Room (John Searle, 1980): simulating a quality $\neq$ possessing that quality

Criticism: the Chinese Room. Searle argued that convincing others you have a quality does not mean you have it. In the unit's version, a Shakespeare scholar who knows English but not Chinese sits in a room with an encyclopedia whose facing pages are Chinese and the English translation. Chinese questions about Shakespeare come through one window; the scholar finds the matching Chinese text, reads the English, works out the answer, finds the corresponding Chinese characters and passes them out through the other window. From outside he

seems to know Chinese, but he does not. Likewise a machine could pass the Turing test without understanding anything.

EXAMPLE: TURING TEST SCENARIOS

(a) An interrogator chats with A and B for 10 minutes and guesses wrongly which one is the machine about as often as she guesses right. (b) The machine instantly answers "What is $98,765,432,123,456,789,012 \times 3$?" correctly in 0.001 s. (c) The judge can see the participants through a glass window.

(a) The machine passes: C cannot distinguish it from the human. (b) The machine likely fails: no human answers that fast, so speed gives it away; a clever machine would pause and perhaps even make a slip. (c) Not a valid Turing test: identity must be judged only from the replies, through terminals, never from appearance.

EXAMPLE

A modern chatbot fools many users into thinking they are talking to a person. Does the Chinese Room argument say it understands language?

No. By Searle's argument it may only be manipulating symbols by rules (as the scholar matches characters); passing as a speaker is not the same as understanding. The unit's analogy: a man persuading people he is a woman does not make him able to bear children.

Exam tip: The unit's text muddles the roles; state them clearly: A = machine, B = human, C = interrogator who questions both. Searle's original (1980) Chinese Room uses a rulebook for matching Chinese symbols with no English translation at all; the unit's "Shakespeare encyclopedia" is a variant with the same conclusion. Either version earns marks if you state the conclusion: imitation $\neq$ understanding.

AI vs ML vs DL

Machine learning (ML): a branch of AI in which algorithms learn patterns from data and improve with experience, without being explicitly programmed.

Deep learning (DL): a subfield of ML that uses multi-layer artificial neural networks to learn increasingly complex features from large amounts of data.

Machine learning (ML): explicitly program ചെയ്താൽ algorithms data-യിൽ നിന്ന് patterns പഠിക്കുകയും experience കൊണ്ട് മെച്ചപ്പെടുകയും ചെയ്യുന്ന AI-യുടെ ശാഖ.

Deep learning (DL): വലിയ അളവ് data-യിൽ നിന്ന് കൂടുതൽ കൂടുതൽ complex ആയ features പഠിക്കാൻ multi-layer artificial neural networks ഉപയോഗിക്കുന്ന ML-ന്റെ subfield.

AI is the most general term and ranges from top-down methods (knowledge representation) to bottom-up methods (machine learning). **Machine learning** is a branch of AI in which algorithms learn patterns from data and experience instead of following explicit programmed instructions, and improve as they see more data. **Deep learning** is a subfield of ML using **artificial neural networks** with many layers, inspired by the brain; each layer learns more complex features, so DL handles images, text and sound with less manual pre-processing and often greater accuracy, given large (usually labelled) datasets.

മലയാളത്തിൽ

മൂന്നും ഒന്നിനുള്ളിൽ ഒന്നായി വരുന്ന വൃത്തങ്ങളാണ്: ഏറ്റവും വലുത് AI, അതിനുള്ളിൽ ML, അതിനുള്ളിൽ DL. Rules എഴുതിവെച്ച expert system AI ആണ്, പക്ഷേ ML അല്ല. ML-ൽ നമ്മൾ rules എഴുതുന്നില്ല, data കാണിച്ചുകൊടുക്കുന്നു, അത് സ്വയം pattern കണ്ടുപിടിക്കുന്നു. DL-ൽ അനേകം layers ഉള്ള neural network ആദ്യ layer-ൽ edges, പിന്നെ shapes, പിന്നെ മുഖം എന്ന മട്ടിൽ പടിപടിയായി features പഠിക്കുന്നു.

$DL \subseteq ML \subseteq AI$

Sub-domains of AI: machine learning, deep learning, neural networks, natural language processing, computer vision

Kinds of ML: supervised, unsupervised, reinforcement, semi-supervised

	Artificial Intelligence	Machine Learning	Deep Learning
Scope	Broadest: any technique that makes machines act intelligently	Subset of AI: learning from data	Subset of ML: multi-layer neural networks
How behaviour is obtained	Rules, search, logic, knowledge representation, or learning	Algorithms find patterns in data; not explicitly programmed	Layers of artificial neurons learn features automatically
Data needs	Can work with little data (rule-based)	Moderate; often needs hand-crafted features	Large datasets and heavy computation
Pre-processing	Depends on method	More manual feature engineering	Less pre-processing; raw images, text, sound
Example	A rule-based chess program; MYCIN expert system	Spam filter trained on labelled e-mails; loan-default prediction	Face recognition; speech-to-text; image classification

EXAMPLE: AI, ML OR DL?

(a) A tax calculator with hand-written IF-THEN rules for deductions. (b) A model that predicts house prices from area and location using past sales. (c) A 50-layer neural network that detects pneumonia from chest X-rays.

(a) AI (rule-based) but not ML: nothing is learned from data. (b) ML: learns the pattern from past data. (c) DL (and therefore also ML and AI): deep neural network on raw images.

Exam tip: For "Compare AI, ML and DL" draw the three nested circles (DL inside ML inside AI) and give the table. The unit's text refers to "Figure 5" for this diagram; it means Figure 2(a).

Descriptive, predictive and prescriptive analytics

Descriptive analytics: analysis of past data to describe what happened.

Predictive analytics: probabilistic forecasting of what will happen.

Prescriptive analytics: recommending what should be done to achieve given goals.

Descriptive analytics: എന്ത് സംഭവിച്ചു എന്ന് വിവരിക്കാൻ past data analyse ചെയ്യൽ.

Predictive analytics: എന്ത് സംഭവിക്കും എന്നതിന്റെ probabilistic forecasting.

Prescriptive analytics: നിശ്ചിത goals നേടാൻ എന്ത് ചെയ്യണം എന്ന് recommend ചെയ്യൽ.

Analytics comes in three types, in order of increasing complexity. **Descriptive** analytics describes what happened and is used heavily in all industries. **Predictive** analytics anticipates what will happen; it is inherently probabilistic and a key source of insight in data-driven organisations. **Prescriptive** analytics recommends what to do to achieve goals and is used heavily by leading data and internet companies. Machine learning's focus is on predictions and prescriptions.

മലയാളത്തിൽ

ഒരു പലചരക്ക് കടയുടെ ഉദാഹരണം: "കഴിഞ്ഞ മാസം എത്ര അരി വിറ്റു?" എന്നത് descriptive. "അടുത്ത മാസം ഓണം ആയതുകൊണ്ട് എത്ര വിൽക്കും?" എന്നത് predictive. "അതിനാൽ 200 kg കൂടുതൽ order ചെയ്യൂ, ഈ offer കൊടുക്കൂ" എന്ന് പറയുന്നത് prescriptive. ML പ്രധാനമായും രണ്ടാമത്തെയും മൂന്നാമത്തെയും കാര്യത്തിലാണ് സഹായിക്കുന്നത്.

	Descriptive	Predictive	Prescriptive
Question answered	What happened?	What will happen?	What should we do?
Nature	Summaries of past data	Probabilistic forecast	Recommendations to achieve goals
Complexity	Lowest	Medium	Highest
Used by	All industries	Data-driven organisations	Leading data and internet companies
Example	Monthly sales report	Forecasting next month's demand	Suggesting stock orders and prices

EXAMPLE: CLASSIFY THE ANALYTICS

(a) A hospital dashboard showing last week's admissions by ward. (b) A model estimating the chance a patient is readmitted within 30 days. (c) A system that recommends which patients get follow-up calls to cut readmissions.

(a) Descriptive. (b) Predictive (a probability). (c) Prescriptive (a recommended action).

Application areas of AI

Definition: An application of AI is the use of AI techniques such as ML, computer vision and NLP to automate or improve tasks and decisions in a sector such as healthcare, agriculture, finance or education.

healthcare, agriculture, finance, education പോലുള്ള ഒരു മേഖലയിലെ tasks-ഉം decisions-ഉം automate ചെയ്യാനോ മെച്ചപ്പെടുത്താനോ ML, computer vision, NLP തുടങ്ങിയ AI techniques ഉപയോഗിക്കുന്നതാണ് AI-യുടെ application.

AI can optimise almost any sector. The unit lists: **healthcare** (AI diagnostics, personalised treatment, early pandemic detection, imaging; helps rural areas short of doctors), **agriculture** (real-time crop advice, pest detection, price forecasting, precision farming with drones, sensors, agribots and autonomous tractors; the world must produce about 50% more food for about 2 billion more people by 2050), **smart mobility** (autonomous fleets, driver assistance, predictive maintenance, traffic control), **retail** (personalised suggestions, image-based search, demand and inventory), **manufacturing** (predictive maintenance, quality inspection by vision, supply-chain demand prediction), **energy** (forecasting, smart grids and meters, grid maintenance), **smart cities** (traffic and crowd management), **education** (personalised learning, predicting dropouts) and **finance** (fraud detection, trading).

മലയാളത്തിൽ

നമ്മുടെ ദൈനംദിന ജീവിതത്തിൽ തന്നെ AI ഉണ്ട്: Gmail-ലെ spam filter, Google Maps-ലെ traffic prediction, Uber-ന്റെ fare, bank-ൽ നിന്ന് "ഈ transaction നിങ്ങളുടേതാണോ?" എന്ന SMS, phone-ലെ face unlock. കൃഷിയിൽ drone കൊണ്ട് കീടനാശിനി തളിക്കുന്നതും, soil sensor നോക്കി എപ്പോൾ നനയ്ക്കണമെന്ന് പറയുന്നതും AI ആണ്.

Top used applications: plagiarism checkers · face recognition · autopilot on commercial planes · ride sharing (Uber, Lyft) · spam filters, voice-to-text, search suggestions · Google Maps predictions · fraud prevention · smart assistants (Siri, Alexa)

EXAMPLE: NAME THE SECTOR AND THE AI USE

(a) A bank's system flags an unusual ₹4 lakh debit-card purchase abroad at 3 a.m. (b) A camera on a production line rejects bottles with cracked caps. (c) An app tells a Wayanad farmer the best week to sow based on weather and price forecasts. (d) A platform alerts a counsellor that a student is likely to drop out.

(a) Finance: fraud detection. (b) Manufacturing: quality assurance with vision and ML. (c) Agriculture: real-time advisory and price forecasting. (d) Education: predicting when a student needs help.

Exam tip: For "discuss application areas", name at least five sectors with one concrete use case each. The unit also notes that adoption has been slowed by technical and regulatory challenges, and mostly driven by business impact.

Intelligent agents: definition and properties

Agent: an entity that perceives its environment through sensors and acts upon it through actuators.

Percept: the input an agent receives through its sensors at a given time.

Percept sequence: the series of percepts received over time, on which the agent bases its choice of action.

Agent: sensors വഴി environment-നെ perceive ചെയ്യുകയും actuators വഴി അതിൽ act ചെയ്യുകയും ചെയ്യുന്ന ഒരു entity.

Percept: ഒരു നിശ്ചിത സമയത്ത് agent-ന് sensors വഴി കിട്ടുന്ന input.

Percept sequence: കാലക്രമത്തിൽ കിട്ടുന്ന percepts-ന്റെ series; ഇതിനെ ആശ്രയിച്ചാണ് agent action തിരഞ്ഞെടുക്കുന്നത്.

An **agent** is an entity that perceives its environment through **sensors** and acts on it through **actuators**, usually on behalf of someone; some experts also require it to be autonomous and goal-directed. A **percept** is the input received through sensors over a unit of time, enough to make sense of; a **percept sequence** is a series of percepts long enough for the agent to start an action. A **software agent** (early definition credited to John McCarthy and colleagues) is a system that, given a goal, carries out the needed computer operations and, if stuck, asks humans for advice, evaluates it and acts suitably.

മലയാളത്തിൽ

Agent എന്ന വാക്ക് നമുക്ക് പരിചയമുണ്ട്: travel agent, insurance agent, നമുക്ക് വേണ്ടി ഒരു പണി ചെയ്യുന്നയാൾ. AI agent-ഉം അങ്ങനെ തന്നെ: sensors കൊണ്ട് ചുറ്റും നോക്കുന്നു (camera, microphone, keyboard input), actuators കൊണ്ട് പ്രവർത്തിക്കുന്നു (wheels, screen, speaker). ഒരു നിമിഷത്തെ input percept, ഇതുവരെയുള്ള എല്ലാ inputs-ഉം ചേർന്നത് percept sequence.

Agent: perceives via sensors, acts via actuators

Computer agent attributes: (i) autonomous control (ii) perceives its environment (iii) persists over a long time (iv) adapts to change (v) can take on others' goals

Two ways to define an agent: (1) ascription, "Agent is that agent does" (2) description of properties it must have

Under the **ascription** view, an agent is whatever acts on someone's behalf for a delegated task; it must know its domain and its user (a football-playing agent differs from an e-mail agent, though both model their user). Under the **description** view, an agent functions continuously and autonomously in an environment that may contain other agents, and should have some or all of these properties:

Property	Meaning
Reactivity	Senses the environment and acts accordingly
Autonomy	Moves towards its goal, changing strategy if needed, without much human intervention
Communicating ability	Communicates with other agents and humans
Coexisting by cooperating	Works in a multi-agent environment towards a common goal
Adaptability	Learns, changes and adapts to new situations
Drawing inferences	Concludes useful facts not directly available
Temporal continuity	Works over long periods of time
Personality	Impersonates or simulates the person on whose behalf it acts
Mobility	Moves from one environment to another

EXAMPLE: SENSORS AND ACTUATORS

Identify the sensors and actuators of (a) a robot vacuum cleaner (b) an e-mail spam-filtering software agent.

(a) Sensors: bump sensors, cliff sensors, camera or lidar, dirt sensor. Actuators: wheels, brushes, suction motor. (b) Sensors: incoming e-mail (headers, text), user's "mark as spam" clicks. Actuators: moving mail to the spam folder, labelling, alerting the user.

EXAMPLE: WHICH PROPERTY?

(a) A delivery drone reroutes around a storm on its own. (b) A team of warehouse robots split up the orders between them. (c) A trading bot runs 24 × 7 for months. (d) A virtual assistant replies to e-mails in your writing style.

(a) Autonomy (and adaptability). (b) Coexisting by cooperating (with communication). (c) Temporal continuity. (d) Personality.

Exam tip: Check Your Progress Q9 wants the definition (sensors + actuators), then the list of properties with one line each. Nine properties, so a mnemonic helps: **R-A-C-C-A-I-T-P-M**.

Task environment and PEAS

Task environment: all the elements of the problem to which the agent is the solution; it is specified using PEAS.

PEAS: Performance measure, Environment, Actuators, Sensors: the four parameters used to specify an agent's task environment.

Task environment: agent പരിഹാരമായി വരുന്ന problem-ലെ എല്ലാ elements-ഉം; ഇത് PEAS ഉപയോഗിച്ച് specify ചെയ്യുന്നു.

PEAS: Performance measure, Environment, Actuators, Sensors: ഒരു agent-ന്റെ task environment specify ചെയ്യാൻ ഉപയോഗിക്കുന്ന നാല് parameters.

A **task environment** (problem environment) contains all the elements involved in the problem for which the agent is the solution; it changes with every new task. The first step in designing an agent is to specify the task environment as fully as possible, using four parameters called **PEAS**: **Performance** measure (the expected performance), **Environment** (the world around the agent), **Actuators** (through which it acts) and **Sensors** (through which it gathers information).

മലയാളത്തിൽ

ഒരു agent ഉണ്ടാക്കുന്നതിനു മുമ്പ് നാല് ചോദ്യങ്ങൾക്ക് ഉത്തരം കണ്ടെത്തണം: agent നന്നായി ജോലി ചെയ്യോ എന്ന് എങ്ങനെ അളക്കും (P), എവിടെയാണ് ജോലി (E), എന്ത് ഉപയോഗിച്ച് പ്രവർത്തിക്കും (A), എന്ത് ഉപയോഗിച്ച് ചുറ്റുപാട് അറിയും (S). ഒരു KSRTC bus automatic ആയി ഓടിക്കുന്ന agent-നെ ആലോചിക്കുക: യാത്രക്കാരുടെ safety-യും comfort-ഉം ആണ് P, ചുരം റോഡും highway-യും ഓട്ടോയും കാളവണ്ടിയും ഉള്ള road ആണ് E, steering-ഉം brake-ഉം A, GPS-ഉം speedometer-ഉം camera-യും S.

P = Performance measure: how success is judged

E = Environment: the world the agent works in

A = Actuators: how the agent acts

S = Sensors: how the agent perceives

EXAMPLE (FROM THE UNIT): AUTOMATED PUBLIC ROAD TRANSPORT DRIVER

Give PEAS for an automated bus/taxi driver agent.

P: maximise passenger safety and comfort; reach the correct destination; minimise travel time while obeying traffic rules; cause minimum disturbance to other road users; minimise cost.

E: roads from 12-lane expressways to bumpy rural roads; left-hand or right-hand traffic rules; varying knowledge of places; all kinds of passengers; other traffic including heavy vehicles, modern cars, three-wheelers and bullock carts.

A: steering wheel, brakes, gears, accelerator; display screen; communication devices.

S: odometer, speedometer, engine sensors, GPS, distance sensors (cameras, radar) to other vehicles, road-condition and turn detection.

EXAMPLE

Give PEAS for a medical diagnosis agent.

P: correct diagnosis, patient recovery, low cost, few unnecessary tests. **E:** patients, hospital, doctors, lab. **A:** display of questions, tests ordered, diagnoses and treatments. **S:** keyboard entry of symptoms, patient answers, lab reports, scan images.

EXAMPLE

Give PEAS for an English-tutoring agent for Class 8 students.

P: improvement in students' test scores, engagement. **E:** set of students, school, exam board. **A:** screen showing exercises, suggestions and corrections; voice output. **S:** keyboard/touch input, microphone for spoken answers.

Exam tip: Check Your Progress Q10 asks exactly this. Write the four expansions first, then a PEAS table for the driver agent. The unit notes that simpler agents (game-playing robot, assembly-line robot, image-processing agent) work in less dynamic environments than the driver agent.

Agent structure; simple reflex and model-based agents

Agent structure: agent = architecture (the device with sensors and actuators) + agent program (which maps percepts to actions).

Simple reflex agent: an agent that selects actions from the current percept only, using condition-action rules, ignoring percept history.

Model-based reflex agent: an agent that keeps an internal state, updated using a model of how the world works, to act in partially observable environments.

Agent structure: agent = architecture (sensors-ഉം actuators-ഉം ഉള്ള device) + agent program (percepts-നെ actions-ലേക്ക് map ചെയ്യുന്നത്).

Simple reflex agent: percept history അവഗണിച്ച്, condition-action rules ഉപയോഗിച്ച് current percept മാത്രം നോക്കി action തിരഞ്ഞെടുക്കുന്ന agent.

Model-based reflex agent: partially observable environments-ൽ act ചെയ്യാൻ, world എങ്ങനെ പ്രവർത്തിക്കുന്നു എന്ന model ഉപയോഗിച്ച് update ചെയ്യുന്ന internal state സൂക്ഷിക്കുന്ന agent.

An agent has two parts: the **architecture** (the hardware device with sensors and actuators in which it lives) and the **agent program** (which maps percepts to actions). A different architecture needs a different program. A **table-driven** agent that stores an action for every possible percept sequence is possible in principle but hopelessly large, so compact agent programs are preferred. A **simple reflex (SR) agent** has no internal state: it reacts only to the current percept using **condition-action rules**. It cannot learn or use past experience, and works correctly only if the environment is **fully observable**. A **model-based reflex agent** handles partially observable environments by keeping an **internal state** built from the percept sequence and a **model** of the world: (i) how the world evolves on its own and (ii) how the agent's actions affect it.

മലയാളത്തിൽ

Simple reflex agent മുട്ടിൽ ചുറ്റിക കൊണ്ട് തട്ടുമ്പോൾ കാൽ ഉയരുന്ന reflex പോലെയാണ്: ചിന്തയില്ല, ഓർമ്മയില്ല, "ഇങ്ങനെയെങ്കിൽ ഇങ്ങനെ" മാത്രം. വഴിയിൽ പെട്ടെന്ന് ആൾ ചാടിയാൽ brake ചവിട്ടുന്നത് ഇതാണ്. Model-based agent-ന് ഓർമ്മയുണ്ട്: മുന്നിലെ lorry കാരണം signal കാണാൻ പറ്റുന്നില്ലെങ്കിലും, "ഒരു നിമിഷം മുമ്പ് പച്ചയായിരുന്നു" എന്ന് ഓർത്ത് തീരുമാനിക്കും. എല്ലാം കാണാൻ പറ്റാത്ത (partially observable) സ്ഥലത്ത് ഇതുതന്നെ വേണം.

Agent = Architecture + Agent program

Simple reflex: action = $f(\text{current percept})$

Model-based reflex: state = $\text{update}(\text{state}, \text{last action}, \text{percept}, \text{model})$; action = $f(\text{state})$

Simple reflex agent:

```
IF a human being comes in front of the automobile suddenly  
THEN apply brakes immediately
```

Model-based reflex agent:

```
state ← UPDATE-STATE(state, last action, percept, model)  
rule ← RULE-MATCH(state, rules)  
return rule.action
```

EXAMPLE: SIMPLE REFLEX OR MODEL-BASED?

(a) A thermostat that switches the heater on whenever the temperature is below 20 °C. (b) Grey Walter's *Machina Speculatrix* "tortoise", which moved towards dim light and away from bright light. (c) A car that keeps track of a motorbike that has just moved into its blind spot. (d) A robot vacuum that remembers which rooms it has already cleaned.

(a) Simple reflex: one condition-action rule on the current reading. (b) Simple reflex (the unit's example; a boundary-following robot is another). (c) Model-based: the bike is no longer visible, so the car relies on internal state and a model of how vehicles move. (d) Model-based: remembers parts of the world it cannot currently see.

EXAMPLE

Why does a simple reflex agent fail in a partially observable environment?

It sees only the current percept. If two different situations produce the same percept (a red light hidden behind a truck vs no signal at all), it must choose the same action for both, and one of them will be wrong. A model-based agent separates them using its remembered state.

Exam tip: The unit says agents fall into "five classes" but lists six, including stimulus-response agents. The standard five (Russell and Norvig) are simple reflex, model-based reflex, goal-based, utility-based and learning agents; stimulus-response (reactive) agents are Nilsson's term for essentially reflex-style agents. Note too that "SR" in Nilsson's book means stimulus-response; the unit reuses "SR" for simple reflex.

Goal-based, utility-based and stimulus-response agents

Goal-based agent: an agent that uses goal information, besides the current state, to choose actions whose outcomes lead to a goal.

Utility-based agent: an agent that uses a utility function, mapping each state to a degree of success, and chooses the action leading to the highest utility.

Stimulus-response agent: a reactive agent that maps sensor input (stimulus) to actuator output (response) with no internal symbolic model of its environment.

Goal-based agent: current state-ന് പുറമേ goal information-ഉം ഉപയോഗിച്ച്, ഫലം goal-ലേക്ക് നയിക്കുന്ന actions തിരഞ്ഞെടുക്കുന്ന agent.

Utility-based agent: ഓരോ state-നെയും ഒരു degree of success-ലേക്ക് map ചെയ്യുന്ന utility function ഉപയോഗിച്ച്, ഏറ്റവും ഉയർന്ന utility-യിലേക്ക് നയിക്കുന്ന action തിരഞ്ഞെടുക്കുന്ന agent.

Stimulus-response agent: environment-ന്റെ internal symbolic model ഇല്ലാതെ sensor input (stimulus)-നെ actuator output (response)-ലേക്ക് map ചെയ്യുന്ന reactive agent.

A **goal-based agent** uses goal information (desirable situations) in addition to the current state, and looks ahead at the outcomes of actions before choosing (for example, at a fork in the road, which branch leads to city B?). It may be slower because it reasons, but it is more flexible: change the goal and it adapts, whereas an SR agent needs its rules rewritten. It should also retract steps that led away from the goal. A **utility-based agent** assigns each state a number (its **utility**: degree of success or "happiness") through a **utility function**, and picks the reachable state with the highest utility. This lets it handle conflicting goals, partial success and choosing the faster or cheaper way. A **stimulus-response (reactive) agent** maps sensor input to actuator output, possibly through complex processing, but has no internal symbolic model of its environment; many simple reactive agents together can produce complex global behaviour.

മലയാളത്തിൽ

Goal-based agent-ന് ലക്ഷ്യം അറിയാം: "എറണാകുളത്ത് എത്തണം". റോഡ് രണ്ടായി പിരിയുമ്പോൾ ഏത് വഴി ലക്ഷ്യത്തിലേക്ക് എന്ന് നോക്കും. പക്ഷേ എത്രതുമോ ഇല്ലയോ എന്ന് മാത്രമേ അതിനറിയൂ. Utility-based agent ഒരുപടി കൂടി: "എത്രതും, പക്ഷേ ഏത് വഴി വേഗം, സുരക്ഷിതം, toll കുറവ്?" എന്ന് ഓരോ വഴിക്കും ഒരു score ഇട്ട് ഏറ്റവും നല്ലത് തിരഞ്ഞെടുക്കും. Racing game-ലെ car control ചെയ്യുന്ന agent റോഡ് നോക്കി നേരിട്ട് steering തിരിക്കുന്നു: അതാണ് stimulus-response agent.

Goal-based: chooses actions that lead to a goal state (yes / no)

Utility-based: utility function $U : \text{state} \rightarrow \text{degree of success}$; choose next state with maximum U

Stimulus-response: stimulus (sensors) $\rightarrow$ processing $\rightarrow$ response (actuators); no internal symbolic model

Agent	Uses	Memory / model	Strength	Limitation
Simple reflex	Current percept, condition-action rules	None	Simple, fast	Needs a fully observable world; cannot learn
Model-based reflex	Internal state + world model	Yes	Handles partial observability	No explicit goals
Goal-based	State + goal information, looks ahead	Yes	Flexible; adapts when the goal changes	Only goal / not goal; cannot rank ways or resolve conflicting goals
Utility-based	Utility function over states	Yes	Chooses best way; handles conflicting goals and partial success	Accurate utility function can be hard to define
Stimulus-response	Direct mapping stimulus $\rightarrow$ response	No internal symbolic model	Simple; complex group behaviour can emerge	Responds only to the present state
Learning	Feedback to improve itself	Knowledge grows over time	Works in unknown, changing environments	Needs experience; may try poor actions while exploring

EXAMPLE: GOAL OR UTILITY?

A navigation agent must reach the airport. Route X takes 40 min with a ₹150 toll; route Y takes 55 min with no toll. The passenger values 1 minute at ₹5. Which route does a utility-based agent pick, and how does a goal-based agent differ?

Cost of X = $150 + 40 \times 5 = ₹350$. Cost of Y = $0 + 55 \times 5 = ₹275$. Taking utility as the negative of cost, Y has the higher utility, so the utility-based agent picks Y. A goal-based agent only knows that both routes reach the airport; it has no built-in way to prefer one.

EXAMPLE (FROM THE UNIT)

A driver agent has the goal "do not hit the vehicle ahead". The vehicle ahead slows down. What does a goal-based agent do that a simple reflex agent would not?

It looks ahead: instead of braking hard (reflex), it brakes gradually so that the vehicles behind do not crash into it. This if-then-else reasoning about outcomes is what makes it goal-based.

EXAMPLE: CLASSIFY THE AGENT

(a) A swarm of simple robots, each just avoiding obstacles and following neighbours, that together form a line. (b) A GPS app that asks "Where to?" and computes a path. (c) A self-driving taxi that balances speed, safety, comfort and fuel.

(a) Stimulus-response (reactive) agents: complex group behaviour emerges. (b) Goal-based. (c) Utility-based (trade-off among several goals).

Learning agents

Definition: A learning agent starts with some built-in knowledge and improves its performance from experience, using four components: learning component, performance component, critic and problem generator.

കുറച്ച് built-in knowledge-കാടെ തുടങ്ങി experience-ൽ നിന്ന് performance മെച്ചപ്പെടുത്തുന്ന agent ആണ് learning agent; ഇതിന് learning component, performance component, critic, problem generator എന്നീ നാല് components ഉണ്ട്.

Not all knowledge can be coded in advance, especially in real, dynamic, multi-agent environments (a pedestrian can step in front of a car even on a green signal). A **learning agent** starts with some coded knowledge and improves it from experience. It has four components. The **learning component** improves performance using percepts and feedback. The **performance component** chooses the actions, using percepts and the knowledge from the learning component. The **critic** judges how well the agent is doing against a fixed performance standard and drives changes to the performance component. The **problem generator** suggests new, possibly sub-optimal actions to gain fresh experience, since short-run sub-optimal actions can pay off in the long run.

മലയാളത്തിൽ

Driving പഠിക്കുന്ന ഒരാളെ ഓർക്കുക. വണ്ടി ഓടിക്കുന്നത് performance component. അടുത്തിരിക്കുന്ന ആശാൻ "clutch നേരത്തെ വിട്ടു" എന്ന് പറയുന്നത് critic. ആ അഭിപ്രായം വെച്ച് അടുത്ത തവണ ശരിയാക്കുന്നത് learning component. "ഇന്ന് ഒരു കയറ്റത്തിൽ വണ്ടി നിർത്തി എടുത്തു നോക്ക്" എന്ന് പുതിയ അനുഭവം നിർദ്ദേശിക്കുന്നത് problem generator. എപ്പോഴും മാറിക്കൊണ്ടിരിക്കുന്ന റോഡിൽ പഠിക്കാത്ത driver agent-നെക്കൊണ്ട് കാര്യമില്ല.

Learning agent = Learning component + Performance component + Critic + Problem generator

Critic: compares behaviour with a fixed performance standard

Problem generator: proposes exploratory (possibly sub-optimal) actions

Learning taxonomies by: type of knowledge learnt · representation used · area of application

EXAMPLE: NAME THE COMPONENT

In an automated driver agent: (a) the module that decides to change lanes; (b) the module that notes a passenger complaint about harsh braking and scores the trip lower; (c) the module that adjusts the braking policy after such complaints; (d) the module that, facing a vehicle coming the wrong way on a one-way road, proposes an unusual manoeuvre to try.

(a) Performance component. (b) Critic. (c) Learning component. (d) Problem generator (the unit's own example of a new situation).

EXAMPLE

Why is the problem generator needed if it sometimes suggests worse actions?

Without it, the agent keeps repeating what it already knows and may never discover better options. A food-delivery agent that always uses the same road never learns that a new bypass is faster; trying it once (a possibly sub-optimal move) can improve every later trip.

Exam tip: The unit's summary says it gives "special emphasis on stimulus-response agents", but most marks in this area come from the learning agent's four components and the differences between agent types. Learn the comparison table in the previous topic.

Quick recap

- Knowledge is what we have learned; intelligence is the ability to use it. AI is incorporating intelligence into machines.
- Definitions of AI fall into four goals: thinking humanly, thinking rationally, acting humanly, acting rationally.

- AI technologies sense (vision, audio), comprehend (NLP, knowledge representation) and act (ML, expert systems). Software-level AI vs hardware (embodied) AI.
- By capability: ANI (weak, one task, exists today), AGI (strong, human-level, hypothetical), ASI (beyond human, speculative). By functionality: reactive, limited memory, theory of mind, self-aware.
- History: McCulloch–Pitts 1943, Turing 1950, Dartmouth 1956 (name "AI"), Logic Theorist, DENDRAL, Shakey, MYCIN 1974, Deep Blue 1997, Mars rovers 2004.
- Components of intelligence: learning, reasoning, problem-solving, perception, language comprehension.
- Five ways to learn: rote, instruction, analogy, deduction (certain), induction (probable).
- Feedback-based learning: supervised (labels), unsupervised (no labels), reinforcement (rewards), delayed reinforcement.
- Four approaches: Turing test (acting humanly), cognitive modelling (thinking humanly), laws of thought (thinking rationally), rational agent (acting rationally).
- Turing test: interrogator cannot tell machine from human. Chinese Room (Searle): imitating a quality is not having it.
- $DL \subseteq ML \subseteq AI$. ML learns from data; DL uses multi-layer neural networks with less manual pre-processing.
- Analytics: descriptive (what happened), predictive (what will happen), prescriptive (what to do).
- Agent: perceives via sensors, acts via actuators; agent = architecture + program. Task environment is specified by PEAS.
- Agent types: simple reflex, model-based reflex, goal-based, utility-based, stimulus-response (reactive), learning (learning component, performance component, critic, problem generator).

Check yourself

Q. 1. How does knowledge differ from intelligence? What do you understand by Artificial Intelligence? List the AI technologies and their illustrative solutions.

Knowledge is the information and abilities accumulated through experience; intelligence is the capacity to comprehend, analyse and apply that knowledge. Having knowledge alone does not make one intelligent ("marks are not the measure of intelligence").

AI is the field of building intelligence into machines: the ability of a machine to show human-like capabilities such as reasoning, learning, planning and creativity (e.g. Kurzweil: "the art of creating machines that perform functions that require intelligence when performed by people").

Technologies: sense (computer vision, audio processing), comprehend (NLP, knowledge representation), act (machine learning, expert systems). Illustrative solutions: virtual agents, identity analytics, cognitive robotics, speech analytics, recommendation systems, data visualisation.

Q.

2. Classify AI on the basis of capability and functionality. Compare ANI, AGI and ASI. Then classify as weak or strong AI: (a) Alexa setting a timer (b) a face-recognition gate at an airport (c) a machine that can learn any intellectual task a human can and is aware of itself.

By capability (Type 1): ANI, AGI, ASI. By functionality (Type 2): reactive machines, limited memory, theory of mind, self-awareness.

ANI (weak/narrow): specialises in one area, simulated thinking without awareness; all present-day AI (chatbots, spam filters). AGI (strong/general): as smart as a human across the board, with a conscious mind; not yet achieved. ASI: far smarter than the best human in practically every field; speculative. Stages: machine learning → machine intelligence → machine consciousness.

(a) Weak AI: narrow voice-command task. (b) Weak AI: one task. (c) Strong AI (AGI): general, human-level, self-aware.

Q. 3. Briefly discuss the components of intelligence.

Learning: rote, instruction, analogy, deduction, induction; generalisation to unseen cases is the hard part; feedback-based learning is supervised, unsupervised, reinforcement or delayed reinforcement.

Reasoning: drawing appropriate conclusions, deductively (certain) or inductively (probable); the difficulty is deciding what is relevant.

Problem-solving: special-purpose or general methods, e.g. means-end analysis reducing the gap between current and goal state.

Perception: analysing sensed scenes into objects, features and relations (e.g. FREDDY at Edinburgh).

Language comprehension: understanding signs with conventional meaning; human languages are productive (infinitely many sentences).

Q. 4. How do we measure whether AI makes a machine behave or act like a human? Which approach does each follow: (a) a system modelled on fMRI studies of how people make decisions (b) a program that proves theorems by strict logical rules (c) a robot that chooses the action with the best expected outcome?

Four approaches: Turing test (acting humanly), cognitive modelling (thinking humanly: introspection, psychological experiments, brain imaging), laws of thought (thinking rationally: logical rules implemented as algorithms), rational agent (acting rationally: best outcome in each situation).

(a) Thinking humanly (cognitive modelling, brain imaging). (b) Thinking rationally (laws of thought). (c) Acting rationally (rational agent).

Q. 5. What is the Turing test? What is the main criticism of it?

Proposed by Alan Turing in 1950. A machine (A) and a human (B) are in separate rooms; an interrogator (C) communicates with both only through terminals and questions them without knowing which is which. If C cannot identify the machine, the machine is deemed intelligent. The machine should not give itself away by, for instance, answering arithmetic impossibly fast.

Criticism: John Searle's Chinese Room. A person who knows no Chinese answers Chinese questions by looking up matching symbols (the unit's version: a Shakespeare encyclopedia with Chinese and English pages). Outsiders think he knows Chinese, but he does not. So passing the test by imitation does not prove real understanding: convincing others you have a quality is not having it.

Q. 6. Compare AI, ML and DL.

$DL \subseteq ML \subseteq AI$. AI is the broadest field: any technique (rules, search, logic, learning) that makes machines act intelligently, e.g. a rule-based expert system. ML is a subset of AI where algorithms learn patterns from data rather than being explicitly programmed, and improve with more data, e.g. a spam filter trained on labelled e-mails. DL is a subset of ML using multi-layer artificial neural networks; each layer learns more complex features, needing less pre-processing but large datasets and computation, e.g. face recognition, speech-to-text.

Q. 7. Compare descriptive, predictive and prescriptive analytics.

Descriptive: what happened; used across all industries (e.g. last quarter's sales report). Predictive: what will happen; probabilistic; used by data-driven organisations (e.g. forecasting demand). Prescriptive: what should be done to reach goals; most complex; used by leading data and internet companies (e.g. recommending how much stock to order). ML focuses on predictive and prescriptive analytics.

Q. 8. What are intelligent agents? Briefly discuss the properties of agents.

An agent perceives its environment through sensors and acts on it through actuators, usually on someone's behalf; many require it to be autonomous and goal-directed. Agent = architecture + agent program.

Properties: reactivity, autonomy, communicating ability, coexisting by cooperating, adaptability, drawing inferences, temporal continuity, personality, mobility (one line each, as in the table in topic 14).

Q. 9. What is a task environment? Explain PEAS with the automated road-transport driver example.

A task environment contains all the elements of the problem for which an agent is the solution. It is specified by PEAS.

P: safety, comfort, correct destination, minimum time within traffic rules, minimum disturbance to others, minimum cost. E: all kinds of roads and traffic rules, places, passengers and other vehicles (trucks, three-wheelers, bullock carts). A: steering, brakes, gears, accelerator, display, communication devices. S: odometer, speedometer, engine sensors, GPS, distance sensors.

Q. 10. Differentiate between a simple reflex agent and a model-based reflex agent, and between a goal-based and a utility-based agent. List the four components of a learning agent.

Simple reflex: no internal state; acts on the current percept via condition-action rules; works only in fully observable environments; cannot learn (e.g. brake when a person suddenly appears). Model-based reflex: keeps an internal state from the percept sequence plus a model of how the world evolves and how its actions affect it; handles partial observability.

Goal-based: uses goal information and looks ahead; knows only whether a state reaches the goal. Utility-based: a utility function maps states to degrees of success, so it can choose the best of several ways, handle partial success and resolve conflicting goals.

Learning agent: learning component (improves performance from percepts and feedback), performance component (selects actions), critic (judges performance against a fixed standard), problem generator (suggests new, possibly sub-optimal, actions to gain experience).

UNIT 2

Problem Solving Using Search

ഒരു AI problem solve ചെയ്യുന്നതിന് മുൻപ് അതിനെ ഒരു state space ആയി എഴുതണം: states, initial state, operators, goal. ഈ unit-ൽ water jug, 8-puzzle, N-Queens എന്നിവ state space ആയി formulate ചെയ്യുന്നതും, രണ്ട് കളിക്കാർ ഉള്ള games-ൽ Minimax, Alpha-Beta pruning എന്നിവ ഉപയോഗിക്കുന്നതും പഠിക്കാം.

Problem solving as search

State space: structure, formulation and solution

Example: the Water Jug problem

Formulation of the 8-puzzle problem

N-Queens problem: formulation

State space tree and backtracking for N-Queens

More formulations: Missionaries and Cannibals, Tower of Hanoi

Adversarial (two-agent) search and game elements

Minimax search strategy

Alpha-Beta pruning

Move ordering in Alpha-Beta pruning

Problem solving as search

Definition: Search is the process of applying rules (operators) under a control strategy to find a path from an initial state to a goal state in the problem's state space.

problem-ന്റെ state space-ൽ initial state-ൽ നിന്ന് goal state-ലേക്കുള്ള path കണ്ടെത്താൻ ഒരു control strategy അനുസരിച്ച് rules (operators) apply ചെയ്യുന്ന process ആണ് search.

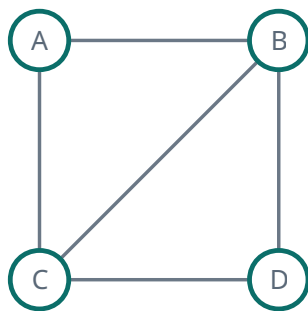
Some problems have a direct, guaranteed procedure (like adding two numbers). Most real-world AI problems do not; we must **search** for a solution. To build a problem solver we (1) define the problem precisely (initial and final configurations), (2) analyse it, (3) isolate and represent the task knowledge, and (4) choose and apply the best technique.

A formal description needs four things: a state space of all possible configurations, one or more initial states, one or more goal states, and a set of rules (operators). Applying the rules with a **control strategy** until a path from an initial state to a goal state is found is called **search**.

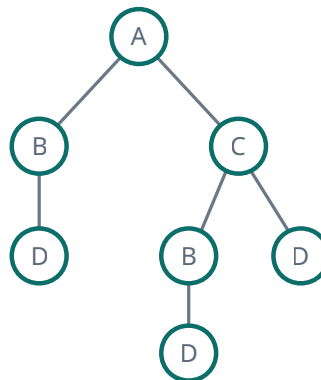
മലയാളത്തിൽ

ചില problems-ന് നേരിട്ടുള്ള formula ഉണ്ട്; പക്ഷേ മിക്ക AI problems-ലും വഴി തിരഞ്ഞു കണ്ടുപിടിക്കണം. ഇതിനെയാണ് search എന്ന് പറയുന്നത്. ആദ്യം എവിടെ നിന്ന് തുടങ്ങുന്നു (initial state), എവിടെ എത്തണം (goal state), ഓരോ step-ലും എന്ത് ചെയ്യാം (operators) എന്ന് കൃത്യമായി എഴുതണം. പുതിയ ഒരു നഗരത്തിൽ Google Maps ഇല്ലാതെ വഴി ചോദിച്ച് ചോദിച്ച് പോകുന്നത് പോലെയാണ് search.

A problem space is a **directed graph**: nodes are states and arcs are operators. It is often easier to unfold the graph into a **tree**. The price is duplication: a node reachable by several paths in the graph appears several times in the tree. A tree is a connected graph with no cycles, so any two vertices are joined by exactly one path.



Graph: B and D are reachable by more than one path.



Same problem as a tree: B and D get duplicated.

മലയാളത്തിൽ

Graph-ൽ ഒരു node-ലേക്ക് പല വഴികൾ ഉണ്ടാകാം. Tree ആക്കുമ്പോൾ ഓരോ വഴിക്കും node-ന്റെ ഒരു copy വരും, അതുകൊണ്ട് B, D പല തവണ കാണാം. Tree-യിൽ algorithm എഴുതാൻ എളുപ്പമാണ്, പക്ഷേ ചില nodes ആവർത്തിക്കും.

Exam tip: "Why do we represent a graph as a tree?" Answer: it simplifies the search algorithm; the cost is that nodes reachable by many paths are duplicated.

State space: structure, formulation and solution

State: a snapshot (configuration) of the problem at one moment.

State space: the set of all states reachable from the initial state by any sequence of actions, written $[S, s_0, O, G]$.

Solution: a path (sequence of states) from the initial state to a goal state.

State: ഒരു നിമിഷത്തിലെ problem-ന്റെ snapshot (configuration).

State space: ഏതെങ്കിലും actions sequence വഴി initial state-ൽ നിന്ന് എത്താവുന്ന എല്ലാ states-ന്റെയും set; $[S, s_0, O, G]$ എന്ന് എഴുതുന്നു.

Solution: initial state-ൽ നിന്ന് goal state-ലേക്കുള്ള ഒരു path (states-ന്റെ sequence).

A **state** is a snapshot of the problem at one moment. The **state space** is the set of all states reachable from the initial state by some sequence of actions. It forms a graph (or tree): nodes are states, arcs are legal moves. A **path** is a sequence of states connected by actions.

State space $S_s = [S, s_0, O, G]$

S = set of all possible states

s_0 = start (initial) state, $s_0 \in S$

O = set of operators / production rules (the arcs), each written as "condition $\rightarrow$ action"

G = set of goal states, $G \subseteq S$

Solution path = $s_0, s_1, s_2, \dots, s_n$ with $s_n \in G$

Path cost = sum of the costs of the actions on the path; optimal solution = lowest path cost

മലയാളത്തിൽ

State space-നെ നാല് കാര്യങ്ങൾ കൊണ്ട് എഴുതാം: എല്ലാ states-ന്റെയും set S , തുടക്കം s_0 , മാറ്റം വരുത്താനുള്ള rules O , ലക്ഷ്യം G . ഓരോ production rule-നും രണ്ട് ഭാഗം ഉണ്ട്: ഇടത് ഭാഗം "എപ്പോൾ ഈ rule ഉപയോഗിക്കാം" (condition), വലത് ഭാഗം "എന്ത് മാറ്റം വരും" (action). ഏറ്റവും കുറഞ്ഞ path cost ഉള്ള solution ആണ് optimal solution.

Structure. In a tree there is exactly one path between any two nodes; in a graph there can be several. **Solution.** A solution is a path from the initial state to a goal state (sometimes just the goal state itself). **Searching.** We start at s_0 and expand it: apply the successor function to get all its children. Then we pick one child to expand next and keep the others aside for later. Which child is picked first is decided by the **search strategy** (for example DFS or BFS). When a goal is reached, we trace back through parent links to read off the solution path.

EXAMPLE

A lift in a 3-storey building. Formulate it as a state space and find the cheapest way from floor 0 to floor 2.

$S = \{0, 1, 2\}$ (current floor). $s_0 = 0$. $O = \{\text{UP: } (f \mid f < 2) \rightarrow f + 1, \text{DOWN: } (f \mid f > 0) \rightarrow f - 1\}$. $G = \{2\}$. Cost of each move = 1.

Expand 0 $\rightarrow \{1\}$. Expand 1 $\rightarrow \{0, 2\}$. $2 \in G$, so stop. Trace back: $0 \rightarrow 1 \rightarrow 2$, path cost 2.

Exam tip: Whenever a question says "formulate", write all five parts in this order: states, initial state, operators (with conditions), goal test, path cost. Missing the conditions on operators is the most common way to lose marks.

Example: the Water Jug problem

Definition: The Water Jug problem: using an unmarked 4-gallon jug, an unmarked 3-gallon jug and a pump, get exactly 2 gallons into the 4-gallon jug; states are (x, y) , from $(0, 0)$ to $(2, n)$.

Water Jug problem: അളവ് അടയാളമില്ലാത്ത ഒരു 4-gallon jug-ഉം 3-gallon jug-ഉം ഒരു pump-ഉം ഉപയോഗിച്ച് 4-gallon jug-ൽ കൃത്യം 2 gallon വെള്ളം എത്തിക്കുക; states (x, y) ആണ്, $(0, 0)$ -ൽ നിന്ന് $(2, n)$ -ലേക്ക്.

You have a 4-gallon jug and a 3-gallon jug with no markings, and a pump. How can you get exactly 2 gallons into the 4-gallon jug?

States: (x, y) , x = water in 4-gal jug $\in \{0, 1, 2, 3, 4\}$, y = water in 3-gal jug $\in \{0, 1, 2, 3\}$

Initial state: $(0, 0)$

Goal test: $(2, n)$ for any $n \in \{0, 1, 2, 3\}$

Operators: rules R1 to R10 below

Path cost: 1 per rule applied

മലയാളത്തിൽ

(x, y) എന്നതിൽ x = 4 gallon jug-ലെ വെള്ളം, y = 3 gallon jug-ലെ വെള്ളം. അളവ് അടയാളം ഇല്ലാത്തതുകൊണ്ട് നമുക്ക് ചെയ്യാവുന്നത് നിറയ്ക്കുക, കാലിയാക്കുക, ഒന്നിൽ നിന്ന് മറ്റൊന്നിലേക്ക് ഒഴിക്കുക എന്നിവ മാത്രം. വീട്ടിൽ അളവുപാത്രം ഇല്ലാതെ ഒരു ചെറിയ ഗ്ലാസും വലിയ കുപ്പിയും വെച്ച് കൃത്യം 2 ലിറ്റർ എടുക്കാൻ ശ്രമിക്കുന്നത് പോലെ.

Rule	Condition $\rightarrow$ Result	Meaning
R1	$(x, y \mid x < 4) \rightarrow (4, y)$	Fill the 4-gal jug
R2	$(x, y \mid y < 3) \rightarrow (x, 3)$	Fill the 3-gal jug
R3	$(x, y \mid x > 0) \rightarrow (0, y)$	Empty the 4-gal jug
R4	$(x, y \mid y > 0) \rightarrow (x, 0)$	Empty the 3-gal jug
R5	$(x, y \mid x + y \geq 4 \text{ and } y > 0) \rightarrow (4, y - (4 - x))$	Pour 3 $\rightarrow$ 4 until the 4-gal jug is full
R6	$(x, y \mid x + y \geq 3 \text{ and } x > 0) \rightarrow (x - (3 - y), 3)$	Pour 4 $\rightarrow$ 3 until the 3-gal jug is full
R7	$(x, y \mid x + y \leq 4 \text{ and } y > 0) \rightarrow (x + y, 0)$	Pour all of the 3-gal jug into the 4-gal jug
R8	$(x, y \mid x + y \leq 3 \text{ and } x > 0) \rightarrow (0, x + y)$	Pour all of the 4-gal jug into the 3-gal jug
R9	$(x, y \mid x > 0) \rightarrow (x - d, y)$	Pour some water d out of the 4-gal jug
R10	$(x, y \mid y > 0) \rightarrow (x, y - d)$	Pour some water d out of the 3-gal jug

EXAMPLE (SOLUTION 1 OF THE UNIT)

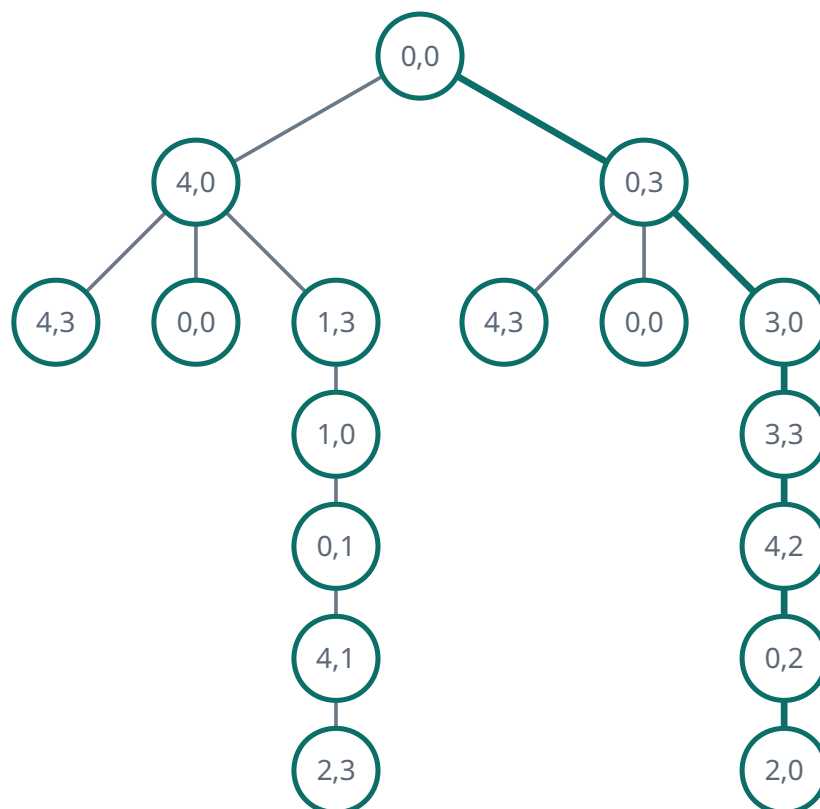
Step	(x, y)	Rule
0	(0, 0)	start
1	(4, 0)	R1 fill 4-gal
2	(1, 3)	R6 pour 4 $\rightarrow$ 3 until 3-gal is full
3	(1, 0)	R4 empty 3-gal
4	(0, 1)	R8 pour all of 4-gal into 3-gal
5	(4, 1)	R1 fill 4-gal
6	(2, 3)	R6 pour 4 $\rightarrow$ 3 until full (only 2 gallons fit)
7	(2, 0)	R4 empty 3-gal (optional)

The goal test $(2, n)$ is already true at step 6, so 6 moves are enough. Step 7 only tidies up.

EXAMPLE (SOLUTION 2 OF THE UNIT)

Step	(x, y)	Rule
0	(0, 0)	start
1	(0, 3)	R2 fill 3-gal
2	(3, 0)	R7 pour all of 3-gal into 4-gal
3	(3, 3)	R2 fill 3-gal
4	(4, 2)	R5 pour 3 → 4 until 4-gal is full (only 1 fits)
5	(0, 2)	R3 empty 4-gal
6	(2, 0)	R7 pour all of 3-gal into 4-gal

6 moves. A breadth-first search over all 14 reachable states confirms that 6 is the minimum number of moves.



State space tree (after the unit's Fig. 5). Highlighted: Solution 2. The left branch through (1, 3) is Solution 1.

Exam tip: Rules R9 and R10 ("pour out some amount d") are listed for completeness but are useless here, because without markings you cannot measure d. When checking any step, test the rule's condition first: R6 on (4, 1) is allowed because $4 + 1 \geq 3$ and $x > 0$, and gives $(4 - 2, 3) = (2, 3)$.

Formulation of the 8-puzzle problem

Definition: The 8-puzzle is a 3×3 board with eight numbered tiles and one blank, where a tile next to the blank can slide into it; the task is to reach a goal arrangement from a start arrangement.

എട്ട് numbered tiles-ഉം ഒരു blank-ഉം ഉള്ള 3×3 board ആണ് 8-puzzle; blank-ന് അടുത്തുള്ള tile-ന് അതിലേക്ക് നീങ്ങാം; start arrangement-ൽ നിന്ന് goal arrangement-ൽ എത്തുകയാണ് task.

A 3×3 frame holds eight numbered tiles and one blank. A tile next to the blank can slide into it. The aim is to find a sequence of moves from a start configuration to a goal configuration. It is easiest to think of the **blank** as the thing that moves.

States: any arrangement of tiles 1 to 8 and the blank on the 9 cells

Initial state: a given arrangement

Operators: move the blank UP, DOWN, LEFT, RIGHT (only if it stays on the board)

Goal test: the arrangement equals the given goal arrangement

Path cost: 1 per blank move, so cost = number of moves; optimal = fewest moves

മലയാളത്തിൽ

Tile-കൾ ഓരോന്നും നീക്കുന്നു എന്ന് ചിന്തിക്കുന്നതിനേക്കാൾ എളുപ്പം blank ആണ് നീങ്ങുന്നത് എന്ന് കരുതുന്നതാണ്. അപ്പോൾ നാല് operators മാത്രം: blank UP, DOWN, LEFT, RIGHT. മൂലയിൽ blank ആണെങ്കിൽ 2 moves, അരികിൽ 3, നടുവിൽ 4. ഓരോ move-നും cost 1, അതിനാൽ ഏറ്റവും കുറച്ച് moves ഉള്ള വഴിയാണ് optimal.

EXAMPLE (FROM THE UNIT)

Start and goal (_ is the blank):

Start	Goal
3 1 2	_ 1 2
4 _ 5	3 4 5
6 7 8	6 7 8

Step 1, expand the start. The blank is in the centre, so all four moves are legal:

UP	DOWN	LEFT	RIGHT
3 _ 2	3 1 2	3 1 2	3 1 2
4 1 5	4 7 5	_ 4 5	4 5 _
6 7 8	6 _ 8	6 7 8	6 7 8

Step 2, none of these is the goal. Expand LEFT; its child "blank UP" is

```
_ 1 2
3 4 5
6 7 8
```

which is the goal. Solution: LEFT, UP. Path cost = 2 (verified minimal by breadth-first search).

EXAMPLE (Q.1 OF THE UNIT)

Find the minimum cost path from {(1,2,3),(4,8,_),(7,6,5)} to {(1,2,3),(4,5,6),(7,8,_)}.

Start	DOWN	LEFT	UP	RIGHT	DOWN = Goal
1 2 3	1 2 3	1 2 3	1 2 3	1 2 3	1 2 3
4 8 _	4 8 5	4 8 5	4 _ 5	4 5 _	4 5 6
7 6 5	7 6 _	7 _ 6	7 8 6	7 8 6	7 8 _

Tile view: 5 moves up, 6 moves right, 8 moves down, 5 moves left, 6 moves up. Path cost = 5, and breadth-first search confirms no 4-move solution exists.

Exam tip: When you draw the state space tree for an 8-puzzle question, do not generate the move that just undoes the parent's move (it recreates the grandparent). Write the operator name on every arc and state the final path cost.

N-Queens problem: formulation

Definition: The N-Queens problem is to place N queens on an $N \times N$ chessboard so that no two queens attack each other, i.e. no two share a row, column or diagonal.

രണ്ട് queens പരസ്പരം attack ചെയ്യാത്ത വിധം, അതായത് ഒരേ row-ലോ column-ലോ diagonal-ലോ രണ്ടെണ്ണം വരാത്ത വിധം, $N \times N$ chessboard-ൽ N queens വയ്ക്കുന്നതാണ് N-Queens problem.

Place N queens on an $N \times N$ chessboard so that no two attack each other: no two queens in the same **row**, **column** or **diagonal**. The puzzle was posed in 1848 by the chess player Max Bezzel; Gauss later worked on it. Solutions that differ only by rotation or reflection count as one "unique" solution.

Formulation (4-Queens, incremental):

States: any arrangement of 0 to 4 queens on the board

Initial state: empty board (0 queens)

Successor function: add a queen to any empty square

Goal test: 4 queens on the board, none attacked

Path cost: not important (every solution has depth 4)

മലയാളത്തിൽ

N queens-നെ ഒരേ row, column, diagonal-ൽ വരാതെ വെക്കണം. കല്യാണ സദ്യയ്ക്ക് പരസ്പരം പിണങ്ങി നിൽക്കുന്ന ബന്ധുക്കളെ ഒരേ വരിയിലോ ഒരേ നിരയിലോ ചരിഞ്ഞ വരിയിലോ ഇരുത്താതെ seat കൊടുക്കുന്നത് പോലെ. ഒരു solution-നെ തിരിച്ചോ mirror ചെയ്തോ കിട്ടുന്നവ ഒന്നായി കണക്കാക്കിയാൽ അത് unique solutions.

EXAMPLE

How many ways are there to put 4 queens on a 4×4 board without any restriction, and for 8 queens on 8×8 ?

$${}^{16}C_4 = 16!/(4! 12!) = 1820.$$

${}^{64}C_8 = 64!/(8! 56!) = 4,426,165,368 \approx 4.4 \times 10^9$. This is why blind placement is too expensive and we restrict the search.

N	1	2	3	4	5	6	7	8	9	10
Distinct solutions	1	0	0	2	10	4	40	92	352	724
Unique solutions	1	0	0	1	2	1	6	12	46	92

EXAMPLE (THE UNIT'S 8-QUEENS SOLUTION)

The 8-tuple (4, 6, 8, 2, 7, 1, 3, 5) means the queen of row i is in column x_i . Check: all eight columns are different, and for every pair of rows i, k we have $|x_i - x_k| \neq |i - k|$. For example rows 1 and 4: $|4 - 2| = 2$ but $|1 - 4| = 3$, safe. All 28 pairs pass, so it is a valid solution.

Exam tip: Learn "4 queens: 2 solutions (1 unique); 8 queens: 92 distinct, 12 unique". There is no known formula for the number of solutions for general N . The unit's full table (up to $N = 26$) prints 14,772,184 distinct solutions for $N = 16$; the correct value is 14,772,512.

State space tree and backtracking for N-Queens

State space tree: a tree whose nodes are partial solutions (queens placed row by row) and whose root-to-leaf paths are all candidate solutions.

Backtracking: a depth-first search of the state space tree that abandons a node as soon as it is non-promising (breaks a constraint) and returns to its parent to try the next option.

State space tree: nodes partial solutions (row by row ആയി വച്ച queens) ആയും root-ൽ നിന്ന് leaf വരെയുള്ള paths എല്ലാ candidate solutions ആയും ഉള്ള tree.

Backtracking: ഒരു node non-promising (ഒരു constraint ലംഘിക്കുന്നു) ആകുമ്പോൾ തന്നെ അത് ഉപേക്ഷിച്ച് parent-ലേക്ക് തിരിച്ചുപോയി അടുത്ത option പരീക്ഷിക്കുന്ന, state space tree-യുടെ depth-first search.

To shrink the search, place queens **row by row** (Q_i in row i) and write the solution as $(x_1, \dots, x_n)$, x_i = column of Q_i . If we only forbid repeated columns (diagonals allowed), the tree is a **permutation tree**: $1 + 4 + 4 \cdot 3 + 4 \cdot 3 \cdot 2 + 4 \cdot 3 \cdot 2 \cdot 1 = 65$ nodes and $4! = 24$ leaves. **Backtracking** also checks diagonals at every step: a node that breaks a rule is **non-promising** (marked $\times$) and its subtree is never generated; we go back to the parent and try the next column.

Left diagonal (upper-left to lower-right): same (row - column)

Right diagonal (upper-right to lower-left): same (row + column)

Queens at (i, j) and (k, l) share a diagonal $\Leftrightarrow |j - l| = |i - k|$

മലയാളത്തിൽ

Backtracking എന്നാൽ: ഒരു row-ൽ queen വെക്കുക, അടുത്ത row-ലേക്ക് പോകുക; എവിടെയും വെക്കാൻ പറ്റില്ലെങ്കിൽ തിരിച്ചു പോയി മുൻപത്തെ queen-നെ അടുത്ത column-ലേക്ക് മാറ്റുക. ഷർട്ടിന്റെ ബട്ടൺ ഇടുമ്പോൾ ഒരേണ്ണം തെറ്റിയാൽ അവിടെ വരെ അഴിച്ച് വീണ്ടും ഇടുന്നത് പോലെ. Diagonal check-ന് ഒരു ലളിതമായ test മതി: $|column\ വ്യത്യാസ| = |row\ വ്യത്യാസ|$ ആണെങ്കിൽ ഒരേ diagonal.

```
Algorithm NQueen(k, n)           // place queens in rows k..n
  for i = 1 to n do
    if place(k, i) then
      x[k] = i
      if (k == n) then print(x[1..n])
      else NQueen(k + 1, n)

Algorithm place(k, i)           // can queen k go in column i?
  for j = 1 to k - 1 do
    if (x[j] == i)                // same column
      or (abs(x[j] - i) == abs(j - k)) // same diagonal
      return false
  return true
```

EXAMPLE: BACKTRACKING TRACE FOR 4-QUEENS (FIRST SOLUTION)

(r, c) = try row r, column c. "ok" = promising, "x" = non-promising.

```
(1,1) ok
  (2,1) x  same column
  (2,2) x  same diagonal
  (2,3) ok
    (3,1) x (3,2) x (3,3) x (3,4) x  dead end, backtrack
  (2,4) ok
    (3,1) x
    (3,2) ok
      (4,1) x (4,2) x (4,3) x (4,4) x  dead end
    (3,3) x (3,4) x                    backtrack to row 1
(1,2) ok
  (2,1) x (2,2) x (2,3) x
  (2,4) ok
    (3,1) ok
      (4,1) x (4,2) x
    (4,3) ok  → solution (2, 4, 1, 3)
```

Continuing the same procedure gives the second solution (3, 1, 4, 2), its mirror image. In total the full run tests 60 (row, column) positions instead of the 1820 blind placements.

Solution (2,4,1,3)	Solution (3,1,4,2)
. Q . .	. . Q .
. . . Q	Q . . .
Q . . .	. . . Q
. . Q .	. Q . .

EXAMPLE: USING PLACE()

$x[1] = 2, x[2] = 4$. Can Q_3 go in column 3?

$j = 1: x[1] = 2 \neq 3; |2 - 3| = 1, |1 - 3| = 2$, not equal. OK.

$j = 2: x[2] = 4 \neq 3; |4 - 3| = 1, |2 - 3| = 1$, equal, so same diagonal. place returns false.

Exam tip: In its walkthrough of Fig. 15 the unit says (2, 2) is non-promising "due to same column". It is actually rejected because it is on the same diagonal as (1, 1). Also, the answer to Check Your Progress 1, Q5 writes the solution as {2, 4, 3, 1}. That is not valid (rows 2 and 3 would share a diagonal); the correct solution is (2, 4, 1, 3).

More formulations: Missionaries and Cannibals, Tower of Hanoi

Missionaries and Cannibals: move 3 missionaries and 3 cannibals across a river in a boat carrying one or two people, so that cannibals never outnumber missionaries on a bank where missionaries are present.

Tower of Hanoi: move n disks from peg A to peg C using peg B, moving one top disk at a time and never placing a larger disk on a smaller one.

Missionaries and Cannibals: ഒന്നോ രണ്ടോ പേരെ കൊണ്ടുപോകുന്ന boat-ൽ 3 missionaries-നെയും 3 cannibals-നെയും നദി കടത്തുക; missionaries ഉള്ള ഒരു കരയിലും cannibals എണ്ണത്തിൽ കൂടരുത്.

Tower of Hanoi: peg B ഉപയോഗിച്ച് n disks peg A-ൽ നിന്ന് peg C-ലേക്ക് മാറ്റുക; ഒരു സമയം ഏറ്റവും മുകളിലെ ഒരു disk മാത്രം നീക്കാം, വലിയ disk ഒരിക്കലും ചെറിയതിന് മുകളിൽ വയ്ക്കരുത്.

Check Your Progress 1 asks you to formulate two more classic puzzles. The method is the same: choose a compact state, then list the operators with their conditions.

Missionaries and Cannibals

State: (M, C, B): M, C = missionaries, cannibals on the left bank; B = 0 boat on left, 1 boat on right

Initial state: (3, 3, 0) Goal state: (0, 0, 1)

Operators: the boat carries $(m, c) \in \{(1,0), (2,0), (0,1), (0,2), (1,1)\}$ across; subtract from the left bank if B = 0, add if B = 1

Condition (safe state): on each bank, missionaries = 0 or missionaries $\geq$ cannibals

Path cost: 1 per crossing

മലയാളത്തിൽ

State-ൽ ഇടത്തെ കരയിലുള്ളവരുടെ എണ്ണം മാത്രം എഴുതിയാൽ മതി; വലത്തെ കരയിൽ ഉള്ളത് $3 - M$, $3 - C$. ഒരു കരയിലും missionaries-നെക്കാൾ കൂടുതൽ cannibals ഉണ്ടാകാൻ പാടില്ല (missionaries ആരും ഇല്ലെങ്കിൽ പ്രശ്നമില്ല). തോണിയിൽ ഒന്നോ രണ്ടോ പേർ. കടത്തുകടവിൽ വള്ളക്കാരൻ ആളുകളെ സുരക്ഷിതമായി അക്കരെ എത്തിക്കുന്നത് പോലെ.

EXAMPLE: SOLVING MISSIONARIES AND CANNIBALS

Crossing	Boat carries	State (M, C, B)	Left bank	Right bank
0	–	(3, 3, 0)	3M 3C	–
1	2C →	(3, 1, 1)	3M 1C	2C
2	1C ←	(3, 2, 0)	3M 2C	1C
3	2C →	(3, 0, 1)	3M	3C
4	1C ←	(3, 1, 0)	3M 1C	2C
5	2M →	(1, 1, 1)	1M 1C	2M 2C
6	1M 1C ←	(2, 2, 0)	2M 2C	1M 1C
7	2M →	(0, 2, 1)	2C	3M 1C
8	1C ←	(0, 3, 0)	3C	3M
9	2C →	(0, 1, 1)	1C	3M 2C
10	1M ←	(1, 1, 0)	1M 1C	2M 1C
11	1M 1C →	(0, 0, 1)	–	3M 3C

Every row is safe on both banks. 11 crossings is the minimum (checked by breadth-first search; only 16 safe states are reachable). The last two crossings can also be done as "1C ←, 2C →"; both give 11.

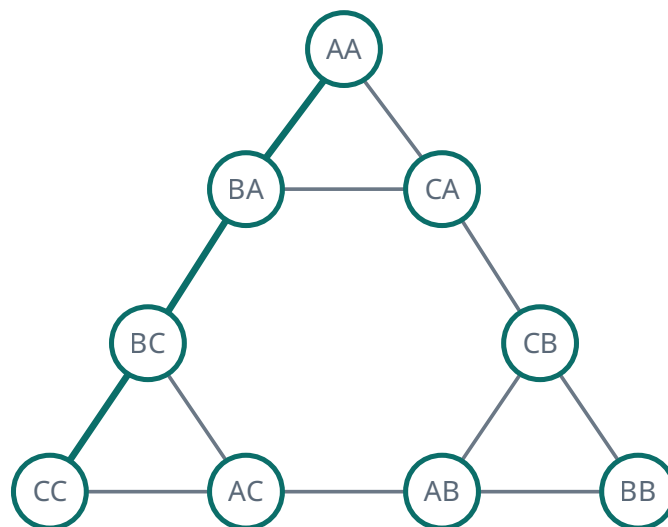
Tower of Hanoi ($n = 2$, pegs A, B, C)

State: (peg of small disk, peg of large disk), e.g. AA

Initial state: AA Goal state: CC

Operators: move the top disk of one peg to another peg, never placing the large disk on the small one

Path cost: 1 per move. State space has 3^n states; minimum moves = $2^n - 1$



Hanoi state space for $n = 2$ (9 states). Highlighted: $AA \rightarrow BA \rightarrow BC \rightarrow CC$, 3 moves.

EXAMPLE: SOLVING TOWER OF HANOI, $N = 2$

AA: move small A $\rightarrow$ B, giving BA. Move large A $\rightarrow$ C (allowed, C is empty), giving BC. Move small B $\rightarrow$ C onto the large disk, giving CC = goal. 3 moves = $2^2 - 1$.

For $n = 3$ the graph has $3^3 = 27$ states and the shortest solution needs $2^3 - 1 = 7$ moves.

Exam tip: For Missionaries and Cannibals, always state the safety condition on both banks and the boat-position bit; without the bit, (3, 3) at the start and (3, 3) with the boat across would be treated as the same state.

Adversarial (two-agent) search and game elements

Definition: Adversarial search is search in a competitive environment where two agents (players) have conflicting goals, so one player's win is the other's loss, as in chess or tic-tac-toe.

രണ്ട് agents (players)-ന് പരസ്പരവിരുദ്ധമായ goals ഉള്ള, ഒരാളുടെ ജയം മറ്റേയാളുടെ തോൽവിയാകുന്ന competitive environment-ലെ search ആണ് adversarial search; ഉദാ. chess, tic-tac-toe.

Adversarial search is search in a competitive environment where two agents have conflicting goals, as in tic-tac-toe, chess or checkers (games of skill, no luck). It is based on game theory: one player's win is the other's loss. Two

techniques make it practical: **pruning** (ignore parts of the tree that cannot change the result) and a **heuristic evaluation function** (estimate the value of a position before the game ends).

Elements of a game:

S_0 : initial state

PLAYER(s): whose turn it is in state s

ACTIONS(s): legal moves in s

RESULT(s, a): state after move a (transition model)

TERMINAL-TEST(s): true if the game is over

UTILITY(s, p): final numeric value (payoff / objective function), e.g. +1 win, 0 draw, -1 loss

മലയാളത്തിൽ

സാധാരണ search-ൽ നമ്മൾ മാത്രമാണ് തീരുമാനം എടുക്കുന്നത്. പക്ഷേ game-ൽ എതിരാളി നമ്മളെ തോൽപ്പിക്കാൻ ശ്രമിക്കും, അതിനാൽ ഓരോ move-ഉം എതിരാളിയുടെ ഏറ്റവും നല്ല മറുപടി കണക്കിലെടുത്ത് വേണം തിരഞ്ഞെടുക്കാൻ. ചെസ്സ് കളിക്കുമ്പോൾ "ഞാൻ ഇത് ചെയ്താൽ അവൻ എന്ത് ചെയ്യും" എന്ന് ആലോചിക്കുന്നത് തന്നെ ഇത്. MAX (X) score കൂട്ടാനും MIN (O) score കുറയ്ക്കാനും ശ്രമിക്കുന്നു.

In the tic-tac-toe game tree, the root is the empty board with 9 blanks and MAX (X) to move; level 1 has 8 blanks and MIN (O) to move; level 2 has 7 blanks, and so on. Utilities are written from MAX's point of view.

Tic-tac-toe as a game:

S_0 : empty 3×3 board

PLAYER(s): MAX (X) if the counts of X and O are equal, else MIN (O)

ACTIONS(s): put your mark in any empty cell

RESULT(s, a): the board with that cell filled

TERMINAL-TEST(s): some line of three is complete, or the board is full

UTILITY: +1 if X wins, -1 if O wins, 0 for a draw

EXAMPLE: A SMALL TIC-TAC-TOE GAME TREE

Cells are numbered 1 to 9 row by row. X (MAX) to move with three empty cells:

X		0		X
X		0		6
0		8		9

O threatens the middle column (cells 2, 5, 8). Evaluate every X move:

X plays	O's replies (MIN)	Outcome	Backed-up value
6	O 8 wins column 2-5-8 (-1); O 9 then X 8 draws (0)	MIN picks -1	-1
8	O 6, X 9 draw (0); O 9, X 6 draw (0)	MIN picks 0	0
9	O 8 wins (-1); O 6, X 8 draw (0)	MIN picks -1	-1

MAX takes $\max(-1, 0, -1) = 0$: X must block at cell 8, and the game is a draw with best play.

Exam tip: Listing the six elements (S_0 , PLAYER, ACTIONS, RESULT, TERMINAL-TEST, UTILITY) with a tic-tac-toe example is a standard 5-mark answer.

Minimax search strategy

Definition: Minimax is a depth-first decision rule for two-player zero-sum games in which MAX picks the move with the highest backed-up value and MIN the lowest, each assuming the opponent plays optimally.

two-player zero-sum games-ഈ depth-first decision rule ആണ് minimax; എതിരാളി optimal ആയി കളിക്കുമെന്ന് കരുതി MAX ഏറ്റവും ഉയർന്ന backed-up value ഉള്ള move-ഉം MIN ഏറ്റവും താഴ്ന്നതും തിരഞ്ഞെടുക്കുന്നു.

Minimax is a backtracking, depth-first decision rule for two-player zero-sum games. MAX picks the move with the highest value, MIN the one with the lowest, and each assumes the opponent plays perfectly. Values are computed at the leaves by the utility (or heuristic) function and backed up level by level to the root. By convention the root is MAX. Initially a MAX node's running value is $-\infty$ and a MIN node's is $+\infty$.

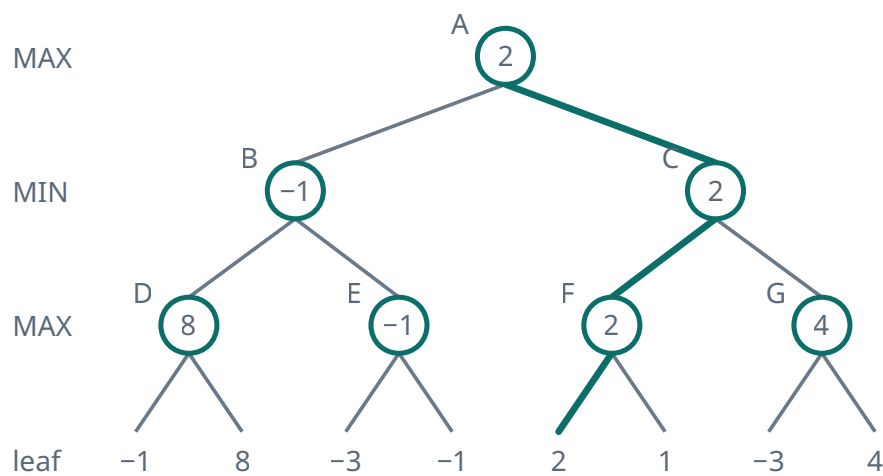
MINIMAX(s) = UTILITY(s) if s is terminal

MINIMAX(s) = max over children c of MINIMAX(c) if PLAYER(s) = MAX

MINIMAX(s) = min over children c of MINIMAX(c) if PLAYER(s) = MIN

മലയാളത്തിൽ

ഇലകളിൽ (leaf nodes) ഉള്ള values താഴെ നിന്നുമുകളിലേക്ക് കൊണ്ടുവരുന്നു. MAX level-ൽ children-ന്റെ ഏറ്റവും വലിയ value, MIN level-ൽ ഏറ്റവും ചെറിയ value. എതിരാളി എപ്പോഴും ഏറ്റവും നല്ല move കളിക്കും എന്ന് കരുതി, ഏറ്റവും മോശം അവസ്ഥയിലും നമുക്ക് കിട്ടാവുന്ന ഏറ്റവും നല്ല ഫലം ആണ് Minimax തരുന്നത്. DFS രീതിയിൽ ഇടത്തു നിന്ന് വലത്തോട്ട് പോകുന്നു.



Minimax Example 1 from the unit. Highlighted: MAX's optimal path $A \rightarrow C \rightarrow F \rightarrow 2$.

EXAMPLE 1 (FROM THE UNIT): STEP-BY-STEP VALUES IN DFS ORDER

Node	Level	Children seen	Computation	Value
D	MAX	-1, 8	$\max(-\infty, -1) = -1, \max(-1, 8) = 8$	8
E	MAX	-3, -1	$\max(-\infty, -3) = -3, \max(-3, -1) = -1$	-1
B	MIN	D = 8, E = -1	$\min(8, -1)$	-1
F	MAX	2, 1	$\max(2, 1)$	2
G	MAX	-3, 4	$\max(-3, 4)$	4
C	MIN	F = 2, G = 4	$\min(2, 4)$	2
A	MAX	B = -1, C = 2	$\max(-1, 2)$	2

MAX moves to C. MIN will then choose F, and MAX will take the leaf 2. Note MAX cannot hope for 8 (under D): MIN would never allow it.

Properties (b = branching factor, m = maximum depth):

Complete: yes, if the tree is finite

Optimal: yes, against an optimal opponent

Time: $O(b^m)$ Space: $O(bm)$ (depth-first)

Advantages: returns an optimal move assuming perfect opponent play; it is the simplest reasonable game-search algorithm. **Disadvantages:** it is infeasible for real games (chess: $b \approx 35$, $m \approx 100$, so 35^{100} nodes). In practice the depth is cut off and an evaluation function is applied at the cut-off states.

മലയാളത്തിൽ

Minimax എല്ലാ nodes-ഉം നോക്കുന്നതിനാൽ time $O(b^m)$ ആണ്, അതായത് exponential. ചെസ്സിൽ ഇത് ഒരിക്കലും തീരില്ല. അതുകൊണ്ട് കുറച്ച് depth വരെ മാത്രം നോക്കി, അവിടെ ഒരു evaluation function കൊണ്ട് value കണക്കാക്കുന്നു.

Exam tip: The unit defines "optimal" as "lowest path cost", which is the definition for ordinary search. For minimax, optimal means it gives the best achievable utility when the opponent also plays its best. Write that in the exam.

Alpha-Beta pruning

Definition: Alpha-beta pruning is minimax with cut-offs: it tracks α (best value MAX can guarantee) and β (best value MIN can guarantee) and prunes the remaining branches when $\alpha \geq \beta$, giving the same result as minimax.

cut-offs ഉള്ള minimax ആണ് alpha-beta pruning: α (MAX-ന് ഉറപ്പാക്കാവുന്ന മികച്ച value), β (MIN-ന് ഉറപ്പാക്കാവുന്ന മികച്ച value) എന്നിവ track ചെയ്ത് $\alpha \geq \beta$ ആകുമ്പോൾ ബാക്കി branches prune ചെയ്യുന്നു; ഫലം minimax-ന്റേതുതന്നെ.

Alpha-beta pruning is minimax with cut-offs. It returns exactly the same root value and move as minimax but skips branches that cannot affect the decision. It carries two bounds down the tree: α = best (highest) value MAX can guarantee so far, starting at $-\infty$; β = best (lowest) value MIN can guarantee so far, starting at $+\infty$.

Only MAX nodes update α : $\alpha = \max(\alpha, \text{child value})$; α never decreases

Only MIN nodes update β : $\beta = \min(\beta, \text{child value})$; β never increases

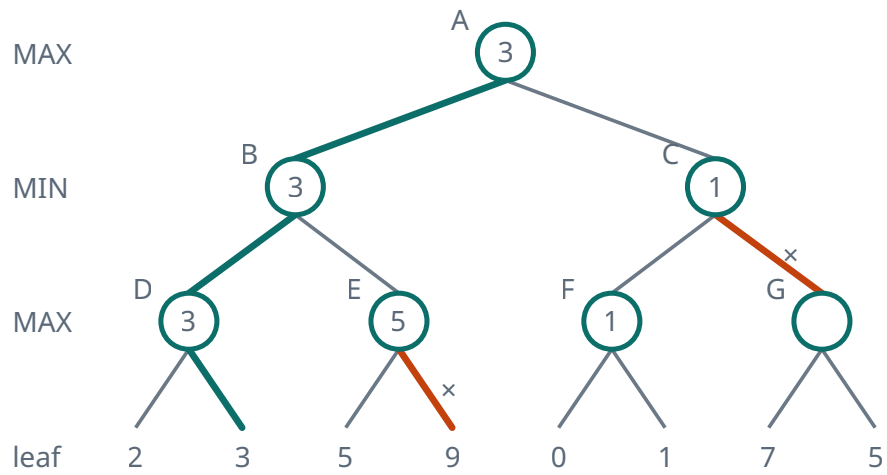
α and β are passed down (parent $\rightarrow$ child), never up

Node values (not α , β) are passed up when backtracking

Before visiting the next child: if $\alpha \geq \beta$, prune the remaining children

മലയാളത്തിൽ

α എന്നാൽ MAX-ന് ഇതിനകം ഉറപ്പായ ഏറ്റവും നല്ല value, β എന്നാൽ MIN-ന് ഉറപ്പായ ഏറ്റവും നല്ല (ചെറിയ) value. ഒരു node-ൽ $\alpha \geq \beta$ ആയാൽ, ആ വഴി ആരും തിരഞ്ഞെടുക്കില്ല എന്ന് ഉറപ്പ്, അതിനാൽ ബാക്കി branches നോക്കേണ്ടതില്ല. രണ്ട് കടകളിൽ ഒന്ന് നിങ്ങൾ (MAX) തിരഞ്ഞെടുക്കണം, പക്ഷേ ഓരോ കടയിലും കടക്കാരൻ (MIN) ഏറ്റവും മോശം സാധനമേ തരൂ എന്ന് കരുതുക. ആദ്യത്തെ കടയിൽ ഏറ്റവും മോശം സാധനത്തിനും 7 മാർക്ക് ഉറപ്പ് ($\alpha = 7$). രണ്ടാമത്തെ കടയിൽ ആദ്യം നോക്കിയ സാധനത്തിന് 4 മാർക്ക് മാത്രം; കടക്കാരൻ അതിലും മോശമേ തരൂ, അതിനാൽ ആ കട 4-ൽ കൂടുതൽ തരില്ല. ബാക്കി സാധനങ്ങൾ നോക്കാതെ ആ കട ഒഴിവാക്കാം: ഇതാണ് pruning.



Alpha-beta Example 1 from the unit. Pruned (x): leaf 9 under E and the whole subtree G. Root value 3 via $A \rightarrow B \rightarrow D \rightarrow 3$.

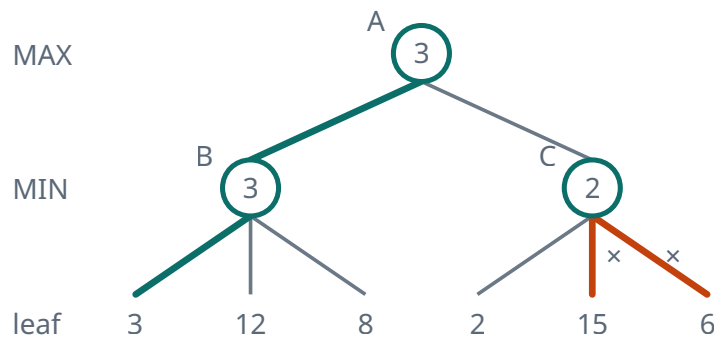
EXAMPLE 1 (FROM THE UNIT): FULL A-B TRACE

Leaves: D(2, 3), E(5, 9), F(0, 1), G(7, 5). A is MAX, B and C are MIN, D to G are MAX.

Step	Node	Action	α, β at node	$\alpha \geq \beta$?
1	A $\rightarrow$ B $\rightarrow$ D	pass $[-\infty, +\infty]$ down	$[-\infty, +\infty]$	–
2	D (MAX)	leaf 2: $\alpha = \max(-\infty, 2) = 2$	$[2, +\infty]$	no, continue
3	D (MAX)	leaf 3: $\alpha = \max(2, 3) = 3$; D = 3	$[3, +\infty]$	–
4	B (MIN)	D returns 3: $\beta = \min(+\infty, 3) = 3$	$[-\infty, 3]$	no, visit E
5	E (MAX)	gets $[-\infty, 3]$; leaf 5: $\alpha = 5$; E = 5	$[5, 3]$	yes: prune leaf 9
6	B (MIN)	E returns 5: $\beta = \min(3, 5) = 3$; B = 3	$[-\infty, 3]$	–
7	A (MAX)	B returns 3: $\alpha = \max(-\infty, 3) = 3$	$[3, +\infty]$	no, visit C
8	C $\rightarrow$ F	pass $[3, +\infty]$ down	$[3, +\infty]$	–
9	F (MAX)	leaf 0: $\alpha = \max(3, 0) = 3$; leaf 1: $\alpha = 3$; F = $\max(0, 1) = 1$	$[3, +\infty]$	no
10	C (MIN)	F returns 1: $\beta = \min(+\infty, 1) = 1$; C = 1	$[3, 1]$	yes: prune G (leaves 7, 5)
11	A (MAX)	C returns 1: $\alpha = \max(3, 1) = 3$; A = 3	$[3, +\infty]$	done

Root value 3 (same as plain minimax), 3 leaves pruned (9, 7, 5), optimal path A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ 3.

Why is G safe to skip? MAX already has 3 from B. At C, MIN can already force 1, which is below 3, so whatever G is worth, $C \leq 1$ and MAX will never choose C.



Alpha-beta Example 2 from the unit (B and C are MIN nodes with leaf children). Leaves 15 and 6 are pruned.

EXAMPLE 2 (FROM THE UNIT)

A (MAX) has two MIN children: B with leaves D = 3, E = 12, F = 8, and C with leaves G = 2, H = 15, I = 6. Children are visited left to right. Which nodes are pruned?

Step 1: B gets $[-\infty, +\infty]$. D = 3: $\beta = 3$. E = 12: $\beta = \min(3, 12) = 3$. F = 8: $\beta = 3$. B = 3. No pruning, since $\alpha = -\infty < 3$ each time.

Step 2: A: $\alpha = \max(-\infty, 3) = 3$, so A is $[3, +\infty]$.

Step 3: C gets $[3, +\infty]$. G = 2: $\beta = \min(+\infty, 2) = 2$. Now $\alpha = 3 \geq \beta = 2$, so prune H and I. C = 2.

Step 4: A: $\alpha = \max(3, 2) = 3$. Root value 3, pruned nodes H and I, optimal path A → B → D.

Exam tip: In Step 6 of Example 1 the unit writes "node values of F(=3) will be passed" and "at node B [$\alpha = 3, \beta = 1$]". The correct wording is F = 1, and the $\alpha = 3, \beta = 1$ values belong to node C. Also remember: a cut at a MAX node ($\alpha \geq \beta$) is called a **beta cut-off**, and a cut at a MIN node ($\beta \leq \alpha$) is called an **alpha cut-off**.

Move ordering in Alpha-Beta pruning

Definition: Move ordering is the order in which a node's children are examined in alpha-beta pruning; examining the best moves first maximises pruning and cuts time from $O(b^m)$ to $O(b^{m/2})$.

alpha-beta pruning-ൽ ഒരു node-ന്റെ children പരിശോധിക്കുന്ന ക്രമമാണ് move ordering; മികച്ച moves ആദ്യം പരിശോധിച്ചാൽ pruning പരമാവധിയാകും, time $O(b^m)$ -ൽ നിന്ന് $O(b^{m/2})$ ആയി കുറയും.

How much alpha-beta prunes depends on the **order** in which children are examined. Pruning never changes the final answer; good ordering only saves work.

Worst-case ordering: best moves examined last, nothing pruned; time $O(b^m)$, same as minimax

Best (ideal) ordering: best move examined first at every node; time $O(b^{m/2})$

So with ideal ordering alpha-beta can search about twice as deep as minimax in the same time

മലയാളത്തിൽ

നല്ല moves ആദ്യം നോക്കിയാൽ α , β വേഗം ശക്തമായ bounds ആകും, അതുകൊണ്ട് കൂടുതൽ pruning നടക്കും. മോശം moves ആദ്യം വന്നാൽ ഒന്നും prune ആകില്ല, സാധാരണ Minimax പോലെ തന്നെ. ഏറ്റവും നല്ല ക്രമത്തിൽ $O(b^{m/2})$, അതായത് അതേ സമയം കൊണ്ട് ഇരട്ടി depth വരെ നോക്കാം.

EXAMPLE

Apply alpha-beta to the minimax Example 1 tree (D(-1, 8), E(-3, -1), F(2, 1), G(-3, 4)). What is pruned?

B: D = 8 gives $\beta = 8$; E gets $[-\infty, 8]$: leaf -3 gives $\alpha = -3$, leaf -1 gives $\alpha = -1$, no cut; B = -1. A: $\alpha = -1$.

C gets $[-1, +\infty]$: F = 2 gives $\beta = 2$. G gets $[-1, 2]$: leaf -3 (α stays -1), leaf 4 ($\alpha = 4 \geq 2$, but there are no children left). C = 2, A = 2.

Nothing is pruned. The ordering is poor: under B the big value 8 came first, so β was weak. If E(-3, -1) had been examined before D, B's β would be -1 after E, and D would get $[-\infty, -1]$: its first leaf -1 makes $\alpha = -1 \geq \beta = -1$, so leaf 8 is pruned.

EXAMPLE

$b = 10$, $m = 6$. Compare the leaves examined.

Minimax: $10^6 = 1,000,000$. Alpha-beta, best ordering: $b^{m/2} + b^{m/2} - 1 = 10^3 + 10^3 - 1 = 1,999$ (Knuth and Moore's count), which is $O(b^{m/2})$, about 500 times fewer. Alpha-beta, worst ordering: 1,000,000.

Exam tip: The unit explains $O(b^{m/2})$ as "we search only the left sub-tree". The real reason: with perfect ordering, each MIN node needs only its first (best) child fully searched, so the effective branching factor drops from b to about $\sqrt{b}$, and $(\sqrt{b})^m = b^{m/2}$.

Quick recap

- Search = applying operators with a control strategy until a path from the initial state to a goal state is found.
- State space $S_s = [S, s_0, O, G]$; a solution is a path $s_0 \dots s_n \in G$; optimal = least path cost.
- A problem graph can be unfolded into a tree at the cost of duplicate nodes.
- Water jug: state (x, y) , start $(0, 0)$, goal $(2, n)$, rules R1 to R10; minimum 6 moves.
- 8-puzzle: move the blank UP/DOWN/LEFT/RIGHT, cost 1 per move.
- N-Queens: no two queens in the same row, column or diagonal; 4-Queens has 2 solutions $(2, 4, 1, 3)$ and $(3, 1, 4, 2)$; 8-Queens has 92 (12 unique).
- Same diagonal $\Leftrightarrow |j - i| = |k - i|$; backtracking prunes non-promising nodes.
- Missionaries and Cannibals: (M, C, B) , start $(3, 3, 0)$, goal $(0, 0, 1)$, 11 crossings.
- Tower of Hanoi: 3^n states, $2^n - 1$ minimum moves.
- Game elements: S_0 , PLAYER, ACTIONS, RESULT, TERMINAL-TEST, UTILITY $(+1, 0, -1)$.
- Minimax: MAX takes max, MIN takes min of children; DFS; complete, optimal; time $O(b^m)$, space $O(bm)$.
- Alpha-beta: α (MAX, starts $-\infty$, never decreases), β (MIN, starts $+\infty$, never increases); prune when $\alpha \geq \beta$.
- α, β travel down; node values travel up. The root value is always the same as minimax.
- Move ordering: worst $O(b^m)$, best $O(b^{m/2})$.

Check yourself

Q. 1. What factors must be considered when developing a state space representation?

(i) The goal to be achieved. (ii) The legal moves or actions. (iii) What knowledge must be represented in the state description. (iv) The type of problem: some need only a representation (crossword), some need a yes/no answer (is there a solution?), and some need a solution path (theorem proving, Tower of Hanoi). (v) Whether the best (optimal) solution is needed, as in the travelling salesman problem, or a good-enough solution will do.

Q. 2. You have a 5-litre and a 3-litre jug and a tap. Formulate the problem of getting exactly 4 litres in the 5-litre jug and give a shortest solution.

States (x, y) , $x \in \{0..5\}$, $y \in \{0..3\}$. Initial $(0, 0)$. Goal $(4, n)$. Operators: fill either jug, empty either jug, pour one into the other until the source is empty or the target is full. Cost 1 per move.

$(0,0) \rightarrow \text{fill } 5 \rightarrow (5,0) \rightarrow \text{pour } 5 \text{ into } 3 \rightarrow (2,3) \rightarrow \text{empty } 3 \rightarrow (2,0) \rightarrow \text{pour } 5 \text{ into } 3 \rightarrow (0,2) \rightarrow \text{fill } 5 \rightarrow (5,2) \rightarrow \text{pour } 5 \text{ into } 3 \text{ until full (1 litre fits)} \rightarrow (4,3)$. 6 moves, and breadth-first search shows none shorter exists.

Q. 3. Find the minimum cost path for the 8-puzzle from $\{(1,2,3), (_,4,6), (7,5,8)\}$ to $\{(1,2,3), (4,5,6), (7,8, _)\}$, each blank move costing 1.

Start	RIGHT	DOWN	RIGHT = Goal
1 2 3	1 2 3	1 2 3	1 2 3
_ 4 6	4 _ 6	4 5 6	4 5 6
7 5 8	7 5 8	7 _ 8	7 8 _

Path cost = 3. At the start the blank is on the left edge, so only UP, DOWN and RIGHT are legal. UP and DOWN move tiles 1 and 7, which are already in place, so RIGHT is the move to take.

Q. 4. Write the backtracking algorithm for N-Queens and draw the state space tree for 4 queens up to the first solution.

Use NQueen(k, n) and place(k, i) as given in the topic "State space tree and backtracking for N-Queens". The tree is the trace shown there: $(1,1)$ leads to dead ends at rows 3 and 4, so we backtrack to $(1,2)$, then $(2,4)$, $(3,1)$, $(4,3)$, giving the first solution $(2, 4, 1, 3)$. Mark each rejected node with $\times$ and write the reason (same column or same diagonal).

Q. 5. Is $(x_1, x_2, x_3, x_4) = (2, 4, 3, 1)$ a solution of 4-Queens? Justify using the diagonal test.

No. Rows 2 and 3: $|x_2 - x_3| = |4 - 3| = 1 = |2 - 3|$, so they share a diagonal. Checking all six row pairs, this is the only clash, but one clash is enough to reject it. The valid solutions are $(2, 4, 1, 3)$ and $(3, 1, 4, 2)$.

Q. 6. How many states does the Tower of Hanoi state space have for $n = 3$, and how many moves does the shortest solution need?

Each of the 3 disks can be on any of 3 pegs, and the order on a peg is forced (largest at the bottom), so there are $3^3 = 27$ states. Shortest solution: $2^3 - 1 = 7$ moves.

Q. 7. Compare Minimax and Alpha-Beta pruning with respect to time complexity.

Minimax examines every node: $O(b^m)$. Alpha-beta gives the same answer but prunes branches that cannot change the result. Its cost depends on move ordering: worst case $O(b^m)$ (no pruning), best case $O(b^{m/2})$, so it can search about twice as deep in the same time. Space for both is $O(bm)$ with depth-first search.

Q. 8. (Unit's CYP 2, Q4) Root A is MAX with MIN children B, C, D. B has MAX children E, F; C has G, H, I; D has J, K. Leaves left to right: E(4, 3), F(6, 2), G(2, 1), H(4, 5), I(3, 1), J(5, 4), K(7, 5). (a) Root value? (b) Nodes pruned by alpha-beta? (c) Optimal path?

Node	Trace	Value
E	$[-\infty, +\infty]$: 4 gives $\alpha = 4$; 3 gives no change	4
B	$\beta = 4$	–
F	$[-\infty, 4]$: 6 gives $\alpha = 6 \geq 4$, prune leaf 2	6
B	$\min(4, 6)$	4
A	$\alpha = 4$	–
G	$[4, +\infty]$: leaves 2, 1; α stays 4	2
C	$\beta = 2$; $\alpha = 4 \geq 2$, prune H and I	2
A	$\max(4, 2)$, $\alpha = 4$	–
J	$[4, +\infty]$: 5 gives $\alpha = 5$; 4 gives no change	5
D	$\beta = 5$	–
K	$[4, 5]$: 7 gives $\alpha = 7 \geq 5$, prune leaf 5	7
D	$\min(5, 7)$	5
A	$\max(4, 2, 5)$	5

(a) Root = 5 (plain minimax gives the same: B = 4, C = 2, D = 5). (b) Pruned: F's second leaf (2), nodes H and I with their leaves (4, 5, 3, 1), and K's second leaf (5). (c) A → D → J → 5.

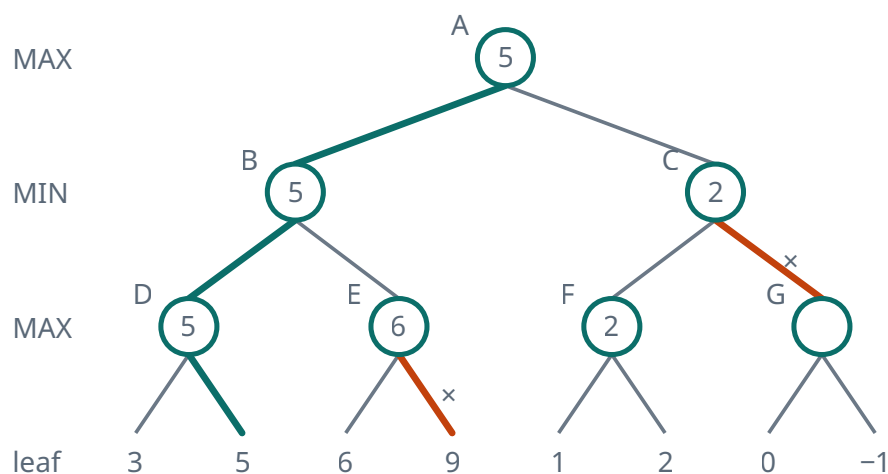
Note: the unit's answer figure shows 4 at the root. That is wrong; D backs up 5, and $\max(4, 2, 5) = 5$.

Q. 9. (Unit's CYP 2, Q3; tree reconstructed from the unit's worked answer) Apply alpha-beta to this tree: A (MAX); B, C (MIN); D, E, F, G (MAX); leaves D(3, 5), E(6, 9), F(1, 2), G(0, -1). Which nodes are pruned?

D: 3 then 5, D = 5. B: $\beta = 5$. E gets $[-\infty, 5]$: leaf 6 gives $\alpha = 6 \geq 5$, prune 9. B = 5. A: $\alpha = 5$.

C gets $[5, +\infty]$. F: leaves 1, 2, so F = 2. C: $\beta = 2$; $\alpha = 5 \geq 2$, prune G (leaves 0, -1). C = 2.

Root = $\max(5, 2) = 5$. Pruned terminals: 9, 0, -1. Optimal path A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ 5.



Final tree: pruned edges marked x.

Q. 10. Why are α and β passed only downwards, while node values are passed upwards?

α and β are bounds coming from choices already available to the ancestors ("MAX can already get α elsewhere", "MIN can already hold MAX to β "). A child needs them to decide whether its remaining branches can still matter. A child's own result is an actual backed-up value, which the parent combines with max or min. Sending α or β upward instead would mix a bound from another branch into this node's value.

UNIT 3

Uninformed and Informed Search

ഒരു problem-നെ state space ആയി എഴുതി, start state-ൽ നിന്ന് goal state-ലേക്ക് വഴി കണ്ടുപിടിക്കുന്ന search algorithms ആണ് ഈ unit. BFS, DFS, IDDFS പോലുള്ള blind (uninformed) search-ഉം, heuristic ഉപയോഗിക്കുന്ന Greedy best-first, A*, AO*, IDA*, RBFS എന്നിവയും ഓരോന്നായി OPEN/CLOSED list trace സഹിതം പഠിക്കാം.

Search: uninformed vs informed

Formulating search in a state space (OPEN and CLOSED)

Evaluating a search algorithm

Uninformed search and the running example

Breadth-first search (BFS)

Depth-first search (DFS)

Comparison of BFS and DFS

Depth-limited search (DLS)

Iterative deepening depth-first search (IDDFS)

Uniform cost search (UCS)

Bidirectional search

Comparison of uninformed search strategies

Informed (heuristic) search and the heuristic function

Best-first search and greedy best-first search

A* algorithm

Admissibility and properties of A*

Problem reduction and AND-OR graphs

AO* algorithm

Memory-bounded search: Iterative Deepening A* (IDA*)

Recursive best-first search (RBFS)

All search algorithms compared

Search: uninformed vs informed

Search: finding a sequence of actions that takes the agent from the initial state to a goal state in the state space.

Uninformed (blind) search: search that uses no information beyond the problem definition (initial state, operators, goal test).

Informed (heuristic) search: search that uses problem-specific knowledge, usually a heuristic function, to guide the search towards the goal.

Search: state space-ൽ agent-നെ initial state-ൽ നിന്ന് goal state-ലേക്ക് എത്തിക്കുന്ന actions-ന്റെ sequence കണ്ടെത്തൽ.

Uninformed (blind) search: problem definition (initial state, operators, goal test)

അല്ലാതെ മറ്റൊരു information-ഉം ഉപയോഗിക്കാത്ത search.

Informed (heuristic) search: search-നെ goal-ലേക്ക് നയിക്കാൻ problem-specific knowledge, സാധാരണ ഒരു heuristic function, ഉപയോഗിക്കുന്ന search.

Before an AI problem can be solved it is represented as a **state space**, and solving it means **searching** for a sequence of actions that takes the agent from the initial state to a goal state. Every search repeats the same loop: check whether the current state is the goal, expand it to generate new states, choose one of the new states according to the search strategy, and repeat until the goal is reached or nothing is left to expand. Search methods fall into two families: **uninformed (blind, brute-force, exhaustive)** search, which uses no knowledge beyond the problem definition, and **informed (heuristic, guided)** search, which uses domain knowledge to steer towards the goal.

മലയാളത്തിൽ

Search എന്നാൽ start state-ൽ നിന്ന് goal-ലേക്കുള്ള ഒരു വഴി (sequence of actions) കണ്ടുപിടിക്കൽ. കൊച്ചിയിൽ നിന്ന് കോഴിക്കോട്ടേക്ക് പോകാൻ map ഇല്ലാതെ ഓരോ junction-ഉം പരീക്ഷിച്ച് നോക്കുന്നതാണ് uninformed search.

"കോഴിക്കോട് ഇങ്ങോട്ട് ഏകദേശം 100 km" എന്ന ബോർഡ് നോക്കി ദിശ തീരുമാനിക്കുന്നതാണ് informed (heuristic) search. രണ്ടിലും ചെയ്യുന്നത് ഒന്നുതന്നെ: goal ആണോ എന്ന് നോക്കുക, അല്ലെങ്കിൽ expand ചെയ്യുക, അടുത്ത node strategy അനുസരിച്ച് തിരഞ്ഞെടുക്കുക.

Uninformed (blind) search	Informed (heuristic) search
No information about the path cost from the current state to the goal; no domain-specific knowledge.	Estimates the cost from the current state to the goal (heuristic $h(n)$); uses domain-specific knowledge.
Finds solutions more slowly; less efficient; higher cost.	Finds solutions more quickly; more efficient; lower cost.
No guidance towards the solution.	Guides the search towards the goal.
BFS, DFS, depth-limited search, uniform cost search, IDDFS, bidirectional search.	Best-first search, greedy search, A^* , AO^* , IDA^* , RBFS.

Exam tip: Uniform cost search uses the path cost $g(n)$ but no estimate of the remaining distance, so it is uninformed. The unit's summary lists it under informed search; the unit's own classification chart (and every textbook) puts it under uninformed search.

Formulating search in a state space (OPEN and CLOSED)

OPEN list: the nodes that have been generated but not yet expanded.

CLOSED list: the nodes that have already been expanded, kept so that no state is revisited.

OPEN list: generate ചെയ്തെങ്കിലും ഇതുവരെ expand ചെയ്യാത്ത nodes.

CLOSED list: ഇതിനകം expand ചെയ്ത nodes; ഒരു state-ഉം വീണ്ടും സന്ദർശിക്കാതിരിക്കാൻ സൂക്ഷിക്കുന്നു.

A **state space** is a graph (V, E) . Each node in V holds a state description (plus bookkeeping such as its parent); each arc in E is an applicable action with a fixed non-negative cost. **Expanding** a node means generating its successors. A **goal test** decides whether a node is a goal. A **solution** is a path from a start node to a goal node, and its cost is the sum of the arc costs on it. Searching makes an implicit graph explicit, starting with $V = \{s\}$, until a goal node is generated and identified. Two lists do the bookkeeping: **OPEN** holds nodes generated but not yet expanded, and **CLOSED** holds nodes already expanded, so that no state is revisited. Search strategies differ only in which node they select from OPEN and where they insert new nodes.

മലയാളത്തിൽ

State space ഒരു graph ആണ്: towns ആണ് nodes, റോഡുകൾ ആണ് arcs, റോഡിന്റെ നീളം ആണ് arc cost. OPEN list എന്നാൽ "ഇനി പോയി നോക്കാൻ ഉള്ള" towns-ന്റെ list; CLOSED list എന്നാൽ "നോക്കിക്കഴിഞ്ഞ" towns. ഒരു town-ന്റെ അയൽക്കാരെ OPEN-ൽ ചേർക്കുമ്പോൾ, അത് ഇതിനകം OPEN-ലോ CLOSED-ലോ ഉണ്ടെങ്കിൽ വീണ്ടും ചേർക്കില്ല; അതുകൊണ്ട് ഒരേ സ്ഥലത്ത് വട്ടം കറങ്ങില്ല. BFS ആണോ DFS ആണോ എന്നത് OPEN-ൽ നിന്ന് ഏത് node എടുക്കുന്നു എന്നതിനെ മാത്രം ആശ്രയിച്ചിരിക്കുന്നു.

General search (with CLOSED):

1. Initialize: $OPEN = \{s\}$, $CLOSED = \{\}$
2. Fail: if $OPEN = \{\}$, terminate with failure
3. Select: select a state n from OPEN and save n in CLOSED
4. Terminate: if $n \in \text{Goal}$, terminate with success
5. Expand: generate the successors of n ; insert each successor m in OPEN only if $m \notin (OPEN \cup CLOSED)$
6. Loop: go to step 2

EXAMPLE

Write the route-finding problem "Kochi to Kozhikode" as a state-space search problem.

States V = the towns {Kochi, Kodungallur, Thrissur, Guruvayur, Palakkad, Malappuram, Kozhikode}. Arcs E = the roads, each with a cost in km (figure in the next-but-one topic). Start s = Kochi. Goal test: "is the current town Kozhikode?". A solution is a road sequence such as Kochi $\rightarrow$ Kodungallur $\rightarrow$ Guruvayur $\rightarrow$ Kozhikode, with cost $35 + 45 + 100 = 180$ km.

Exam tip: Without the CLOSED list (the first "general search algorithm" in the unit) a search on a graph with cycles can revisit states forever. Always state the rule "insert m only if $m \notin OPEN \cup CLOSED$ ".

Evaluating a search algorithm

Completeness: the algorithm always finds a solution if one exists.

Optimality (admissibility): the solution found is guaranteed to have the lowest path cost.

Time / space complexity: the number of nodes generated, and the maximum number of nodes held in memory, expressed in terms of b , d and m .

Completeness: ഒരു solution ഉണ്ടെങ്കിൽ algorithm അത് എപ്പോഴും കണ്ടെത്തും.

Optimality (admissibility): കണ്ടെത്തുന്ന solution-ന് ഏറ്റവും കുറഞ്ഞ path cost ആണെന്ന് ഉറപ്പ്.

Time / space complexity: generate ചെയ്യുന്ന nodes-ന്റെ എണ്ണവും memory-യിൽ ഒരേസമയം സൂക്ഷിക്കുന്ന പരമാവധി nodes-ന്റെ എണ്ണവും; b , d , m ഉപയോഗിച്ച് എഴുതുന്നു.

Every search algorithm is judged on four criteria. **Completeness:** does it always find a solution if one exists? **Optimality (admissibility):** is the solution found guaranteed to have the lowest path cost? **Time complexity:** how many nodes are generated or expanded? **Space complexity:** the maximum number of nodes held in memory (roughly the largest size of OPEN). Complexities are written in terms of b , d and m .

മലയാളത്തിൽ

ഒരു algorithm നല്ലതാണോ എന്ന് അളക്കാൻ നാല് ചോദ്യങ്ങൾ: വഴി ഉണ്ടെങ്കിൽ ഉറപ്പായും കണ്ടുപിടിക്കുമോ (complete)? കണ്ടുപിടിക്കുന്നത് ഏറ്റവും ചെലവ് കുറഞ്ഞ വഴി ആണോ (optimal)? എത്ര സമയം (time)? എത്ര memory (space)? b എന്നത് ഒരു node-ന് പരമാവധി എത്ര children എന്നതും, d എന്നത് ഏറ്റവും അടുത്ത goal എത്ര ആഴത്തിൽ എന്നതും, m എന്നത് tree-യുടെ പരമാവധി ആഴവും ആണ്.

b = branching factor (maximum number of successors of any node)

d = depth of the shallowest goal node (least-cost solution)

m = maximum depth of any path in the state space (may be ∞)

Nodes in a complete tree of depth d : $1 + b + b^2 + \dots + b^d = (b^{d+1} - 1)/(b - 1) = O(b^d)$

EXAMPLE

A complete search tree has branching factor $b = 10$ and depth $d = 12$. How many nodes does it have?

$1 + 10 + 10^2 + \dots + 10^{12} = (10^{13} - 1)/(10 - 1) = 9\,999\,999\,999\,999/9 = 1\,111\,111\,111\,111 \approx 1.1 \times 10^{12} = O(10^{12})$ nodes. At a million nodes per second that is almost 13 days of work, which is why exponential complexity matters.

Uninformed search and the running example

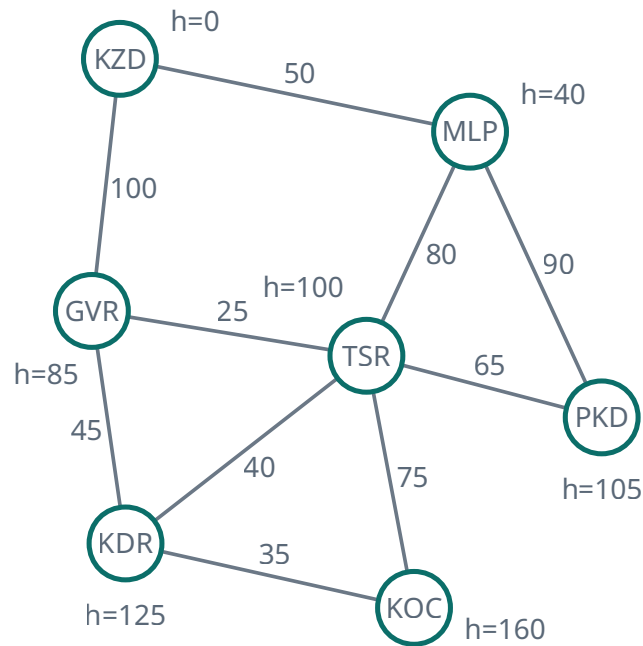
Definition: Uninformed search strategies (BFS, DFS, DLS, UCS, IDDFS, bidirectional) explore the state space using only the initial state, operators and goal test, with no knowledge of how close a node is to the goal.

ഒരു node goal-നോട് എത്ര അടുത്താണ് എന്ന അറിവില്ലാതെ, initial state, operators, goal test എന്നിവ മാത്രം ഉപയോഗിച്ച് state space explore ചെയ്യുന്നവയാണ് uninformed search strategies (BFS, DFS, DLS, UCS, IDDFS, bidirectional).

Uninformed search has no domain knowledge such as closeness or the location of the goal. It knows only the initial state, the operators and the goal test, and it examines nodes until it reaches the goal. The unit's analogy: you have lost your car key and remember nothing, so you search the table drawer, then the wardrobe, then the whole house. The uninformed methods are breadth-first, depth-first, depth-limited, uniform cost, iterative deepening and bidirectional search. To compare them fairly, this chapter runs every algorithm on the same small Kerala road map below.

മലയാളത്തിൽ

Uninformed search-ന് goal എവിടെയാണ് എന്ന് ഒരു സൂചനയും ഇല്ല. താക്കോൽ എവിടെ വെച്ചെന്ന് ഓർമ്മയില്ലാതെ വീട് മുഴുവൻ തപ്പുന്നതുപോലെ. താഴെയുള്ള Kerala map ആണ് ഈ chapter-ലെ എല്ലാ algorithm-ഉം പരീക്ഷിക്കുന്ന ഒരേ example: കൊച്ചിയിൽ നിന്ന് കോഴിക്കോട്ടേക്ക്. റോഡിലെ സംഖ്യ road distance (km), ഓരോ town-ന്റെ അടുത്തുള്ള h എന്നത് കോഴിക്കോട്ടേക്കുള്ള നേർരേഖാ ദൂരം (straight-line distance). Uninformed algorithms h നോക്കില്ല; informed algorithms-ൽ മാത്രമേ h ഉപയോഗിക്കൂ.



Running example. Road distances (km) on the roads; h = straight-line distance to Kozhikode (KZD).
Start KOC, goal KZD.

Code	Town	Neighbours (road km)	h = straight-line km to Kozhikode
KOC	Kochi (start)	KDR 35, TSR 75	160
KDR	Kodungallur	GVR 45, KOC 35, TSR 40	125
TSR	Thrissur	GVR 25, KDR 40, KOC 75, MLP 80, PKD 65	100
GVR	Guruvayur	KDR 45, KZD 100, TSR 25	85
PKD	Palakkad	MLP 90, TSR 65	105
MLP	Malappuram	KZD 50, PKD 90, TSR 80	40
KZD	Kozhikode (goal)	GVR 100, MLP 50	0

Conventions used in every trace below:

- Roads are two-way. Successors are generated in alphabetical order of the town's name (Guruvayur, Kochi, Kodungallur, Kozhikode, Malappuram, Palakkad, Thrissur), as in the neighbour lists above.
- The goal test is done when a node is removed from OPEN (as in the unit's algorithms).
- Ties in f or g are broken in favour of the node that entered OPEN first.
- The best route is KOC → KDR → GVR → KZD = 35 + 45 + 100 = 180 km.

Exam tip: Distances are rounded, realistic figures chosen so that the numbers are easy to add. In the exam, always write the successor order you are using; a different order gives a different (but still correct) trace.

Breadth-first search (BFS)

Definition: Breadth-first search (BFS) expands every node at depth k before any node at depth $k + 1$, using OPEN as a FIFO queue.

depth $k + 1$ -ലെ ഏതെങ്കിലും node-ന് മുമ്പ് depth k -ലെ എല്ലാ nodes-ഉം expand ചെയ്യുന്ന search ആണ് Breadth-first search (BFS); OPEN ഒരു FIFO queue ആയി ഉപയോഗിക്കുന്നു.

BFS expands the root first, then all its successors, then their successors, and so on: every node at depth k is expanded before any node at depth $k + 1$. OPEN is used as a **queue (FIFO)**: new successors are inserted at the right end and nodes are removed from the left end. BFS is best when the solution is shallow and the branching factor is small.

മലയാളത്തിൽ

BFS ഒരു കല്ല് കളത്തിൽ ഇട്ടാൽ ഉണ്ടാകുന്ന വട്ടങ്ങൾ പോലെ layer-by-layer ആണ് പോകുന്നത്. ആദ്യം കൊച്ചിയോട് നേരിട്ട് ചേർന്ന towns, പിന്നെ അവയോട് ചേർന്ന towns. OPEN ഒരു ബസ്-ക്യൂ പോലെ: ആദ്യം വന്നയാൾ ആദ്യം കയറും (FIFO), പുതിയവർ പിന്നിൽ നിൽക്കും. ഏറ്റവും കുറവ് റോഡുകൾ (edges) ഉള്ള വഴി BFS കണ്ടുപിടിക്കും, പക്ഷേ എല്ലാ layer-ഉം memory-യിൽ സൂക്ഷിക്കണം എന്നതാണ് പ്രശ്നം.

BFS algorithm:

1. Initialize: OPEN = {s}, CLOSED = { }
2. Fail: if OPEN = { }, terminate with failure
3. Select: remove the leftmost state a from OPEN; put a in CLOSED
4. Terminate: if a ∈ Goal, terminate with success
5. Expand: generate the successors of a; discard those already in OPEN ∪ CLOSED; insert the rest at the **right end** of OPEN (queue)
6. Loop: go to step 2

EXAMPLE (TRACE)

Apply BFS to the Kerala map from KOC to KZD, showing OPEN and CLOSED.

Step	Removed from OPEN	OPEN (after step)	CLOSED	Remark
1	–	KOC	{ }	start
2	KOC	KDR, TSR	KOC	not goal; expand KOC
3	KDR	TSR, GVR	KOC, KDR	not goal; expand KDR
4	TSR	GVR, MLP, PKD	KOC, KDR, TSR	not goal; expand TSR
5	GVR	MLP, PKD, KZD	KOC, KDR, TSR, GVR	not goal; expand GVR
6	MLP	PKD, KZD	KOC, KDR, TSR, GVR, MLP	not goal; expand MLP
7	PKD	KZD	KOC, KDR, TSR, GVR, MLP, PKD	not goal; expand PKD
8	KZD	{ }	KOC, KDR, TSR, GVR, MLP, PKD, KZD	goal test succeeds: stop

Expansion order: KOC → KDR → TSR → GVR → MLP → PKD → KZD.

Following parent pointers from KZD (reached from GVR, which was reached from KDR): path KOC → KDR → GVR → KZD, 3 roads, cost 180 km. Note step 3: TSR was already in OPEN, so KDR did not add it again; step 4: MLP and PKD joined the back of the queue.

Time = $1 + b + b^2 + \dots + b^d = O(b^d)$

Space = $O(b^d)$ (the whole last level sits in OPEN)

Complete: yes (if b is finite)

Optimal: yes, if every step costs the same (it finds the path with the fewest edges)

Advantages: never trapped down a blind alley; guaranteed to find a solution if one exists; finds the shallowest solution. **Disadvantages:** time and space are both exponential, $O(b^d)$; every generated node must be stored, even useless ones.

Exam tip: BFS here returned the cheapest route only by luck. BFS minimises the number of roads, not kilometres. If the question has unequal edge costs and asks for the cheapest path, BFS is not optimal; use uniform cost search or A^* .

Depth-first search (DFS)

Definition: Depth-first search (DFS) always expands the deepest (most recently generated) node, going down one path as far as possible before backtracking, using OPEN as a LIFO stack.

backtrack ചെയ്യുന്നതിന് മുമ്പ് ഒരു path-യുടെ കഴിയുന്നത്ര ആഴത്തിൽ പോയി, എപ്പോഴും ഏറ്റവും ആഴത്തിലുള്ള (ഏറ്റവും ഒടുവിൽ generate ചെയ്ത) node expand ചെയ്യുന്ന search ആണ് Depth-first search (DFS); OPEN ഒരു LIFO stack ആയി ഉപയോഗിക്കുന്നു.

DFS always expands the deepest (most recently generated) node, going as far down one path as possible before backtracking to try alternatives. OPEN is used as a **stack (LIFO)**: new successors are inserted at the left end. The only difference from BFS is step 5, the end of OPEN where successors are inserted. DFS needs to store only the current path and the unexplored siblings along it, so its memory is linear.

മലയാളത്തിൽ

DFS ഒരു വഴി കിട്ടിയാൽ അതിലൂടെ അവസാനം വരെ പോകും; dead end ആയാൽ മാത്രം തിരിച്ചു വന്ന് അടുത്ത വഴി നോക്കും (backtracking). പ്ലേറ്റുകൾ അടുക്കിവെച്ചതുപോലെ OPEN ഒരു stack ആണ്; അവസാനം വെച്ചത് ആദ്യം എടുക്കും. Memory കുറവ് മതി എന്നതാണ് ഗുണം; പക്ഷേ ആദ്യം പോയ വഴി നീളമുള്ളതാണെങ്കിലും അതുതന്നെ answer ആയി തരും, അതുകൊണ്ട് optimal അല്ല.

DFS algorithm: same as BFS except

5. Expand: generate the successors of a ; discard those already in $OPEN \cup CLOSED$; insert the rest at the **left end** of OPEN (stack)

EXAMPLE (TRACE)

Apply DFS to the Kerala map from KOC to KZD.

Step	Removed from OPEN	OPEN (after step)	CLOSED	Remark
1	–	KOC	{ }	start
2	KOC	KDR, TSR	KOC	not goal; expand KOC
3	KDR	GVR, TSR	KOC, KDR	not goal; expand KDR
4	GVR	KZD, TSR	KOC, KDR, GVR	not goal; expand GVR
5	KZD	TSR	KOC, KDR, GVR, KZD	goal test succeeds: stop

Expansion order: KOC → KDR → GVR → KZD. Path KOC → KDR → GVR → KZD, cost 180 km, found after only 4 removals (BFS needed 7).

EXAMPLE

Repeat DFS but generate successors in reverse alphabetical order (Thrissur first). What happens?

Expansion order: KOC → TSR → PKD → MLP → KZD. DFS dives KOC → TSR → PKD, backtracks, and returns KOC → TSR → MLP → KZD = 75 + 80 + 50 = 205 km. That is 25 km worse than the best route, which shows DFS is **not optimal**: its answer depends on the order in which children happen to be generated.

Nodes generated (worst case): $1 + b + b^2 + \dots + b^m = O(b^m)$

Time = $O(b^m)$, Space = $O(bm)$ (linear)

Complete: no (it can run down an infinite or looping path); yes on a finite graph when repeated states are checked

Optimal: no

Advantages: very little memory (only the current path plus siblings); can find a solution quickly if it happens to lie on an early path. **Disadvantages:** may go down an infinite path and never return; no guarantee of the shortest or cheapest solution.

Exam tip: Section 3.3.5 of the unit says "BFS algorithm is Complete ... The BFS algorithm is not optimal" under DFS; both lines mean DFS. Also, its complexities use m, the maximum depth, not d: time $O(b^m)$, space $O(bm)$.

Comparison of BFS and DFS

BFS goes level by level and needs space $O(b^d)$; DFS goes deep and needs only $O(bm)$. The weakness of DFS is that a node close to the root, but not in the first subtrees explored, is reached very late; for example, if node 2 is a child of the root 0, DFS may wander deep under node 1 (possibly forever in an infinite tree) before it comes back to node 2. Iterative deepening (topic after next) fixes this.

മലയാളത്തിൽ

BFS-ന് സമയം പോലെ തന്നെ memory-യും exponential; DFS-ന് memory വളരെ കുറവ്, പക്ഷേ വഴി തെറ്റി ആഴത്തിലേക്ക് പോയാൽ തൊട്ടടുത്തുള്ള goal പോലും വളരെ വൈകിയേ കാണൂ. അതുകൊണ്ട് ഏത് ഉപയോഗിക്കണം എന്നത് problem-ന്റെ സ്വഭാവം നോക്കി തീരുമാനിക്കണം.

Feature	BFS	DFS
Data structure for OPEN	Queue (FIFO), insert at right	Stack (LIFO), insert at left
Order	Level by level	Deepest node first, then backtrack
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$	$O(bm)$
Complete	Yes (finite b)	No (infinite paths)
Optimal	Yes for equal step costs	No
Kerala trace	7 removals, 180 km	4 removals, 180 km (205 km with the other order)

Use DFS when	Use BFS when	Use IDDFS when
Many solutions exist, and you know (or can estimate) the depth of a solution.	Some solutions are known to be shallow.	Space is limited and the shortest solution path is required.

Depth-limited search (DLS)

Definition: Depth-limited search (DLS) is DFS with a predefined depth limit l , where nodes at depth l are treated as having no successors.

depth l -ലെ nodes-ന് successors ഇല്ലാത്തതായി കണക്കാക്കുന്നു, മുൻകൂട്ടി നിശ്ചയിച്ച depth limit l ഉള്ള DFS ആണ് Depth-limited search (DLS).

Depth-limited search is DFS with a predefined depth limit l : nodes at depth l are treated as if they had no successors. This stops DFS from running down an infinite path, but if the shallowest goal lies deeper than l ($d > l$) the search fails. The unit lists DLS among the uninformed methods and uses it inside IDDFS. In the traces below, a node's subscript is its depth, and a successor is skipped only if it is already on the current path (tree search), which is how DLS is used inside IDDFS.

മലയാളത്തിൽ

DLS എന്നാൽ "ഇത്ര ആഴം വരെ മാത്രം" എന്ന് നിയന്ത്രണം വെച്ച DFS. "രണ്ട് junction-ൽ കൂടുതൽ ദൂരം പോകില്ല" എന്ന് ഉറപ്പിച്ച് ഓട്ടോക്കാരൻ വഴി തിരയുന്നതുപോലെ. അനന്തമായ വഴിയിൽ കുടുങ്ങില്ല; പക്ഷേ goal limit-നേക്കാൾ ആഴത്തിലാണെങ്കിൽ കിട്ടില്ല.

DLS(l): run DFS, but do not expand a node whose depth = l

Time = $O(b^l)$, Space = $O(bl)$

Complete: only if $l \geq d$; Optimal: no

EXAMPLE (TRACE)

Run DLS with limit $l = 2$ on the Kerala map from KOC.

Step	Removed	OPEN (stack, after step)	Visited so far	Remark
1	–	KOC_0	$\{\}$	start
2	KOC_0	KDR_1, TSR_1	KOC_0	expand (depth 0)
3	KDR_1	GVR_2, TSR_2, TSR_1	KOC_0, KDR_1	expand (depth 1)
4	GVR_2	TSR_2, TSR_1	KOC_0, KDR_1, GVR_2	depth 2 = limit: not expanded
5	TSR_2	TSR_1	$KOC_0, KDR_1, GVR_2, TSR_2$	depth 2 = limit: not expanded
6	TSR_1	$GVR_2, KDR_2, MLP_2, PKD_2$	$KOC_0, KDR_1, GVR_2, TSR_2, TSR_1$	expand (depth 1)
7	GVR_2	KDR_2, MLP_2, PKD_2	$KOC_0, KDR_1, GVR_2, TSR_2, TSR_1, GVR_2$	depth 2 = limit: not expanded
8	KDR_2	MLP_2, PKD_2	$KOC_0, KDR_1, GVR_2, TSR_2, TSR_1, GVR_2, KDR_2$	depth 2 = limit: not expanded
9	MLP_2	PKD_2	$KOC_0, KDR_1, GVR_2, TSR_2, TSR_1, GVR_2, KDR_2, MLP_2$	depth 2 = limit: not expanded
10	PKD_2	$\{\}$	$KOC_0, KDR_1, GVR_2, TSR_2, TSR_1, GVR_2, KDR_2, MLP_2, PKD_2$	depth 2 = limit: not expanded

Order: $KOC_0 \rightarrow KDR_1 \rightarrow GVR_2 \rightarrow TSR_2 \rightarrow TSR_1 \rightarrow GVR_2 \rightarrow KDR_2 \rightarrow MLP_2 \rightarrow PKD_2$. OPEN becomes empty without reaching KZD, because every route to Kozhikode needs 3 roads ($d = 3 > l = 2$). DLS(2) reports **failure (cut-off)** even though a solution exists.

Exam tip: Distinguish the two kinds of failure: "no solution" (OPEN empties and no node was cut off) and "cut-off" (nodes existed below the limit). Only the second tells IDDFS to try a larger limit.

Iterative deepening depth-first search (IDDFS)

Definition: Iterative deepening DFS (IDDFS) runs depth-limited search with limits 0, 1, 2, ... until a goal is found, combining the completeness and optimality of BFS with the low memory of DFS.

goal കിട്ടുന്നതുവരെ limit 0, 1, 2, ... എന്നിങ്ങനെ depth-limited search ആവർത്തിച്ച് run ചെയ്യുന്ന search ആണ് iterative deepening DFS (IDDFS); BFS-ന്റെ completeness-ഉം optimality-യും DFS-ന്റെ കുറഞ്ഞ memory-യും ഇതിൽ ഒന്നിക്കുന്നു.

IDDFS runs depth-limited DFS with limit 0, then 1, then 2, and so on, until a goal is found. Each iteration is a DFS (so it uses little memory), but because a whole layer is completed before the limit grows, it finds the shallowest goal like BFS. The price is repeated work: the top levels are regenerated in every iteration. It is the preferred uninformed method when the search space is large and the depth of the solution is unknown.

മലയാളത്തിൽ

IDDFS ആദ്യം 0 ആഴം വരെ, പിന്നെ 1, പിന്നെ 2... എന്നിങ്ങനെ ഓരോ തവണയും limit ഒന്ന് കൂട്ടി DFS ആവർത്തിക്കുന്നു. വീട്ടിൽ താക്കോൽ തിരയുമ്പോൾ ആദ്യം ഹാളിൽ മാത്രം, കിട്ടിയില്ലെങ്കിൽ ഹാളും മുറികളും, പിന്നെ ടെറസ് വരെ എന്ന് പരിധി കൂട്ടുന്നതുപോലെ. DFS-ന്റെ കുറഞ്ഞ memory-യും BFS-ന്റെ shortest path ഉറപ്പും ഒരുമിച്ച് കിട്ടും; മുകളിലെ levels വീണ്ടും വീണ്ടും നോക്കേണ്ടിവരുന്നതാണ് നഷ്ടം.

IDDFS algorithm:

L = 0; found = false

while found = false:

 perform DFS from the start to depth L (DLS(L))

 if a goal state is obtained then found = true

 else discard the nodes generated and set L = L + 1

report the solution, stop

EXAMPLE (TRACE)

Apply IDDFS to the Kerala map from KOC to KZD.

Limit L	Nodes visited in this iteration (subscript = depth)	Count	Result
0	KOC ₀	1	cut off, not found
1	KOC ₀ → KDR ₁ → TSR ₁	3	cut off, not found
2	KOC ₀ → KDR ₁ → GVR ₂ → TSR ₂ → TSR ₁ → GVR ₂ → KDR ₂ → MLP ₂ → PKD ₂	9	cut off, not found
3	KOC ₀ → KDR ₁ → GVR ₂ → KZD ₃	4	found: KOC → KDR → GVR → KZD

Detailed OPEN list for the last iteration (L = 3):

Step	Removed	OPEN (stack, after step)	Visited so far	Remark
1	–	KOC ₀	{}	start
2	KOC ₀	KDR ₁ , TSR ₁	KOC ₀	expand (depth 0)
3	KDR ₁	GVR ₂ , TSR ₂ , TSR ₁	KOC ₀ , KDR ₁	expand (depth 1)
4	GVR ₂	KZD ₃ , TSR ₃ , TSR ₂ , TSR ₁	KOC ₀ , KDR ₁ , GVR ₂	expand (depth 2)
5	KZD ₃	TSR ₃ , TSR ₂ , TSR ₁	KOC ₀ , KDR ₁ , GVR ₂ , KZD ₃	goal: stop

IDDFS stops at L = 3, the depth of the shallowest goal, and returns KOC → KDR → GVR → KZD (3 roads, 180 km). In total it made 1 + 3 + 9 + 4 = 17 node visits, many of them repeats of KOC and KDR.

EXAMPLE

How much extra work does IDDFS do compared with BFS for $b = 10$, $d = 5$?

In IDDFS, depth-1 nodes are generated d times, depth-2 nodes $d - 1$ times, ..., depth- d nodes once:

$$N(\text{IDDFS}) = 5(10) + 4(100) + 3(1000) + 2(10\,000) + 1(100\,000) = 50 + 400 + 3000 + 20\,000 + 100\,000 = 123\,450.$$

$$N(\text{BFS}) = 10 + 100 + 1000 + 10\,000 + 100\,000 = 111\,110.$$

The repetition costs only about 11% extra, because most nodes are on the bottom level.

Time = $O(b^d)$, Space = $O(bd)$

Complete: yes (finite b); Optimal: yes for equal step costs

Advantages: combines the speed and shortest-path guarantee of BFS with the memory efficiency of DFS; preferred when the space is large and the solution depth is unknown. **Disadvantages:** repeats all the work of the previous iteration; time is still exponential.

Exam tip: The unit's IDDFS pseudocode initialises $d = 1$, but its worked example (A to M) begins with limit 0. Start from $L = 0$ in your answers; that is also how the unit's figures proceed ($L = 0, 1, 2, 3$, goal M found at $L = 3$).

Uniform cost search (UCS)

Definition: Uniform cost search (UCS) expands the node in OPEN with the lowest path cost $g(n)$ from the start, using a priority queue ordered by g .

start-ൽ നിന്നുള്ള path cost $g(n)$ ഏറ്റവും കുറഞ്ഞ OPEN-ലെ node expand ചെയ്യുന്ന search ആണ് Uniform cost search (UCS); g അനുസരിച്ച് ക്രമീകരിച്ച priority queue ഉപയോഗിക്കുന്നു.

Uniform cost search expands the node with the **lowest path cost $g(n)$** from the start, using OPEN as a priority queue ordered by g . If a cheaper path to a node already in OPEN is found, its g is lowered. Because the goal test is done when a node is selected (not when it is generated), the first goal selected has the least cost. With all step costs equal, UCS behaves like BFS. The unit mentions UCS in its list of uninformed methods and as "A*" with $h = 0$.

മലയാളത്തിൽ

UCS ഓരോ തവണയും ഇതുവരെ ഏറ്റവും കുറച്ച് km ഓടിയ town ആണ് expand ചെയ്യുന്നത്. "ഇനി എത്ര ദൂരം" എന്ന് അറിയില്ല, "ഇതുവരെ എത്ര ഓടി" മാത്രം അറിയാം. അതുകൊണ്ട് എല്ലാ ദിശയിലേക്കും ഒരുപോലെ വളരും, പക്ഷേ ഏറ്റവും ചെലവ് കുറഞ്ഞ വഴി ഉറപ്പായും കിട്ടും. Goal OPEN-ൽ വന്നാൽ ഉടനെ നിർത്തരുത്; അത് OPEN-ൽ നിന്ന് select ചെയ്യുമ്പോൾ മാത്രമേ നിർത്താവൂ.

$f(n) = g(n)$ = cost of the path from start to n

Select from OPEN the node with the least g ; on expansion, for each successor m : $g(m) = g(n) + c(n, m)$; keep the smaller g if m is already in OPEN

Complete: yes, if every step cost $\geq \epsilon > 0$; Optimal: yes

Time and space: $O(b^{1 + \lceil C^*/\epsilon \rceil})$, C^* = cost of the optimal solution

EXAMPLE (TRACE)

Apply UCS to the Kerala map; the number in brackets is g (km from Kochi).

Step	Selected (least g)	OPEN sorted by g (after step)	CLOSED	Remark
1	–	KOC(0)	{ }	start
2	KOC(0)	KDR(35), TSR(75)	KOC	expand
3	KDR(35)	TSR(75), GVR(80)	KOC, KDR	expand
4	TSR(75)	GVR(80), PKD(140), MLP(155)	KOC, KDR, TSR	expand
5	GVR(80)	PKD(140), MLP(155), KZD(180)	KOC, KDR, TSR, GVR	expand
6	PKD(140)	MLP(155), KZD(180)	KOC, KDR, TSR, GVR, PKD	expand
7	MLP(155)	KZD(180)	KOC, KDR, TSR, GVR, PKD, MLP	expand
8	KZD(180)	{ }	KOC, KDR, TSR, GVR, PKD, MLP, KZD	KZD is the goal: stop, cost 180

Expansion order: KOC → KDR → TSR → GVR → PKD → MLP → KZD. Path KOC → KDR → GVR → KZD, cost 180 km (optimal). Note: KZD entered OPEN with g = 180 at step 5 but was not selected until step 8, after PKD (140) and MLP (155); MLP → KZD would cost 205, so the stored 180 was kept.

Exam tip: UCS with positive costs is the same as Dijkstra's algorithm (the unit notes this in Check Your Progress 2). Compare its 7 selections here with A*'s 5 selections in the A* topic: the heuristic saves work.

Bidirectional search

Definition: Bidirectional search runs two searches at once, forward from the start and backward from the goal, and stops when the two frontiers meet.

start-ൽ നിന്ന് forward ആയും goal-ൽ നിന്ന് backward ആയും ഒരേസമയം രണ്ട് searches run ചെയ്ത്, രണ്ട് frontiers-ഉം കൂടിച്ചേർന്നപ്പോൾ നിർത്തുന്ന search ആണ് Bidirectional search.

Bidirectional search runs two searches at once: forward from the start and backward from the goal, and checks at each stage whether a node generated by one search has already been generated by the other. When the two frontiers meet, the solution is the forward path joined to the reversed backward path. It is used when there is a single, explicitly known goal and every operator has an inverse (predecessors are easy to generate). In the unit's figure, the forward search visits 1, 4, 8, 9 and the backward search visits 16, 12, 10, 9, meeting at intersection node 9.

മലയാളത്തിൽ

കൊച്ചിയിൽ നിന്ന് ഒരു സൂപ്പർമാർക്കറ്റ് കോഴിക്കോട്ടുനിന്ന് മറ്റൊരു സൂപ്പർമാർക്കറ്റ് ഒരേ സമയം പുറപ്പെട്ട് നടുവിൽ എവിടെയെങ്കിലും കണ്ടുമുട്ടുന്നതുപോലെ. രണ്ടുപേരും $d/2$ ആഴം മാത്രം പോയാൽ മതി, അതുകൊണ്ട് b^d -ന് പകരം $2b^{d/2}$ nodes മതി. പക്ഷേ goal എന്താണെന്ന് മുൻകൂട്ടി അറിയണം, തിരിച്ച് നടക്കാനും (backward) കഴിയണം.

Nodes expanded $\approx 2b^{d/2} = O(b^{d/2})$

Time = Space = $O(b^{d/2})$

Complete: yes (BFS in both directions); Optimal: yes, if step costs are uniform in both directions

EXAMPLE (TRACE)

Run bidirectional BFS on the Kerala map, alternating one expansion forward (from KOC) and one backward (from KZD). After each generated node, check whether it is in the other search's OPEN $\cup$ CLOSED.

Step	Action	Forward OPEN	Forward CLOSED	Backward OPEN	Backward CLOSED	Check
1	start	KOC	{}	KZD	{}	no common node
2	forward expands KOC	KDR, TSR	KOC	KZD	{}	no common node
3	backward expands KZD	KDR, TSR	KOC	GVR, MLP	KZD	no common node
4	forward expands KDR	TSR, GVR	KOC, KDR	GVR, MLP	KZD	meet at GVR

The forward search generates GVR, which is already in the backward OPEN. Forward path KOC $\rightarrow$ KDR $\rightarrow$ GVR plus backward path GVR $\rightarrow$ KZD gives KOC $\rightarrow$ KDR $\rightarrow$ GVR $\rightarrow$ KZD, cost 180 km. Only 3 expansions were needed.

EXAMPLE

For $b = 10$ and $d = 6$, compare BFS with bidirectional search.

BFS: about $b^d = 10^6 = 1\,000\,000$ nodes. Bidirectional: $2 \times 10^3 = 2000$ nodes, 500 times fewer.

Advantages: much faster and uses less memory (each half can use BFS, DFS or DLS). **Disadvantages:** hard to implement; the goal state must be known in advance; checking every new node against the other frontier adds overhead.

Comparison of uninformed search strategies

The table collects the four criteria for all the uninformed methods, with b , d and m as defined earlier, l the depth limit, C^* the optimal cost and ϵ the smallest step cost.

മലയാളത്തിൽ

Exam-ൽ ഏറ്റവും കൂടുതൽ ചോദിക്കുന്ന table ആണിത്. BFS, IDDFS, Bidirectional എന്നിവ complete-ഉം (equal costs-ൽ) optimal-ഉം ആണ്; DFS, DLS അല്ല. Memory-യിൽ DFS, DLS, IDDFS ആണ് മികച്ചത്.

Criterion	BFS	DFS	DLS	IDDFS	UCS	Bidirectional
Time	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{d/2})$
Space	$O(b^d)$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{d/2})$
Optimal?	Yes*	No	No	Yes*	Yes	Yes*
Complete?	Yes	No	Only if $l \geq d$	Yes	Yes (costs $\geq \epsilon$)	Yes

* when all step costs are equal.

Exam tip: In Table 1 of the unit (and its summary and answer key) the DFS time is printed as b^d . The correct entry is $O(b^m)$: DFS may explore to the maximum depth m before finding a goal at depth d . The unit's own text in 3.3.4 gives $O(b^m)$.

Informed (heuristic) search and the heuristic function

Definition: A heuristic function $h(n)$ estimates the cost of the cheapest path from node n to a goal node, computed from the state description; $h(\text{goal}) = 0$.

state description-ൽ നിന്ന് കണക്കാക്കുന്ന, node n -ൽ നിന്ന് goal node-ലേക്കുള്ള ഏറ്റവും cheap ആയ path-ന്റെ cost-ന്റെ estimate ആണ് heuristic function $h(n)$; $h(\text{goal}) = 0$.

Informed search uses domain-dependent information beyond the problem definition to search more efficiently. Heuristic information can be used to decide which node to expand next, which successors to generate, and which nodes to prune. It is supplied as a **heuristic function $h(n)$** : an estimate of the cost of the cheapest path from node n to a goal, computed from the state description; $h(\text{goal}) = 0$. A heuristic may not always give the best solution, but with a good heuristic it gives a good solution in reasonable time. For route finding, the natural heuristic is the straight-line (Euclidean) distance to the destination, h_{SLD} .

മലയാളത്തിൽ

Heuristic എന്നാൽ ഒരു നല്ല ഊഹം. കോഴിക്കോട്ടേക്ക് റോഡ് വഴി എത്ര km എന്ന് അറിയില്ലെങ്കിലും, map-ൽ ഒരു സ്ക്രീനിൽ വെച്ച് നേർരേഖാ ദൂരം അളക്കാം; അതാണ് $h(n)$. റോഡ് ഒരിക്കലും നേർരേഖയേക്കാൾ ചെറുതാകില്ല, അതുകൊണ്ട് ഈ ഊഹം യഥാർത്ഥ ദൂരത്തേക്കാൾ കൂടില്ല. ഈ ഒരു വിവരം കൊണ്ട് search തെറ്റായ ദിശയിലേക്ക് (ഉദാ: Palakkad) പോകുന്നത് കുറയും.

$h(n)$ = estimated cost of the cheapest path from n to a goal node; $h(\text{goal}) = 0$

Route finding: $h(n)$ = straight-line distance(n , goal), e.g. $h(\text{Kochi}) = \text{SLD}(\text{Kochi}, \text{Kozhikode}) = 160$

Informed state space = 5-tuple $[S, s_0, O, G, h]$

S = set of states, $s_0 \in S$ start state, O = operators (arcs), $G \subseteq S$ goal states, h = heuristic function

EXAMPLE

Compare the straight-line estimate $h(n)$ with the true shortest road distance $h^*(n)$ to Kozhikode for every town on the Kerala map.

Town	KOC	KDR	TSR	GVR	PKD	MLP	KZD
$h(n)$ (straight line)	160	125	100	85	105	40	0
$h^*(n)$ (best road)	180	145	125	100	140	50	0

Every $h(n) \leq h^*(n)$: the heuristic never overestimates. (h^* was found by running uniform cost search from each town; e.g. $\text{PKD} \rightarrow \text{MLP} \rightarrow \text{KZD} = 90 + 50 = 140$.)

Best-first search keeps the same two lists: OPEN holds nodes that have been generated and evaluated but not yet examined; CLOSED holds nodes already examined.

Best-first search and greedy best-first search

Best-first search: search that expands the node in OPEN with the best (lowest) value of an evaluation function $f(n)$.

Greedy best-first search: best-first search with $f(n) = h(n)$: it always expands the node that appears closest to the goal.

Best-first search: evaluation function $f(n)$ -ന്റെ ഏറ്റവും മികച്ച (ഏറ്റവും കുറഞ്ഞ) value ഉള്ള OPEN-ലെ node expand ചെയ്യുന്ന search.

Greedy best-first search: $f(n) = h(n)$ ഉള്ള best-first search: goal-നോട് ഏറ്റവും അടുത്തതായി തോന്നുന്ന node എപ്പോഴും expand ചെയ്യുന്നു.

Best-first search uses an evaluation function $f(n)$ to decide which node in OPEN to expand next (OPEN is a priority queue). Its two main forms are greedy best-first search and A^* . **Greedy best-first search** uses $f(n) = h(n)$: it always expands the node that appears closest to the goal, ignoring how much has already been spent getting there. It behaves like DFS when it follows one promising path and like BFS when it switches between branches.

മലയാളത്തിൽ

Greedy search ഓരോ junction-ലും "കോഴിക്കോട്ടേക്ക് ഏറ്റവും അടുത്തെന്ന് തോന്നുന്ന" town തിരഞ്ഞെടുക്കും; ഇതുവരെ എത്ര ഓടി എന്ന് നോക്കില്ല. അത്യാഗ്രഹി ആയതുകൊണ്ട് വേഗം ഒരു വഴി കിട്ടും, പക്ഷേ അത് ഏറ്റവും ചെറിയ വഴി ആകണമെന്നില്ല. താഴത്തെ trace-ൽ Thrissur ($h = 100$) Kodungallur-നേക്കാൾ (125) അടുത്തെന്ന് തോന്നിയതുകൊണ്ട് greedy 205 km-ന്റെ വഴി എടുക്കുന്നു, 180 km-ന്റെ വഴി വിട്ടുപോകുന്നു.

Greedy best-first: $f(n) = h(n)$

1. Initialize: OPEN = {s}, CLOSED = { }, $f(s) = h(s)$
2. Fail: if OPEN = { }, terminate with failure
3. Select: the state n in OPEN with minimum f ; save n in CLOSED
4. Terminate: if $n \in \text{Goal}$, terminate with success
5. Expand: for each successor m of n , if $m \notin \text{OPEN} \cup \text{CLOSED}$, set $f(m) = h(m)$ and insert m in OPEN
6. Loop: go to step 2

Time = Space = $O(b^m)$ worst case; Complete: no; Optimal: no

EXAMPLE (TRACE)

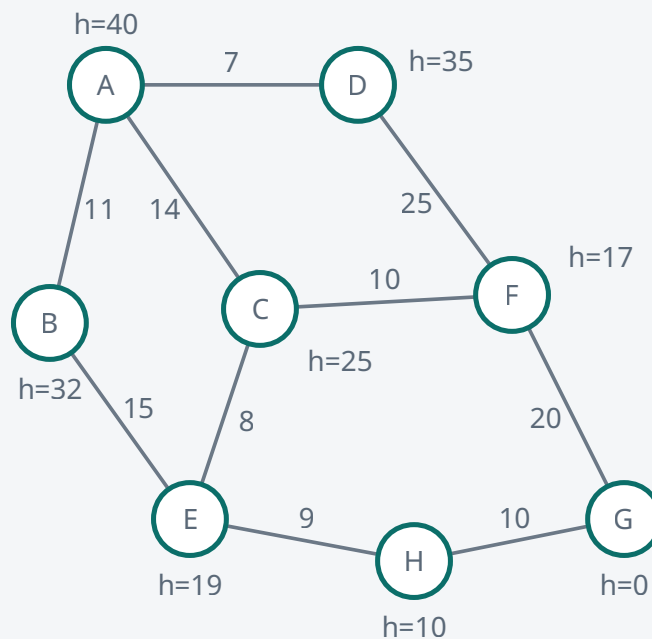
Apply greedy best-first search to the Kerala map; the number in brackets is h.

Step	Selected (least h)	OPEN sorted by h (after step)	CLOSED
1	–	KOC(160)	{}
2	KOC(160)	TSR(100), KDR(125)	KOC
3	TSR(100)	MLP(40), GVR(85), PKD(105), KDR(125)	KOC, TSR
4	MLP(40)	KZD(0), GVR(85), PKD(105), KDR(125)	KOC, TSR, MLP
5	KZD(0)	GVR(85), PKD(105), KDR(125)	KOC, TSR, MLP, KZD

Expansion order: KOC → TSR → MLP → KZD. Path KOC → TSR → MLP → KZD = 75 + 80 + 50 = 205 km. Only 4 selections, but the answer is 25 km longer than the optimal 180 km: greedy is fast but **not optimal**.

EXAMPLE (UNIT'S EXAMPLE 1)

Apply greedy best-first search from A to goal G on the graph below (edge costs on edges, straight-line h beside each node).



The unit's greedy best-first example. Start A, goal G.

Step	Selected (least h)	OPEN sorted by h (after step)	CLOSED
1	–	A(40)	{ }
2	A(40)	C(25), B(32), D(35)	A
3	C(25)	F(17), E(19), B(32), D(35)	A, C
4	F(17)	G(0), E(19), B(32), D(35)	A, C, F
5	G(0)	E(19), B(32), D(35)	A, C, F, G

Greedy path A → C → F → G, cost 14 + 10 + 20 = 44.

Exam tip: The unit calls A → C → F → G (cost 44) "the shortest path". It is not: A → C → E → H → G costs 14 + 8 + 9 + 10 = 41, and A* with the same h finds it ($f(C) = 14 + 25 = 39$, then $f(E) = 22 + 19 = 41$, $f(H) = 31 + 10 = 41$, $f(G) = 41$ via H against 44 via F). Greedy search only guarantees a quick answer, not the shortest one. The unit also writes the greedy evaluation function as " $f(n) = g(n)$ "; it should be $f(n) = h(n)$.

EXAMPLE (UNIT'S EXAMPLE 3)

Start S has children A(2), B(6), C(5); A → D(10), E(8); B → F(13), G(14); C → H(7); H → I(5), J(6); I → K(1), L(0), M(2). L is the goal. Apply greedy best-first search.

Select S → OPEN A2, C5, B6. Select A → OPEN C5, B6, E8, D10. Select C → OPEN B6, H7, E8, D10. Select B → OPEN H7, E8, D10, F13, G14. Select H → OPEN I5, J6, E8, D10, F13, G14. Select I → OPEN L0, K1, M2, J6, ... Select L: goal. Path S → C → H → I → L.

The unit's table stops when L is generated (step 6) and lists H again among the available nodes after it was expanded; H is in CLOSED by then and should not appear.

Advantages: can switch between BFS-like and DFS-like behaviour; usually far more efficient than blind search.

Disadvantages: can get stuck in loops (without CLOSED); in the worst case behaves like an unguided DFS; not optimal. Without repeated-state checking it is incomplete even in a finite space; with the CLOSED list it is complete on finite graphs.

A* algorithm

Definition: A* is a best-first search that expands the node with minimum $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start to n and $h(n)$ the estimated cost from n to the goal.

$f(n) = g(n) + h(n)$ ഏറ്റവും കുറഞ്ഞ node expand ചെയ്യുന്ന best-first search ആണ് A*; $g(n)$ start-ൽ നിന്ന് n വരെയുള്ള cost, $h(n)$ n -ൽ നിന്ന് goal വരെയുള്ള estimated cost.

A* is the best-known form of best-first search. It evaluates each node by $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start to n and $h(n)$ is the estimated cost from n to the goal, so $f(n)$ estimates the cost of the cheapest solution through n . It combines uniform cost search (g) with greedy search (h). If a node is reached again by a cheaper path, its g and f are lowered and, if it was in CLOSED, it is moved back to OPEN. With an admissible h , the first goal selected from OPEN is optimal.

മലയാളത്തിൽ

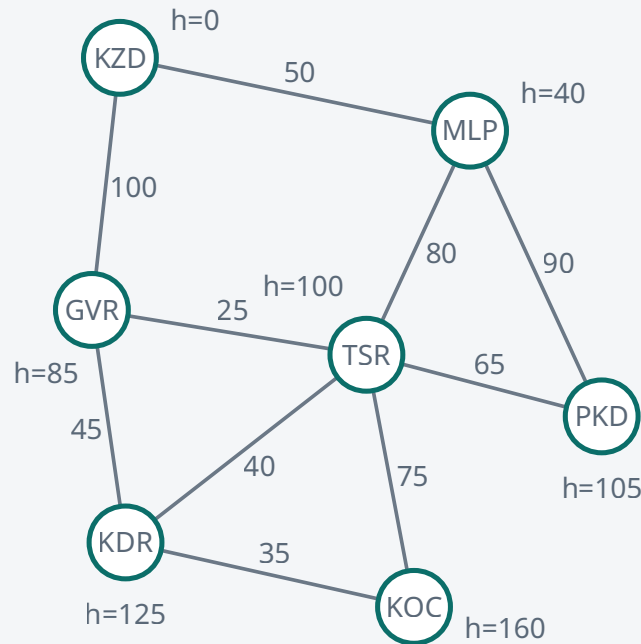
A* രണ്ടും നോക്കും: ഇതുവരെ ഓടിയ km (g) + ഇനി ഏകദേശം എത്ര (h). ഒരു ടാക്സി ഡ്രൈവർ "Thrissur വഴി പോയാൽ ഇതുവരെ 75, ഇനി ഏകദേശം 100, ആകെ 175; Kodungallur വഴി ഇതുവരെ 35, ഇനി ഏകദേശം 125, ആകെ 160" എന്ന് കണക്കുകൂട്ടുന്നതുപോലെ. ആകെ (f) ഏറ്റവും കുറഞ്ഞത് ആദ്യം expand ചെയ്യും. h ഒരിക്കലും യഥാർത്ഥ ദൂരത്തേക്കാൾ കൂടുതൽ പറയാത്തതിടത്തോളം A* ഏറ്റവും ചെറിയ വഴി തന്നെ തരും.

$$f(n) = g(n) + h(n)$$

1. Initialize: $OPEN = \{s\}$, $CLOSED = \{\}$, $g(s) = 0$, $f(s) = h(s)$
2. Fail: if $OPEN = \{\}$, terminate with failure
3. Select: the state n in $OPEN$ with minimum f ; save n in $CLOSED$
4. Terminate: if $n \in \text{Goal}$, terminate with success and return $f(n)$
5. Expand: for each successor m of n :
 - if $m \notin OPEN \cup CLOSED$: $g(m) = g(n) + c(n, m)$; $f(m) = g(m) + h(m)$; insert m in $OPEN$
 - if $m \in OPEN \cup CLOSED$: $g(m) = \min\{g(m), g(n) + c(n, m)\}$; $f(m) = g(m) + h(m)$
 - if $f(m)$ has decreased and $m \in CLOSED$, move m to $OPEN$
6. Loop: go to step 2

EXAMPLE (TRACE)

Apply A* to the Kerala map from KOC to KZD. The number in brackets is $f = g + h$.



Same map: road km on roads, straight-line h to Kozhikode beside each town.

Step 2: KDR: $g = 35, f = 35 + 125 = 160$; TSR: $g = 75, f = 75 + 100 = 175$.

Step 3: GVR: $g = 35 + 45 = 80, f = 80 + 85 = 165$; TSR via KDR: $g = 35 + 40 = 75$, not lower than 75, no change.

Step 4: KZD: $g = 80 + 100 = 180, f = 180 + 0 = 180$; TSR via GVR: $g = 105 > 75$, no change.

Step 5: TSR (175) is still below KZD (180), so TSR is expanded: MLP $f = 155 + 40 = 195$, PKD $f = 140 + 105 = 245$; GVR via TSR $g = 100 > 80$ and KDR via TSR $g = 115 > 35$, no change.

Step 6: KZD (180) is now the least f: goal.

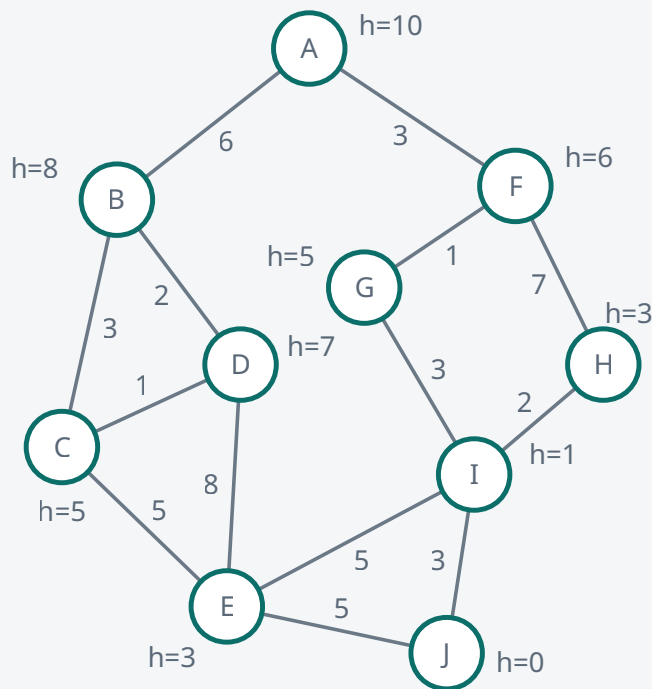
Step	Selected (least f)	OPEN sorted by f (after step)	CLOSED	Remark
1	–	KOC(160)	{ }	start
2	KOC(160)	KDR(160), TSR(175)	KOC	expand
3	KDR(160)	GVR(165), TSR(175)	KOC, KDR	expand
4	GVR(165)	TSR(175), KZD(180)	KOC, KDR, GVR	expand
5	TSR(175)	KZD(180), MLP(195), PKD(245)	KOC, KDR, GVR, TSR	expand
6	KZD(180)	MLP(195), PKD(245)	KOC, KDR, GVR, TSR, KZD	KZD is the goal: stop, cost 180

Expansion order: KOC → KDR → GVR → TSR → KZD. Path KOC → KDR → GVR → KZD, cost 180 km: optimal, with 5 selections against 7 for UCS, and without the greedy mistake.

Exam tip: Do not stop when the goal is generated (step 4 above); stop when it is selected as the minimum- f node. Here TSR (175) still had to be checked, because a route through Thrissur might have been cheaper than 180.

EXAMPLE (UNIT'S EXAMPLE 1)

Find the most cost-effective path from A to J using A*. Edge costs are on the edges; h is written beside each node.



The unit's A* example graph. Start A, goal J.

Step	Selected (least f)	OPEN sorted by f (after step)	CLOSED	Remark
1	–	A(10)	{ }	start
2	A(10)	F(9), B(14)	A	expand
3	F(9)	G(9), H(13), B(14)	A, F	expand
4	G(9)	I(8), H(13), B(14)	A, F, G	expand
5	I(8)	J(10), H(12), B(14), E(15)	A, F, G, I	H: g lowered 10 → 9
6	J(10)	H(12), B(14), E(15)	A, F, G, I, J	J is the goal: stop, cost 10

Working: $f(B) = 6 + 8 = 14$, $f(F) = 3 + 6 = 9$, so F. $f(G) = (3 + 1) + 5 = 9$, $f(H) = (3 + 7) + 3 = 13$, so G. $f(I) = (3 + 1 + 3) + 1 = 8$, so I. From I: $f(E) = 12 + 3 = 15$, $f(H) = (3 + 1 + 3 + 2) + 3 = 12$ (lower than 13, so H's g drops from 10 to 9), $f(J) = 10 + 0 = 10$. J has the least f: goal.

Path $A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$, cost $3 + 1 + 3 + 3 = 10$.

Exam tip: The unit's text for this example has typos: "Since $f(F) < f(F)$ " should read $f(F) < f(B)$, and " $f(G) = (3+1)=5=9$ " should read $f(G) = (3 + 1) + 5 = 9$.

EXAMPLE (UNIT'S EXAMPLE 2)

Directed arcs: $S \rightarrow A$ 1, $S \rightarrow G$ 10, $A \rightarrow B$ 2, $A \rightarrow C$ 1, $B \rightarrow D$ 5, $C \rightarrow D$ 3, $C \rightarrow G$ 4, $D \rightarrow G$ 2. Heuristic: $h(S) = 5$, $h(A) = 3$, $h(B) = 4$, $h(C) = 2$, $h(D) = 6$, $h(G) = 0$. Find the cheapest path from S to G with A*.

Step	Selected (least f)	OPEN sorted by f (after step)	CLOSED	Remark
1	–	S(5)	{ }	start
2	S(5)	A(4), G(10)	S	expand
3	A(4)	C(4), B(7), G(10)	S, A	expand
4	C(4)	G(6), B(7), D(11)	S, A, C	G: g lowered 10 → 6
5	G(6)	B(7), D(11)	S, A, C, G	G is the goal: stop, cost 6

Path $S \rightarrow A \rightarrow C \rightarrow G$, cost $1 + 1 + 4 = 6$. The unit lists the f-values of all partial paths ($S \rightarrow A = 4$, $S \rightarrow A \rightarrow C = 4$, $S \rightarrow A \rightarrow C \rightarrow G = 6$, $S \rightarrow A \rightarrow B = 7$, ...) but never states the answer: the optimal path is $S \rightarrow A \rightarrow C \rightarrow G$ with cost 6.

Exam tip: In the unit's Example 2, $h(D) = 6$ overestimates: the true cost from D is $D \rightarrow G = 2$. So that heuristic is not admissible; A* still happens to find the optimum here, but it is not guaranteed.

Admissibility and properties of A*

Admissible heuristic: a heuristic that never overestimates the true cost to the goal: $h(n) \leq h^*(n)$ for every n .

Admissible algorithm: a search algorithm that always terminates with an optimal solution when one exists; A* with an admissible h is admissible.

Consistent (monotone) heuristic: a heuristic with $h(n) \leq c(n, m) + h(m)$ for every arc $n \rightarrow m$.

Admissible heuristic: goal വരെയുള്ള യഥാർത്ഥ cost-നെ ഒരിക്കലും കൂട്ടി estimate ചെയ്യാത്ത heuristic: എല്ലാ n -നും $h(n) \leq h^*(n)$.

Admissible algorithm: ഒരു solution ഉണ്ടെങ്കിൽ എപ്പോഴും optimal solution-ഓടെ അവസാനിക്കുന്ന search algorithm; admissible h ഉള്ള A* admissible ആണ്.

Consistent (monotone) heuristic: എല്ലാ arc $n \rightarrow m$ -നും $h(n) \leq c(n, m) + h(m)$ ആകുന്ന heuristic.

A heuristic is **admissible** if it never overestimates: $h(n) \leq h^*(n)$ for every n , where $h^*(n)$ is the true minimum cost from n to a goal. A* with an admissible heuristic is **admissible**: if a path to a goal exists, it terminates with an optimal one. Three conditions guarantee this: (1) every node has a finite number of successors; (2) every arc cost is greater than some positive amount ϵ ; (3) $h(n) \leq h^*(n)$ for every node. Admissible heuristics are optimistic, like air distance versus road distance.

മലയാളത്തിൽ

Admissible heuristic എന്നാൽ എപ്പോഴും "ശുഭാപ്തി" ഉള്ള ഊഹം: യഥാർത്ഥ ദൂരത്തേക്കാൾ കൂടുതൽ ഒരിക്കലും പറയില്ല. നേർരേഖാ ദൂരം റോഡ് ദൂരത്തേക്കാൾ കൂടില്ലല്ലോ, അതുകൊണ്ട് അത് admissible ആണ്. $h = 0$ ആക്കിയാൽ A* വെറും UCS ആകും (വിവരം ഇല്ല); $h = h^*$ ആണെങ്കിൽ നേരെ goal-ലേക്ക്, ഒരു അനാവശ്യ node പോലും expand ചെയ്യാതെ പോകും. h യഥാർത്ഥ ദൂരത്തോട് എത്ര അടുത്താണോ, അത്രയും കുറച്ച് nodes മതി.

Admissible: $h(n) \leq h^*(n)$ for all n

$h(n) = 0$ for all $n \Rightarrow A^* =$ uniform cost search (= BFS only if all step costs are equal)

$h(n) = h^*(n) \Rightarrow A^*$ goes straight along an optimal path

Dominance: if $h_1(n) \geq h_2(n)$ for all n (both admissible), A* with h_1 is more informed and expands no more nodes than A* with h_2

$\max(h_1, h_2)$ of admissible heuristics is admissible and dominates both

Consistent (monotone): $h(n) \leq c(n, m) + h(m)$ for every arc $n \rightarrow m$

EXAMPLE

Show that the straight-line heuristic on the Kerala map is admissible, and check consistency on two roads.

Admissible: from the table in the heuristic topic, $160 \leq 180$, $125 \leq 145$, $100 \leq 125$, $85 \leq 100$, $105 \leq 140$, $40 \leq 50$, $0 \leq 0$. Every $h(n) \leq h^*(n)$.

Consistent: KOC-KDR: $h(\text{KOC}) = 160 \leq 35 + h(\text{KDR}) = 35 + 125 = 160 \checkmark$. TSR-MLP: $h(\text{TSR}) = 100 \leq 80 + 40 = 120 \checkmark$. (All ten roads satisfy it.)

EXAMPLE

What happens to the Kerala A* trace if we use $h = 0$ everywhere?

Then $f = g$ and A* becomes uniform cost search: the selections become KOC, KDR, TSR, GVR, PKD, MLP, KZD (7 instead of 5). The answer is still 180 km, but more nodes are examined. A better-informed heuristic does less work.

Results on A*: (1) A* is admissible: it finds an optimal path when one exists. (2) A* is complete: if a path exists it terminates, even on an infinite graph (given conditions 1 and 2). (3) Dominance: if A1 is more informed than A2, A2 expands at least as many states as A1.

Advantages: optimal and complete; among the best general search algorithms; solves complex problems.

Disadvantages: keeps every generated node in memory (space is its main drawback, exponential in the worst case); the quality depends on the heuristic.

Exam tip: The unit states the dominance property as " $h_1(n) > h_2(n)$ "; the usual statement is $h_1(n) \geq h_2(n)$ for all n (both admissible). The unit also says A* "does not always produce the shortest path"; that is true only when h is not admissible. And its note that "A* allows negative edge costs" needs care: the unit's own admissibility conditions require every arc cost to exceed a positive amount, so optimality is not guaranteed with negative costs.

Problem reduction and AND-OR graphs

Problem reduction: decomposing a problem into sub-problems, solving them and combining their solutions (divide and conquer).

AND-OR graph: a graph of such decompositions in which an OR node needs any one successor solved and an AND arc needs all its successors solved.

Problem reduction: ഒരു problem-നെ sub-problems ആയി വിഭജിച്ച് അവ solve ചെയ്ത് അവയുടെ solutions ചേർക്കൽ (divide and conquer).

AND-OR graph: ഇത്തരം decompositions കാണിക്കുന്ന graph; ഇതിൽ OR node-ന് ഏതെങ്കിലും ഒരു successor solve ആയാൽ മതി, AND arc-ന് അതിന്റെ എല്ലാ successors-ഉം solve ആകണം.

Problem reduction decomposes a problem into sub-problems (divide and conquer), solves them, and combines the sub-solutions. Such decompositions are drawn as an **AND-OR graph**. An **OR** node offers alternative ways to solve a problem (solve any one successor). An **AND** arc, drawn as a curve across several edges, means all those successors must be solved. The unit's example: "Get a bike" = (steal a bike) OR (get some money AND buy a bike). A problem is given as $[G, s, T]$: G the implicit AND-OR graph, s the start node, T the terminal (SOLVED) nodes, with $h(n)$ estimating the cost of solving the sub-problem at n .

മലയാളത്തിൽ

ഒരു വലിയ പണി ചെറിയ പണികളായി മുറിക്കുന്നതാണ് problem reduction. കല്യാണം നടത്താൻ "ഓഡിറ്റോറിയം ബുക്ക് ചെയ്യുക AND സദ്യ ഏർപ്പാടാക്കുക" രണ്ടും വേണം (AND). സദ്യക്ക് "കാറ്ററിംഗ് OR വീട്ടിൽ പാചകം", ഏതെങ്കിലും ഒന്ന് മതി (OR). AND node-ന്റെ cost എല്ലാ children-ന്റെയും cost-ന്റെ തുക; OR node-ന്റെ cost ഏറ്റവും കുറഞ്ഞ child-ന്റേത്.

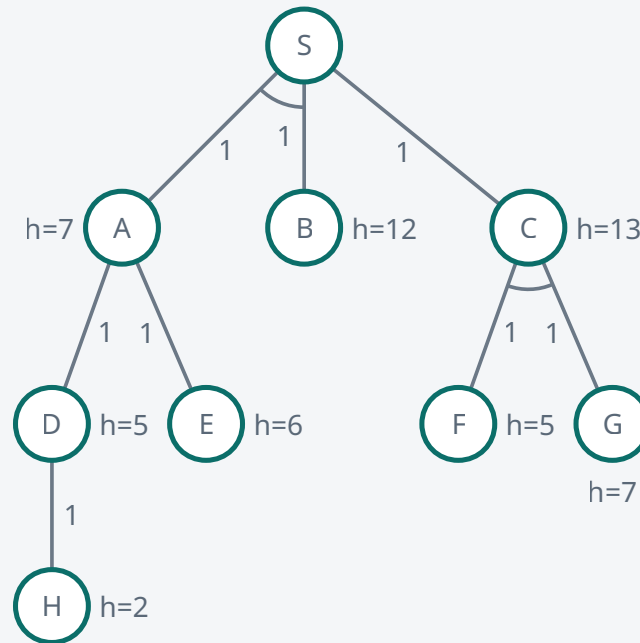
For an AND arc from m to $r_1, \dots, r_k$: $\text{cost} = \sum [f(r_i) + c(m, r_i)]$

For an OR choice among $r_1, \dots, r_k$: $\text{cost} = \min \{ f(r_i) + c(m, r_i) \}$

After any change, revise costs bottom-up (from the changed node back to the root)

EXAMPLE (UNIT'S EXAMPLE 1)

In the AND-OR graph below every edge costs 1. S can be solved either through the AND arc (A and B) or through C. C has an AND arc to F and G; A is an OR node over D and E; D leads to H with $h(H) = 2$. Find the optimal cost.



Curved arcs mark AND connectors: $S \rightarrow (A \text{ AND } B)$, $C \rightarrow (F \text{ AND } G)$.

Initial estimates: via (A, B): $1 + 1 + 7 + 12 = 21$. Via C: $1 + 13 = 14$. So C is marked.

Expand C (AND node): $f(C) = (1 + 5) + (1 + 7) = 14$. Revise S: via C = $1 + 14 = 15$, still better than 21.

For comparison, the unit also explores A: A is an OR node, $f(A-D) = 1 + 5 = 6$, $f(A-E) = 1 + 6 = 7$, so $f(A) = 6$; expanding D gives $f(D) = 1 + 2 = 3$, so $f(A) = 1 + 3 = 4$. Then via (A, B): $1 + 1 + 4 + 12 = 18$.

Final: path 1 (S, A, B) costs 18, path 2 (S, C) costs 15. Optimal cost = **15**, through C, F and G.

Exam tip: The unit writes " $f(C-F-G) = 1 + 1 + 5 + 7 = 14$ "; so the revised cost of node C is 14". Always show the bottom-up revision to the root afterwards ($S = 1 + 14 = 15$); marks are lost for giving only the child's cost.

AO* algorithm

Definition: AO* is a best-first search on an AND-OR graph that finds a minimum-cost solution graph by expanding the best partial solution and revising costs bottom-up.

ഏറ്റവും മികച്ച partial solution expand ചെയ്ത് costs bottom-up ആയി revise ചെയ്ത് minimum-cost solution graph കണ്ടെത്തുന്ന, AND-OR graph-ലെ best-first search ആണ് AO*.

AO* searches an AND-OR graph for a minimum-cost **solution graph**. Instead of OPEN and CLOSED it keeps one structure, GRAPH (G^*), the part of the graph generated so far; each node points to its successors and predecessors and carries an estimate $h'(n)$ (no g is stored, since a node may be reached along many paths). It repeatedly expands a non-terminal leaf of the currently marked best partial solution, then revises costs bottom-up, re-marking the best connector at each OR choice. It stops when the start node is labelled SOLVED. An AND node is SOLVED when all its successors are SOLVED; an OR node when its marked successor is SOLVED.

മലയാളത്തിൽ

AO* ഓരോ തവണയും ഇപ്പോൾ ഏറ്റവും നല്ലതെന്ന് mark ചെയ്ത വഴിയിലെ ഒരു leaf മാത്രം expand ചെയ്യും, എന്നിട്ട് താഴെ നിന്ന് മുകളിലേക്ക് cost revise ചെയ്യും. Revise ചെയ്തപ്പോൾ വേറൊരു വഴി ഇപ്പോൾ കുറഞ്ഞതായി തോന്നിയാൽ mark ആ വഴിയിലേക്ക് മാറ്റും. A* ഒരു വഴി (OR) മാത്രം കണ്ടുപിടിക്കുമ്പോൾ, AO* ഒരുമിച്ച് ചെയ്യേണ്ട പല sub-problems (AND) ഉള്ള ഒരു solution tree ആണ് തരുന്നത്.

AO*:

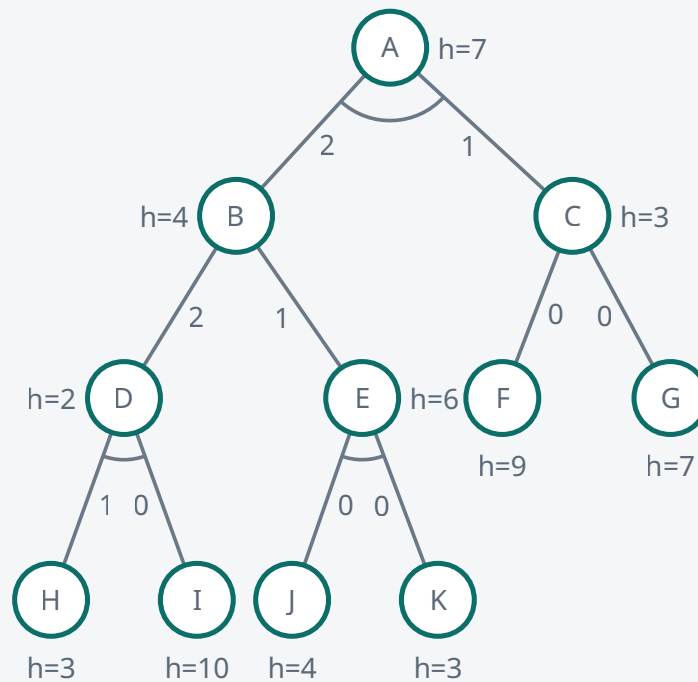
1. Initialize: $G^* = \{s\}$, $f(s) = h(s)$; if $s \in T$, label s SOLVED
2. Terminate: if s is SOLVED, terminate
3. Select: a non-terminal leaf node n of the marked sub-graph
4. Expand: generate the successors of n ; for each new successor m set $f(m) = h(m)$; if m is terminal, label m SOLVED
5. Cost revision: call Cost-Revise(n)
6. Loop: go to step 2

Cost-Revise(n):

1. $Z = \{n\}$; 2. if $Z = \{\}$ return
3. select m from Z such that m has no descendants in Z
4. AND node m with successors $r_1 \dots r_k$: $f(m) = \sum [f(r_i) + c(m, r_i)]$; mark all; SOLVED if every r_i is SOLVED
5. OR node m : $f(m) = \min\{f(r_i) + c(m, r_i)\}$; mark the best; SOLVED if the marked successor is SOLVED
6. if the cost or label of m changed, insert into Z the parents of m for which m is a marked successor; go to 2

EXAMPLE (UNIT'S EXAMPLE 2)

A, D, E are AND nodes; B, C are OR nodes; edge costs and h-values as shown; the leaves F, G, H, I, J, K are terminal. Apply AO*.



Curved arcs mark AND connectors at A, D and E.

Cycle	Node expanded	Cost revision (bottom-up)	Marked best	f(A)
1	A (AND)	$f(A) = (2 + 4) + (1 + 3) = 10$	A → B, C	10
2	C (OR)	$f(C) = \min(0 + 9, 0 + 7) = 7$, C SOLVED (G terminal); $f(A) = (2 + 4) + (1 + 7) = 14$	C → G	14
3	B (OR)	$f(B) = \min(2 + 2, 1 + 6) = 4$; $f(A) = (2 + 4) + (1 + 7) = 14$	B → D	14
4	D (AND)	$f(D) = (1 + 3) + (0 + 10) = 14$, D SOLVED; $f(B) = \min(2 + 14, 1 + 6) = 7$; $f(A) = (2 + 7) + 8 = 17$	B → E (switch)	17
5	E (AND)	$f(E) = (0 + 4) + (0 + 3) = 7$, E SOLVED; $f(B) = \min(16, 1 + 7) = 8$, SOLVED; $f(A) = (2 + 8) + (1 + 7) = 18$, A SOLVED	B → E	18

Solution graph: A → (B AND C), B → E → (J AND K), C → G. Minimum cost = 18.

Exam tip: The unit jumps from "D = 14, E = 7" straight to the answer 18. Show the step where B's mark switches from D to E once D's cost rises to 14; this switching is the heart of AO*.

A*	AO*
Works on an OR graph; finds a single path (this or that).	Works on an AND-OR graph; finds a solution graph that may AND several branches.
Uses OPEN and CLOSED lists; stores g(n).	Uses one structure GRAPH; stores only h'(n), revised bottom-up.
$f(n) = g(n) + h(n)$.	AND: sum over successors; OR: minimum over successors.
Optimal with admissible h.	Finds a minimum-cost solution if $h'(n) \leq h^*(n)$ and arc costs are positive; terminates even on graphs with cycles. With only OR nodes it works like A*.

Exam tip: The unit contradicts itself: 3.9 says AO* does not guarantee an optimal solution, while 3.9.3 says it always finds a minimum-cost solution if h' never overestimates. The second statement is the correct result; AO* may stop without exploring other solution graphs only because their estimated costs are already higher.

Memory-bounded search: Iterative Deepening A* (IDA*)

Definition: IDA* is iterative deepening with the depth limit replaced by an f-limit ($f = g + h$): each iteration is a DFS that prunes nodes above the limit, which then rises to the smallest pruned f-value.

depth limit-ന് പകരം f-limit ($f = g + h$) ഉപയോഗിക്കുന്ന iterative deepening ആണ് IDA*: ഓരോ iteration-ഉം limit കവിയുന്ന nodes prune ചെയ്യുന്ന ഒരു DFS ആണ്; അടുത്ത limit prune ചെയ്തവയിലെ ഏറ്റവും ചെറിയ f-value.

A* stores every generated node, so it runs out of memory on large problems. Memory-bounded heuristic searches fix this: Iterative Deepening A* (IDA*), Recursive Best-First Search (RBFS) and Memory-bounded A* (MBA*). IDA* is iterative deepening with the depth limit replaced by an **f-limit**: it starts with $\text{limit} = h(\text{start})$, does a depth-first search that prunes any node with $f(n) > \text{limit}$, and if the goal is not found, sets the next limit to the smallest f-value that was pruned. Each iteration is a plain DFS, so no priority queue is needed.

മലയാളത്തിൽ

IDA* ഒരു "budget" വെച്ച് DFS ചെയ്യുന്നു: $f(n)$ budget-നേക്കാൾ കൂടിയ node വെട്ടിക്കളയും (prune). Goal കിട്ടിയില്ലെങ്കിൽ, വെട്ടിക്കളഞ്ഞവയിൽ ഏറ്റവും കുറഞ്ഞ f ആണ് അടുത്ത budget. "ആദ്യം 160 km ബജറ്റിൽ നോക്കാം, പറ്റിയില്ലെങ്കിൽ 165, പിന്നെ 175..." എന്നപോലെ. Memory DFS-ന്റേത് പോലെ വളരെ കുറവ്, ഉത്തരം A*-ന്റേത് പോലെ optimal.

IDA*:

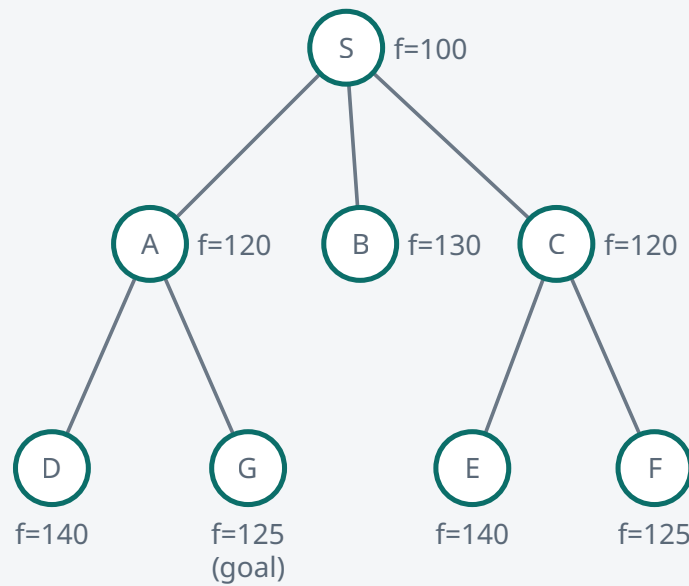
1. Set $C = f(s) = h(s)$
2. Perform depth-first branch-and-bound with cut-off C : expand a state n only if $f(n) \leq C$; if a goal is selected for expansion, return C and terminate
3. Set $C =$ the minimum f -value that exceeded C among the states examined; go to step 2

Complete and optimal (admissible h); space linear, about $b \cdot f^* / \delta$ nodes ($\delta =$ smallest step cost)

Worst case: if A* expands N states and each iteration adds only one, IDA* expands $1 + 2 + \dots + N = O(N^2)$

EXAMPLE (UNIT'S EXAMPLE)

S has $f = 100$; its children A(120), B(130), C(120); A's children D(140) and G(125, the goal); C's children E(140) and F(125). Trace IDA*.



The unit's IDA* tree with f -values; G is the goal.

Iteration	f -limit	Expanded	Pruned (f)	Next f -limit
1	100	S	A(120), B(130), C(120)	120
2	120	S, A, C	D(140), G(125), B(130), E(140), F(125)	125
3	125	S, A, G	D(140)	goal G selected: success

EXAMPLE (TRACE)

Apply IDA* to the Kerala map ($f = g + h$, successors in the usual order, towns already on the current path skipped).

Iteration	f-limit	Nodes expanded , DFS order (f)	Nodes pruned ($f > \text{limit}$)	Next f-limit = least pruned f
1	160	KOC(160), KDR(160)	GVR(165), TSR(175), TSR(175)	165
2	165	KOC(160), KDR(160), GVR(165)	KZD(180), TSR(205), TSR(175), TSR(175)	175
3	175	KOC(160), KDR(160), GVR(165), TSR(175), TSR(175)	KZD(180), TSR(205), GVR(185), MLP(195), PKD(245), GVR(185), KDR(240), MLP(195), PKD(245)	180
4	180	KOC(160), KDR(160), GVR(165), KZD(180)	–	goal KZD reached with $f = 180$

Four iterations with limits 160, 165, 175, 180; answer KOC → KDR → GVR → KZD, 180 km, the same optimal path as A*. IDA* never holds more than the current path and its siblings in memory, but it re-expands KOC, KDR and GVR in every iteration.

Exam tip: The next limit is the minimum pruned f , not "old limit + 1". In the Kerala trace, iteration 1 prunes GVR(165) and TSR(175), so the new limit is 165.

Recursive best-first search (RBFS)

Definition: Recursive best-first search (RBFS) is a linear-space recursive algorithm that imitates A* by remembering the f-value of the best alternative path and unwinding, with backed-up f-values, when the current path exceeds it.

ഏറ്റവും മികച്ച alternative path-ന്റെ f-value ഓർത്തുവെച്ച്, current path അത് കവിയുമ്പോൾ backed-up f-values-ഓടെ തിരിച്ചുപോകുന്ന, A*-നെ അനുകരിക്കുന്ന linear-space recursive algorithm ആണ് Recursive best-first search (RBFS).

RBFS imitates A* (best-first order) using only linear space, $O(bd)$. It is a recursive DFS that also remembers the f-value of the best **alternative** path available from any ancestor. If the current node's best child exceeds that limit, the recursion unwinds to the alternative, and each node on the way back is given the best f-value of its children (the "backed-up" value). It therefore remembers how good a forgotten subtree was and can decide later whether to re-expand it.

മലയാളത്തിൽ

RBFS ഒരു വഴിയിലൂടെ പോകുമ്പോൾ "പിന്നിൽ ഒരു നല്ല alternative ഉണ്ട്, അതിന്റെ f ഇത്ര" എന്ന് ഓർത്തുവെക്കും. മുന്നോട്ട് പോകുന്ന വഴി ആ alternative-നേക്കാൾ ചെലവേറിയതായാൽ തിരിച്ചു വരും, പക്ഷേ ഉപേക്ഷിച്ച വഴിയുടെ ഏറ്റവും നല്ല f ഓർമ്മയിൽ സൂക്ഷിക്കും. പിന്നീട് ആ വഴി വീണ്ടും മികച്ചതായാൽ അതിലേക്ക് തിരികെ പോകും. Memory കുറവ്, പക്ഷേ ഒരേ nodes വീണ്ടും വീണ്ടും ഉണ്ടാക്കേണ്ടിവരും.

RBFS(node, f-limit):

1. if node is a goal, return success
 2. generate successors; if none, return failure, ∞
 3. for each successor s: $f(s) = \max(g(s) + h(s), f(\text{node}))$
 4. repeat:
 - best = successor with the lowest f
 - if $f(\text{best}) > f\text{-limit}$, return failure, $f(\text{best})$
 - alternative = second-lowest f among the successors
 - result, $f(\text{best}) = \text{RBFS}(\text{best}, \min(f\text{-limit}, \text{alternative}))$
 - if result is success, return result
- Call RBFS(start, ∞)

EXAMPLE (TRACE)

Apply RBFS to the Kerala map (towns on the current path are skipped).

Call	Node	f-limit	Children (f)	Best	Alternative	What happens
1	KOC	∞	KDR(160), TSR(175)	KDR	175	recurse into KDR with limit $\min(\infty, 175)$
2	KDR	175	GVR(165), TSR(175)	GVR	175	recurse into GVR with limit $\min(175, 175)$
3	GVR	175	KZD(180), TSR(205)	KZD	205	best f 180 > limit 175: back up, GVR.f = 180
4	KDR	175	TSR(175), GVR(180)	TSR	180	recurse into TSR with limit $\min(175, 180)$
5	TSR	175	GVR(185), MLP(195), PKD(245)	GVR	195	best f 185 > limit 175: back up, TSR.f = 185
6	KDR	175	GVR(180), TSR(185)	GVR	185	best f 180 > limit 175: back up, KDR.f = 180
7	KOC	∞	TSR(175), KDR(180)	TSR	180	recurse into TSR with limit $\min(\infty, 180)$
8	TSR	180	GVR(185), MLP(195), KDR(240), PKD(245)	GVR	195	best f 185 > limit 180: back up, TSR.f = 185
9	KOC	∞	KDR(180), TSR(185)	KDR	185	recurse into KDR with limit $\min(\infty, 185)$
10	KDR	185	GVR(180), TSR(180)	GVR	180	recurse into GVR with limit $\min(185, 180)$

Call	Node	f-limit	Children (f)	Best	Alternative	What happens
11	GVR	180	KZD(180), TSR(205)	KZD	205	recurse into KZD with limit min(180, 205)
12	KZD	180	–	–	–	goal reached: stop

RBFS switches between the Kodungallur and Thrissur subtrees three times, each time remembering the backed-up value (KDR.f = 180, TSR.f = 185), and finally returns KOC → KDR → GVR → KZD, 180 km, the same optimal path as A*.

Advantages: optimal with an admissible h; more efficient than IDA* because it follows the next-best f-contour and saves the backed-up values, so fewer nodes are regenerated; linear memory. **Disadvantages:** still suffers from excessive node regeneration; cannot detect repeated states other than those on the current path, so IDA* and RBFS are poor on graphs; time complexity is hard to characterise.

Exam tip: The pseudocode box "Algorithm for recursive Best First Search" in the unit is empty in the printed text; learn the version above (it is the standard one from Russell and Norvig). Remember the key line $f(s) = \max(g(s) + h(s), f(\text{node}))$.

All search algorithms compared

Here are all the algorithms of the unit run on the same Kerala problem (Kochi to Kozhikode, optimal 180 km), together with their theoretical properties. The informed methods reach the goal with fewer selections, and only greedy search returns a worse route.

മലയാളത്തിൽ

ഒരേ problem-ൽ എല്ലാ algorithms-ഉം ഒരുമിച്ച് താരതമ്യം ചെയ്യുമ്പോൾ വ്യത്യാസം വ്യക്തമാകും. Greedy മാത്രം 205 km-ന്റെ വഴി തന്നെ; A* ഏറ്റവും കുറച്ച് selections കൊണ്ട് optimal വഴി കണ്ടെത്തി; IDA*, RBFS ഒരേ ഉത്തരം കുറഞ്ഞ memory-യിൽ, പക്ഷേ ആവർത്തിച്ചുള്ള ജോലിയോടെ തന്നെ.

Algorithm	Selects from OPEN	Complete	Optimal	Time	Space	Kerala result
BFS	shallowest (FIFO)	Yes	Equal costs only	$O(b^d)$	$O(b^d)$	180 km, 7 selections
DFS	deepest (LIFO)	No	No	$O(b^m)$	$O(bm)$	180 or 205 km (order-dependent)
DLS (l)	deepest, depth $\leq l$	If $l \geq d$	No	$O(b^l)$	$O(bl)$	$l = 2$: fails
IDDFS	DLS with $l = 0, 1, 2, \dots$	Yes	Equal costs only	$O(b^d)$	$O(bd)$	180 km, found at $L = 3$
UCS	least g	Yes	Yes	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	180 km, 7 selections
Bidirectional	two BFS frontiers	Yes	Equal costs only	$O(b^{d/2})$	$O(b^{d/2})$	180 km, meet at GVR
Greedy best-first	least h	No	No	$O(b^m)$	$O(b^m)$	205 km, 4 selections
A*	least g + h	Yes	Yes (admissible h)	exponential, $O(b^d)$	$O(b^d)$	180 km, 5 selections
IDA*	DFS within f-limit	Yes	Yes (admissible h)	up to $O(N^2)$ vs A*'s N	$O(bd)$	180 km, 4 iterations
RBFS	best child, with alternative limit	Yes	Yes (admissible h)	hard to characterise	$O(bd)$	180 km
AO*	leaf of best marked	Yes	Yes ($h' \leq h^*$)	exponential	whole explicit graph	(AND-OR problems)

Algorithm	Selects from OPEN	Complete	Optimal	Time	Space	Kerala result
	AND-OR subgraph					

Quick recap

- Search finds a sequence of actions from the start state to a goal state in a state space graph; OPEN holds nodes to be expanded, CLOSED holds nodes already expanded.
- Four criteria: completeness, optimality (admissibility), time and space, measured with b (branching factor), d (depth of shallowest goal) and m (maximum depth).
- Uninformed search uses no domain knowledge: BFS, DFS, DLS, IDDFS, UCS, bidirectional.
- BFS: OPEN is a queue; complete; optimal for equal step costs; $O(b^d)$ time and space.
- DFS: OPEN is a stack; $O(bm)$ space; $O(b^m)$ time; neither complete nor optimal.
- DLS cuts DFS at depth l ; IDDFS repeats DLS with $l = 0, 1, 2, \dots$, giving BFS's guarantees with DFS's $O(bd)$ memory.
- UCS expands the least $g(n)$; it is optimal and equals A^* with $h = 0$ (Dijkstra's algorithm).
- Bidirectional search meets in the middle: $O(b^{d/2})$, but needs a known goal and invertible operators.
- A heuristic $h(n)$ estimates the cost from n to the goal; straight-line distance is the classic example.
- Greedy best-first uses $f = h$: fast but neither optimal nor complete (Kerala: 205 km instead of 180).
- A^* uses $f = g + h$; with admissible h ($h \leq h^*$) it is complete and optimal; stop only when the goal is selected.
- A better-informed admissible heuristic (larger h , still $\leq h^*$) makes A^* expand fewer nodes.
- AO^* searches AND-OR graphs: AND nodes add their children's costs, OR nodes take the minimum, and costs are revised bottom-up.
- IDA^* (f -limit = least pruned f) and RBFS (backed-up f -values) give A^* 's answer in linear memory at the cost of re-expansion.

Check yourself

Q. 1. (Check Your Progress 1, Q1) A tree has root 1 with children 2, 3, 4; node 2 has child 6, node 3 has child 5, node 4 has child 7; node 6 has child 9 and node 5 has child 8, the goal. Apply BFS and DFS, showing OPEN and CLOSED.

BFS (queue):

Step	Removed from OPEN	OPEN (after step)	CLOSED	Remark
1	–	1	{ }	start
2	1	2, 3, 4	1	not goal; expand 1
3	2	3, 4, 6	1, 2	not goal; expand 2
4	3	4, 6, 5	1, 2, 3	not goal; expand 3
5	4	6, 5, 7	1, 2, 3, 4	not goal; expand 4
6	6	5, 7, 9	1, 2, 3, 4, 6	not goal; expand 6
7	5	7, 9, 8	1, 2, 3, 4, 6, 5	not goal; expand 5
8	7	9, 8	1, 2, 3, 4, 6, 5, 7	not goal; expand 7
9	9	8	1, 2, 3, 4, 6, 5, 7, 9	not goal; expand 9
10	8	{ }	1, 2, 3, 4, 6, 5, 7, 9, 8	goal test succeeds: stop

BFS removal order: 1, 2, 3, 4, 6, 5, 7, 9, 8.

DFS (stack, children inserted at the left end in left-to-right order):

Step	Removed from OPEN	OPEN (after step)	CLOSED	Remark
1	–	1	{}	start
2	1	2, 3, 4	1	not goal; expand 1
3	2	6, 3, 4	1, 2	not goal; expand 2
4	6	9, 3, 4	1, 2, 6	not goal; expand 6
5	9	3, 4	1, 2, 6, 9	not goal; expand 9
6	3	5, 4	1, 2, 6, 9, 3	not goal; expand 3
7	5	8, 4	1, 2, 6, 9, 3, 5	not goal; expand 5
8	8	4	1, 2, 6, 9, 3, 5, 8	goal test succeeds: stop

DFS removal order: 1, 2, 6, 9, 3, 5, 8.

The edges are read from the unit's figure, which is hard to read in the printed text. The unit's answer key removes 5 before 6 from the BFS queue (a FIFO queue must remove 6 first, since 6 entered before 5) and never generates node 9; the tables above follow the algorithm exactly.

Q. 2. (Q4) Compare BFS, DFS, IDDFS and bidirectional search with respect to time, space, optimality and completeness.

	BFS	DFS	IDDFS	Bidirectional
Time	$O(b^d)$	$O(b^m)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(bm)$	$O(bd)$	$O(b^{d/2})$
Optimal	Yes (equal costs)	No	Yes (equal costs)	Yes (uniform costs)
Complete	Yes	No	Yes	Yes

The answer key prints the DFS time as b^d ; it should be b^m .

Q. 3. Apply greedy best-first search (unit's Example 2): $S \rightarrow A(3), B(2); A \rightarrow C(4), D(1); B \rightarrow E(3), F(1); E \rightarrow H(5); F \rightarrow I(2), G(3)$. h : $S(13), A(12), B(4), C(7), D(3), E(8), F(2), H(4), I(9), G(0)$. G is the goal.

Step	Selected (least h)	OPEN sorted by h (after step)	CLOSED
1	–	S(13)	{ }
2	S(13)	B(4), A(12)	S
3	B(4)	F(2), E(8), A(12)	S, B
4	F(2)	G(0), E(8), I(9), A(12)	S, B, F
5	G(0)	E(8), I(9), A(12)	S, B, F, G

Path $S \rightarrow B \rightarrow F \rightarrow G$, cost $2 + 1 + 3 = 6$. This matches the unit's answer $S \rightarrow B \rightarrow F \rightarrow G$.

Q. 4. Apply A^* on the Kerala map from Palakkad (PKD) to Kozhikode (KZD). How many selections does A^* make, compared with uniform cost search?

Step	Selected (least f)	OPEN sorted by f (after step)	CLOSED	Remark
1	–	PKD(105)	{ }	start
2	PKD(105)	MLP(130), TSR(165)	PKD	expand
3	MLP(130)	KZD(140), TSR(165)	PKD, MLP	expand
4	KZD(140)	TSR(165)	PKD, MLP, KZD	KZD is the goal: stop, cost 140

Path $PKD \rightarrow MLP \rightarrow KZD$, cost $90 + 50 = 140$ km. A^* makes 3 selections (PKD, MLP, KZD). UCS from PKD selects $PKD \rightarrow TSR \rightarrow MLP \rightarrow GVR \rightarrow KDR \rightarrow KOC \rightarrow KZD$: 7 selections, because without h it also explores the southern towns. At step 2, TSR ($f = 65 + 100 = 165$) loses to MLP ($f = 90 + 40 = 130$).

Q. 5. (Q3) Differentiate between the A^* and AO^* algorithms.

A^* searches an OR graph and returns a single path (one alternative or another); it keeps OPEN and CLOSED and evaluates $f(n) = g(n) + h(n)$. AO^* searches an AND-OR graph and returns a solution graph that may require solving several sub-problems together (AND); it keeps one structure GRAPH, stores only $h'(n)$, and revises costs bottom-up: sum over an AND connector, minimum over OR choices. Both find a minimum-cost solution when the heuristic never overestimates; with only OR nodes, AO^* behaves like A^* .

Q. 6. (Q4) Root A ($h = 9$) has an AND arc to B and C and another AND arc to C and D. $h(B) = 3$, $h(C) = 4$, $h(D) = 5$. B is an OR node over E (5) and F (7); D is an AND node over G (4) and H (4). Every edge costs 1; C, E, F, G, H are terminal. Apply AO*.

Cycle 1, expand A: via (B, C) = $1 + 3 + 1 + 4 = 9$; via (C, D) = $1 + 4 + 1 + 5 = 11$. Mark (B, C); $f(A) = 9$.

Cycle 2, expand B (OR): $f(B-E) = 1 + 5 = 6$, $f(B-F) = 1 + 7 = 8$, so $f(B) = 6$ (marked E, B SOLVED). Revise A: via (B, C) = $1 + 6 + 1 + 4 = 12$; via (C, D) = 11 is now cheaper, so the mark **switches** to (C, D); $f(A) = 11$.

Cycle 3, expand D (AND): $f(D) = (1 + 4) + (1 + 4) = 10$, SOLVED. Revise A: via (C, D) = $1 + 4 + 1 + 10 = 16$; via (B, C) = 12. Mark switches back to (B, C); both B and C are SOLVED, so A is SOLVED with $f(A) = 12$.

The unit's answer (12) is correct, but it computes both paths in full without showing that AO* switches to (C, D) after cycle 2. In AO*, only the currently best-marked partial solution is expanded.

Q. 7. (Example 6) Matrices A1 (3×4), A2 (4×10), A3 (10×1) are to be multiplied. Use an AND-OR graph to find the minimum-cost order.

The root "A1A2A3" is an OR node with two choices; each choice is an AND node (both sub-products must be computed). Multiplying $p \times q$ by $q \times r$ costs pqr .

Choice 1, (A1A2)A3: A1A2 costs $3 \times 4 \times 10 = 120$ and is 3×10 ; then $(3 \times 10)(10 \times 1)$ costs $3 \times 10 \times 1 = 30$. Total 150.

Choice 2, A1(A2A3): A2A3 costs $4 \times 10 \times 1 = 40$ and is 4×1 ; then $(3 \times 4)(4 \times 1)$ costs $3 \times 4 \times 1 = 12$. Total 52.

The OR node takes the minimum: A1(A2A3) with **52** multiplications.

The unit's answer key solves a different problem (dimensions 3×5 , 5×6 , 6×10 , giving (A1A2)A3 = 270). For the dimensions in the question, the answer is 52.

Q. 8. (Q6) A* always finds an optimal solution if: (A) h' is always 0 (B) g is always 1 (C) h' never overestimates h (D) h' never underestimates h .

(C). Admissibility requires $h'(n) \leq h(n)$, i.e. the estimate never overestimates the true cost. $h' = 0$ in (A) is just one special case of (C), which turns A* into uniform cost search; (D) allows overestimates, which can make A* miss the optimum.

Q. 9. (Q7) A* uses $f^* = g + h^*$. To find a path with the fewest number of steps, we should set: (A) $g = 1$ (B) $g = 0$ (C) $h^* = 0$ (D) $h^* = 1$.

(A). Making every step cost 1 turns $g(n)$ into the number of steps taken so far, so minimising $g + h^*$ minimises the number of steps. Setting $g = 0$ would ignore the path length altogether (pure greedy search).

Q. 10. In the unit's A* Example 2, $h(D) = 6$. Is this heuristic admissible? Is the path found still optimal?

The true cost from D to G is 2 (arc $D \rightarrow G$), so $h(D) = 6 > h^*(D) = 2$: the heuristic overestimates at D and is **not admissible**. The other values are fine: $h^*(S) = 6$, $h^*(A) = 5$, $h^*(B) = 7$, $h^*(C) = 4$ are all $\geq h$. A* still returns $S \rightarrow A \rightarrow C \rightarrow G$ with cost 6, which is optimal, but only by luck; an overestimate could have hidden a cheaper path through D.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.